

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP VÀ PTNT
TRƯỜNG ĐẠI HỌC THỦY LỢI



VŨ VĂN TRUNG

**XÂY DỰNG MÔ HÌNH CHATBOT RAG HOÀN TOÀN
LOCAL**

ĐỒ ÁN TỐT NGHIỆP

HÀ NỘI, NĂM 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP VÀ PTNT
TRƯỜNG ĐẠI HỌC THỦY LỢI

VŨ VĂN TRUNG

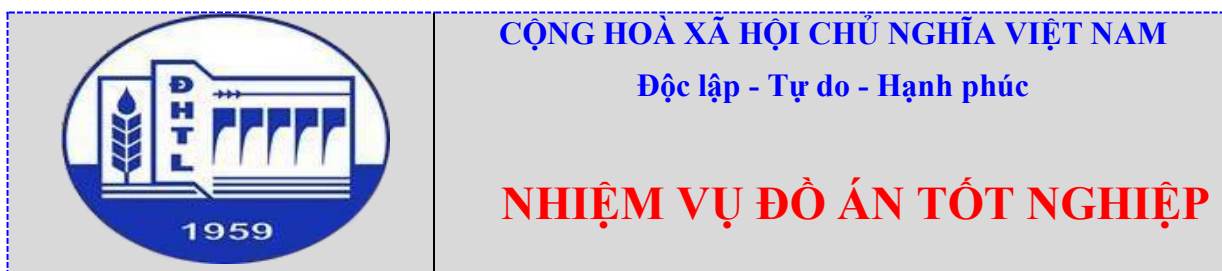
XÂY DỰNG MÔ HÌNH CHATBOT RAG HOÀN TOÀN LOCAL

Ngành : Công nghệ thông tin

Mã số: 7480201

NGƯỜI HƯỚNG DẪN 1. TS. Đinh Phú Hùng

HÀ NỘI, NĂM 2026



Họ tên sinh viên: Vũ Văn Trung Hệ đào tạo: Đại học chính quy

Lớp: 64CNTT2 Ngành: Công nghệ thông tin

Khoa: Công nghệ thông tin

1. TÊN ĐỀ TÀI: Xây dựng mô hình Chatbot RAG hoàn toàn local

2. CÁC TÀI LIỆU CƠ BẢN:

A Simple Guide to Retrieval Augmented Generation, A. Kimothi. Shelter Island, NY, USA: Manning Publications, 2025.

Speech and Language Processing, D. Jurafsky and J. H. Martin, 3rd ed., draft. Stanford, CA, USA: Stanford University, 2023.

Enterprise Guide to Building Intelligent AI Chatbots With RAG, NVIDIA Generative AI Team. Santa Clara, CA, USA: NVIDIA, 2024.

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS), 2020.

3. NỘI DUNG CÁC PHẦN THUYẾT MINH CẦN TÍNH TOÁN:

Nội dung cần thuyết minh	Tỉ lệ %
CHƯƠNG 1: GIỚI THIỆU	10%
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	25%
CHƯƠNG 3: PHÂN TÍCH HẠN CHẾ CỦA RAG TRUYỀN THỐNG VÀ ĐỊNH HƯỚNG CẢI TIẾN	15%
CHƯƠNG 4: TRIỂN KHAI XÂY DỰNG HỆ THỐNG HOÀN TOÀN LOCAL	40%
CHƯƠNG 5: ĐÁNH GIÁ, KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	10%

4. NỘI DUNG CÁC PHẦN THUYẾT MINH CẦN TÍNH TOÁN:

Giảng viên hướng dẫn toàn bộ dự án: TS. Đinh Phú Hùng

5. GIẢNG VIÊN HƯỚNG DẪN TỪNG PHẦN

Ngày tháng.....năm 2025

Trưởng Bộ môn

Giáo viên hướng dẫn chính

Nhiệm vụ Đồ án tốt nghiệp đã được Hội đồng thi tốt nghiệp của Khoa thông qua

Ngày tháng năm 2025

Chủ tịch Hội đồng

Sinh viên đã hoàn thành và nộp bản Đồ án tốt nghiệp cho Hội đồng thi

Ngày tháng năm 2025

Sinh viên làm Đồ án tốt nghiệp



TRƯỜNG ĐẠI HỌC THỦY LỢI
KHOA CÔNG NGHỆ THÔNG TIN

BẢN TÓM TẮT ĐỀ CƯƠNG ĐỒ ÁN TỐT NGHIỆP

Tên đề tài: *Xây dựng mô hình Chatbot RAG hoàn toàn local*

Sinh viên thực hiện: Vũ Văn Trung

Mã sinh viên: 2251061907

Số điện thoại: 0342387314

Địa chỉ email: 2251061907@e.tlu.edu.vn

Lớp: 64CNTT2

Giáo viên hướng dẫn: TS. Đinh Phú Hùng

1. GIỚI THIỆU & TÍNH CẤP THIẾT

1.1. Bối cảnh

Trong bối cảnh công nghệ AI phát triển nhanh, nhiều doanh nghiệp tại Việt Nam triển khai chatbot qua API của các nhà cung cấp như OpenAI, Google hay Microsoft. Tuy nhiên, mô hình này bộc lộ nhiều hạn chế như chi phí API cao, phụ thuộc máy chủ bên ngoài, rủi ro downtime, độ trễ mạng và vấn đề bảo mật dữ liệu nội bộ.

Với các dữ liệu nội bộ chỉ riêng nhân viên trong công ty được biết hay dữ liệu từ lĩnh vực nhạy cảm như tài chính, ngân hàng, bảo hiểm, giáo dục, hành chính công, việc sử dụng mô hình AI qua mạng Internet thường không đáp ứng yêu cầu bảo mật và chi phí dài hạn.

1.2. Tính cấp thiết

Đề tài hướng tới xây dựng một mô hình Chatbot RAG (Retrieval-Augmented Generation) chạy 100% local, nghĩa là toàn bộ các bước từ tiền xử lý tài liệu, embedding, indexing, truy xuất và sinh câu trả lời đều chạy trên hạ tầng nội bộ không phụ thuộc

API bên thứ ba. Như OpenAI dù có tùy chọn chia sẻ dữ liệu nhưng theo cuộc thảo luận trên Reddit vẫn có ý kiến cho rằng OpenAI vẫn sử dụng dữ liệu của người dùng để tối ưu model, làm dữ liệu train cho model ¹.

Mô hình này giúp doanh nghiệp:

- Tiết kiệm chi phí vận hành dài hạn (so với API trả phí theo token).
- Bảo vệ dữ liệu nội bộ tuyệt đối.
- Giảm độ trễ phản hồi.
- Có khả năng hoạt động cả khi không có Internet.

Ngoài ra, đề tài tập trung xử lý các tài liệu PDF có cấu trúc phức tạp bao gồm văn bản, bảng biểu, hình ảnh, công thức toán điều mà phần lớn các pipeline RAG hiện có vẫn còn hạn chế.

2. CÁC MỤC TIÊU CHÍNH

2.1. Mục tiêu tổng quát

- Xây dựng một hệ thống Chatbot RAG hoạt động hoàn toàn local, tối ưu chi phí vận hành và đảm bảo bảo mật dữ liệu.

2.2. Mục tiêu cụ thể

- Tích hợp mô hình ngôn ngữ mã nguồn mở (DeepSeek-R1 Distill, Gemma 3...) để thử nghiệm trên GPU tầm trung.
- Thiết lập pipeline RAG: tiền xử lý tài liệu → chunking → embedding → retrieval → generation.
- Triển khai thử nghiệm chatbot nội bộ trên dữ liệu thực tế.

¹ OpenAI, “ChatGPT trained to lie to you about using your data to train itself”, https://www.reddit.com/r/OpenAI/comments/1hew9ta/chatgpt_trained_to_lie_to_you_about_using_your/, 2025

- So sánh hiệu suất (tốc độ, độ chính xác, chi phí) giữa mô hình local và API thương mại (GPT, Gemini).
- Đề xuất khả năng mở rộng mô hình khi triển khai trên phần cứng mạnh hơn.

2.3. Phạm vi

- Không bao gồm huấn luyện lại model từ đầu → sử dụng mô hình pre-trained, inference local.
- Tập trung xử lý dữ liệu tiếng Việt, định dạng PDF phức tạp (văn bản, bảng, hình ảnh, công thức).
- Đánh giá trên máy có GPU RTX 3080 - 10GB VRAM. Trên Reddit cũng có 1 cuộc thảo luận về số VRAM/RAM cho model DeepSeek-R1 Distill 14B và khả thi với RTX 3080 ².

3: PHƯƠNG PHÁP NGHIÊN CỨU

3.1. Kiến trúc hệ thống

- Sử dụng mô hình DeepSeek-R1 Distill (8B/14B) và Gemma 3 ... làm backbone.
- Framework, thư viện: Ollama, vLLM, LMStudio cho inference local.
- Pipeline xử lý:
 1. Extract & Chunking: sử dụng MinerU/MagicPDF để tách văn bản, bảng, hình ảnh.
 2. Embedding: chọn mô hình embedding tiếng Việt (VD: BGE, Instructor, M3E).
 3. Vector Store: sử dụng Milvus hoặc Qdrant.

² LocalLLaMA, “Can I run DeepSeek R1 Distill Qwen 14B on 12GB of vram or less??”, https://www.reddit.com/r/LocalLLaMA/comments/1i5wahr/can_i_run_deepseek_r1_distill_qwen_14b_on_12gb_of/, 2025

4. Reranking & Generation: RAG pipeline kết hợp context retrieval với model inference.

3.2. Đánh giá hiệu suất

- Độ chính xác trả lời: F1-score, BLEU.
- Tốc độ phản hồi: thời gian trung bình/truy vấn.
- Chi phí vận hành: so sánh với API theo token.
- Mức tiêu thụ tài nguyên phần cứng (RAM/VRAM).

3.3. Kỹ thuật bổ trợ

- Tích hợp TTS/STT (Coqui TTS, VITS) cho tiếng Việt.
- Multi-conversation & session context management.
- Module quản trị tri thức nội bộ (KB Management).

4. KẾT QUẢ DỰ KIẾN

1. Xây dựng thành công một mô hình Chatbot RAG có thể hoạt động độc lập, không cần kết nối Internet với khả năng xử lý được tài liệu pdf với văn bản, bảng, hình ảnh, công thức ở mức độ thử nghiệm.
2. Khả năng trả lời chính xác với độ trễ < 5 giây trên RTX 3080 10GB VRAM.
3. So sánh hiệu năng giữa sử dụng mô hình local DeepSeek-R1 Distill 8B/14B, Gemma 3 ... và mô hình API.
4. Thử nghiệm nội bộ để khai thác dữ liệu riêng.

GÁY BÌA ĐỒ ÁN TỐT NGHIỆP, KHÓA LUẬN TỐT NGHIỆP

HỌ VÀ TÊN

ĐỒ ÁN/KL TỐT NGHIỆP

HÀ NỘI, NĂM

LỜI CAM ĐOAN

Em xin cam đoan đây là Đồ án tốt nghiệp của bản thân em. Các kết quả trong Đồ án tốt nghiệp này là trung thực, và không sao chép từ bất kỳ một nguồn nào và dưới bất kỳ hình thức nào. Việc tham khảo các nguồn tài liệu (nếu có) đã được thực hiện trích dẫn và ghi nguồn tài liệu tham khảo đúng quy định.

Tác giả ĐATN/KLTN

Chữ ký

Vũ Văn Trung

LỜI CẢM ƠN

Trước tiên, em xin bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu trường Đại học Thủy Lợi, các thầy cô khoa Công nghệ Thông tin đã tạo điều kiện thuận lợi và hỗ trợ em trong suốt quá trình học tập và thực hiện đồ án tốt nghiệp này.

Đặc biệt, em xin gửi lời cảm ơn chân thành đến TS Đinh Phú Hùng, người đã tận tình hướng dẫn, động viên và cung cấp những kiến thức quý báu, giúp em hoàn thành đồ án tốt nghiệp. Sự chỉ dẫn tận tâm của thầy không chỉ giúp em giải quyết những khó khăn trong quá trình nghiên cứu mà còn định hướng cho em những phương pháp học tập và làm việc hiệu quả.

Em cũng xin gửi lời cảm ơn đến gia đình và bạn bè, những người đã luôn ở bên, ủng hộ và khích lệ em trong suốt thời gian học tập và thực hiện đồ án.

Cuối cùng, em xin cảm ơn tất cả những ai đã giúp đỡ, đóng góp ý kiến và chia sẻ kinh nghiệm để em có thể hoàn thành đồ án này. Mặc dù đã cố gắng hết sức, nhưng chắc chắn đồ án còn nhiều thiếu sót, em rất mong nhận được sự góp ý của các thầy cô và các bạn để hoàn thiện hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

DANH MỤC CÁC HÌNH ẢNH.....	vi
DANH MỤC BẢNG BIỂU	vii
DANH MỤC CÁC TỪ VIẾT TẮT VÀ GIẢI THÍCH CÁC THUẬT NGỮ	viii
CHƯƠNG 1 GIỚI THIỆU	1
1.1 Bài toán ứng dụng Chatbot RAG hoàn toàn local trong thực tiễn	1
1.1.1 . Giới hạn của việc sử dụng LLM thông qua API và Web	1
1.1.2 . Vấn đề bảo mật và tích hợp trong doanh nghiệp	2
1.2 Yêu cầu xây dựng hệ thống Chatbot RAG hoàn toàn local	4
1.2.1 . Yêu cầu chức năng	4
1.2.2 . Yêu cầu phi chức năng	5
1.3 Mục tiêu và phạm vi của đề tài	5
1.3.1 . Mục tiêu	5
1.3.2 . Phạm vi	6
1.4 Đóng góp chính của đề tài	7
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT	8
2.1 Mô hình ngôn ngữ lớn và giới hạn trong bài toán hỏi đáp tri thức	8
2.1.1 . Năng lực sinh văn bản và suy luận của LLM và VLM	8
2.1.2 . Giới hạn cốt lõi của mô hình thuần tham số	10
2.2 Bộ nhớ tham số và bộ nhớ phi tham số	11
2.3 Retrieval-Augmented Generation	12
2.3.1 Khái niệm	12
2.3.2 Kiến trúc cơ bản của RAG	13
2.4 Tiền xử lý dữ liệu	15
2.4.1 Văn bản đầu vào	15
2.4.2 Phân mảnh dữ liệu	16
2.4.3 Nhóm phương pháp chunking không có cấu trúc	17
2.4.4 Nhóm chunking theo ngữ nghĩa	18
2.4.5 Nhóm chunking nâng cao	19
2.4.6 Giới hạn độ dài ngữ cảnh	19
2.4.7 So sánh các chiến lược chunking	20
2.5 Lưu trữ và biểu diễn tri thức	21
2.5.1 Biểu diễn tri thức bằng biểu diễn vector dày	21
2.5.2 Biểu diễn tri thức bằng biểu diễn vector thưa	23
2.5.3 Vector database	23
2.5.4 So sánh biểu diễn dày và biểu diễn thưa	24
2.6 Truy xuất thông tin	25

2.6.1 Các chiến lược truy xuất	26
2.6.2 Đơn vị truy xuất và mối liên hệ với chunking	27
2.6.3 Nhiễu ngữ cảnh	27
2.6.4 Đánh đổi giữa recall và precision trong truy xuất.....	28
2.7 Sinh câu trả lời	29
2.7.1 Vai trò của thành phần sinh câu trả lời trong kiến trúc RAG	29
2.7.2 Chiến lược xây dựng prompt	29
2.7.3 Chèn ngữ cảnh	30
2.7.4 Các chiến lược prompt	31
2.7.5 Hallucination trong giai đoạn sinh	32
2.8 Đánh giá hệ thống RAG	33
CHƯƠNG 3 PHÂN TÍCH HẠN CHẾ RAG TRUYỀN THỐNG VÀ ĐỊNH HƯỚNG CẢI TIẾN	36
3.1 Hạn chế của RAG Text-Only truyền thống.....	36
3.2 Vấn đề nhiễu ngữ cảnh trong truy xuất một giai đoạn	37
3.3 Thách thức khi xây dựng pipeline hoàn toàn Local	39
3.3.1 Hạn chế phần cứng.....	39
3.3.2 Thách thức dữ liệu đa phương thức	40
3.3.3 Định hướng giải pháp	41
CHƯƠNG 4 TRIỂN KHAI XÂY DỰNG HỆ THỐNG RAG HOÀN TOÀN LOCAL	41
4.1 Tổng quan kiến trúc hệ thống.....	41
4.1.1 Mục tiêu thiết kế kiến trúc	41
4.1.2 Sơ đồ kiến trúc tổng quát	43
4.2 Dữ liệu và phạm vi thí nghiệm.....	45
4.2.1 Nhóm dữ liệu nội bộ doanh nghiệp.....	45
4.2.2 Dữ liệu thử nghiệm	46
4.3 Cấu hình phần cứng.....	46
4.4 Tiền xử lý dữ liệu	47
4.4.1 Chuyển đổi PDF thành markdown với Marker.....	48
4.4.2 Xây dựng tập biểu diễn tài liệu	50
4.4.3 Phân tích theo cấu trúc	50
4.5 Lưu trữ dữ liệu lại	51
4.5.1 Vai trò từng kho	52
4.5.2 Schema MySQL.....	53
4.5.3 Cấu trúc kho lưu trữ Qdrant.....	54
4.6 Truy xuất dữ liệu	54
4.6.1 Giai đoạn thứ nhất.....	55

4.6.2 Giai đoạn thứ hai.....	55
4.6.3 Chiến lược hợp nhất truy xuất dày và truy xuất thưa.....	55
4.7 Sinh câu trả lời	56
4.8 Đánh giá hệ thống	59
4.8.1 Bộ chỉ số đánh giá.....	59
4.8.2 Sử dụng VLM làm giám khảo	59
4.8.3 Prompt cho nhiều loại dữ liệu	60
CHƯƠNG 5 ĐÁNH GIÁ, KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	64
5.1 Tổng kết kết quả đạt được	64
5.1.1 Xây dựng hệ thống RAG hoàn toàn nội bộ và khả năng triển khai trong môi trường doanh nghiệp.....	64
5.1.2 Hiệu quả với tài liệu có cấu trúc	64
5.1.3 Khả năng hoạt động trên phần cứng được sử dụng	65
5.2 Đánh giá kết quả thực nghiệm.....	65
5.2.1 Kết quả chất lượng end-to-end.....	65
5.2.2 Kết quả chất lượng retrieval	66
5.2.3 Precision trung bình nhưng Overall vẫn cao.....	67
5.3 Hạn chế hiện tại.....	68
5.3.1 Hạn chế đối với hình ảnh	69
5.3.2 Bảng lớn và cấu trúc phức tạp	69
5.3.3 Công thức toán học	69
5.3.4 Giới hạn của đánh giá nội bộ	69
5.4 Hướng phát triển	70
5.4.1 Mở rộng kênh tương tác giọng nói	70
5.4.2 Hỗ trợ thêm định dạng tài liệu đầu vào.....	70
5.4.3 Tối ưu cho các loại dữ liệu.....	70
5.4.4 Hoàn thiện yêu cầu đáp ứng triển khai doanh nghiệp.....	71
5.5 Kết luận chung	71
TÀI LIỆU THAM KHẢO	72

DANH MỤC CÁC HÌNH ẢNH

Hình 1.1 Tổng quan bài toán và mục tiêu dự án.....	7
Hình 2.1 Kiến trúc transformer/LLM trong QA.....	8
Hình 2.2 Kiến trúc tổng quát của RAG baseline [20]	14
Hình 3.1 Failure point của text-only one-stage RAG.....	36
Hình 4.1 Minh hoạ kiến trúc tổng quát của hệ thống	43
Hình 4.2 Minh hoạ luồng tiền xử lý dữ liệu.	48
Hình 4.3 Hiệu năng của marker so với các framework khác.....	49
Hình 4.4 Schema MySQL sử dụng trong dự án	53
Hình 4.5 Two-stage retrieval pipeline	54
Hình 4.6 Output với câu hỏi hình ảnh	61
Hình 4.7 Output với câu hỏi công thức	62
Hình 4.8 Output với câu hỏi công thức	63
Hình 5.1 Kết quả chất lượng end-to-end (overall).....	65
Hình 5.2 Recall và Precision theo từng dataset	66
Hình 5.3 Quan hệ trade-off giữa Recall, Precision và Overall với Recall (x), Overall (y) và Precision (màu).....	67
Hình 5.4 Heatmap tổng hợp các chỉ số đánh giá theo dataset	68

DANH MỤC BẢNG BIỂU

Bảng 2.1 Tổng hợp các chiến lược chunking phổ biến theo các trục đánh giá có ý nghĩa trực tiếp đối với hiệu quả của RAG baseline, bao gồm độ mịn (granularity), mức độ mạch lạc của ngữ cảnh (context coherence), khả năng cải thiện recall trong truy xuất, mức độ nhiễu ngữ cảnh và chi phí lập chỉ mục.	20
Bảng 2.2 So sánh dense embedding và sparse representation trong hệ thống RAG..	25
Bảng 2.3 Tóm tắt chiến lược prompt stuffing và prompt grounding.....	32
Bảng 2.4 Tóm tắt các phương pháp đánh giá phổ biến.	35
Bảng 4.1 Mục tiêu thiết kế và thành phần đáp ứng	43
Bảng 4.2 Cấu hình phần cứng được sử dụng cho dự án.	47
Bảng 4.3 Tóm tắt vai trò từng kho lưu trữ.....	53
Bảng 4.4 Cấu trúc prompt.....	58

DANH MỤC CÁC TỪ VIẾT TẮT VÀ GIẢI THÍCH CÁC THUẬT NGỮ

ANN - Approximate Nearest Neighbor
API - Application programming interface
BM25 - Best Matching 25 (mô hình truy xuất dựa trên xác suất trong Information Retrieval)
Cutoff - mốc thời gian mà tri thức huấn luyện của mô hình dừng lại
Chunk - đoạn tài liệu đã được phân mảnh
Chunking - Phân mảnh tài liệu cho mục đích truy xuất
Context Noise - Nhiễu ngữ cảnh trong quá trình retrieval
Context Length - Giới hạn ngữ cảnh của mô hình ngôn ngữ
CRM - Customer Relationship Management
Downstream - Những bước, tác vụ hoặc ứng dụng diễn ra sau một giai đoạn xử lý trước đó, và phụ thuộc trực tiếp vào kết quả của giai đoạn đó
DPR - Dense Passage Retrieval
EM - Exact Match tức chỉ số đánh giá trong QA
Embedding - biểu diễn vector liên tục, có số chiều cố định, được học từ dữ liệu nhằm mã hóa thông tin ngữ nghĩa của đối tượng và cho phép đo lường mức độ tương đồng trong không gian vector
F1-score - Chỉ số đánh giá kết hợp Precision và Recall
GPU - Graphics Processing Unit
Hallucination - Hiện tượng sinh thông tin không có căn cứ
HNSW - Hierarchical Navigable Small World (cấu trúc chỉ mục vector)
Hybrid Retrieval - Kết hợp dense và sparse retrieval
IR - Information Retrieval
Ingest - nạp dữ liệu vào hệ thống để lưu trữ, xử lý hoặc lập chỉ mục
KB - Knowledge Base
Knowledge Grounding - Neo câu trả lời vào nguồn tri thức
LaTeX - hệ thống soạn thảo tài liệu dựa trên ngôn ngữ đánh dấu để biểu diễn công thức toán học và tài liệu khoa học một cách chính xác và nhất quán
Lazy loading encoder - chiến lược trì hoãn việc khởi tạo hoặc nạp encoder cho đến khi thực sự cần sử dụng, thay vì nạp sẵn ngay khi hệ thống khởi động
LLM - Large Language Model
Markdown - ngôn ngữ đánh dấu nhẹ dùng để biểu diễn cấu trúc văn bản bằng ký hiệu đơn giản dễ đọc và dễ xử lý bằng máy
MRAG - Multimodal Retrieval-Augmented Generation
Modality - một loại hình dữ liệu hoặc kênh thông tin riêng biệt, phản ánh cách thức biểu diễn và tiếp nhận thông tin trong hệ thống.
NLP - Natural Language Processing
OCR - Optical Character Recognition
PDF - Portable Document Format
Pipeline - quy trình
Prompt - đầu vào dạng ngôn ngữ tự nhiên hoặc có cấu trúc, được cung cấp cho mô hình ngôn ngữ nhằm định hướng hành vi sinh nội dung của mô hình
Proposition-based Chunking - Chunking theo mệnh đề tri thức
QA - Question Answering
RAG - Retrieval-Augmented Generation: Kiến trúc kết hợp truy xuất tri thức và sinh ngôn ngữ
RTX - NVIDIA GeForce RTX (dòng GPU)
SaaS - Software as a Service

Semantic Chunking - Chunking dựa trên đơn vị ngữ nghĩa

Seq2Seq - Sequence-to-Sequence model

Source Traceability - Khả năng truy vết nguồn thông tin

Token - đơn vị biểu diễn cơ bản của chuỗi đầu vào, được tạo ra bởi bộ tách token (tokenizer) và được sử dụng trực tiếp trong quá trình xử lý và suy luận của mô hình ngôn ngữ.

Two-stage Retrieval - Truy xuất hai giai đoạn (document-level → chunk-level)

TTS - Text-to-Speech

VLM - Vision-Language Model

VRAM - Video Random Access Memory

CHƯƠNG 1 GIỚI THIỆU

1.1 Bài toán ứng dụng Chatbot RAG hoàn toàn local trong thực tiễn

Trong bối cảnh công nghệ trí tuệ nhân tạo (AI) phát triển mạnh mẽ, các mô hình ngôn ngữ lớn (Large Language Models - LLM) như ChatGPT hay Gemini đã trở thành công cụ hỗ trợ đắc lực cho con người trong việc xử lý thông tin và trả lời câu hỏi. Tuy nhiên, việc áp dụng các mô hình này dưới dạng API hoặc web-based gặp phải nhiều hạn chế, đặc biệt trong môi trường doanh nghiệp nơi dữ liệu nhạy cảm và nhu cầu tích hợp sâu là yếu tố then chốt. Đề tài này tập trung vào việc xây dựng một hệ thống chatbot dựa trên Retrieval-Augmented Generation (RAG) hoàn toàn local, nhằm khắc phục những vấn đề trên bằng cách đảm bảo tính tự chủ, bảo mật và hiệu suất cao.

Hệ thống RAG kết hợp giữa khả năng truy xuất thông tin từ cơ sở kiến thức và khả năng sinh văn bản của LLM, cho phép chatbot trả lời dựa trên dữ liệu thực tế thay vì chỉ dựa vào kiến thức được huấn luyện trước. Trong đề tài này, pipeline của hệ thống bao gồm việc parse tài liệu PDF thành markdown để giữ cấu trúc, chunking multimodal để xử lý riêng văn bản, bảng biểu, hình ảnh, công thức, lưu trữ lại với Qdrant và MySQL, cùng với truy xuất hai giai đoạn, giai đoạn một là chọn tài liệu qua tập vector biểu diễn tài liệu và giai đoạn hai là truy xuất trong phạm vi tài liệu đã chọn. Điều này không chỉ nâng cao độ chính xác mà còn giảm thiểu ảo giác.

1.1.1 . Giới hạn của việc sử dụng LLM thông qua API và Web

Các mô hình LLM dạng API hoặc web như ChatGPT của OpenAI và Gemini của Google đã cách mạng hóa cách con người tương tác với AI, cho phép xử lý ngôn ngữ tự nhiên một cách mượt mà. Tuy nhiên, chúng tồn tại nhiều hạn chế khiến chúng không phù hợp cho các ứng dụng thực tiễn đòi hỏi độ tin cậy cao.

Thứ nhất, tri thức giới hạn: Các mô hình này chỉ dựa vào dữ liệu huấn luyện đến một thời điểm cutoff nhất định, ví dụ, GPT-4 cutoff vào tháng 10/2023 [1], dù có cập nhật nhưng không bao quát dữ liệu mới nhất. Chúng không thể truy cập dữ liệu riêng tư hoặc cập nhật thời gian thực của doanh nghiệp, dẫn đến câu trả lời thiếu chính xác hoặc lỗi thời. Chẳng hạn, khi hỏi về một chính sách nội bộ mới của công ty, LLM web chỉ có thể

dựa vào kiến thức chung, dẫn đến sai sót. Theo nghiên cứu của Liu et al. [2], các LLM thường gặp khó khăn khi xử lý context dài, và việc thiếu dữ liệu cụ thể làm tăng nguy cơ này.

Thứ hai, rủi ro ảo giác, ảo giác là hiện tượng mô hình ngôn ngữ lớn (LLM) sinh thông tin trông có vẻ hợp lý nhưng không được hỗ trợ bởi dữ liệu đầu vào hoặc tri thức được xác thực. Trong thực tế, khi người dùng đặt câu hỏi về một công thức toán học, quy trình kỹ thuật hoặc nội dung chuyên môn mà không cung cấp đủ dữ liệu, tài liệu gốc, các mô hình ngôn ngữ lớn sẽ trả lời trôi chảy dựa trên kiến thức chung nhưng có thể sai lệch về mặt nội dung, giảm độ tin cậy trong các ứng dụng đòi hỏi tính chính xác cao. Nghiên cứu từ Ji et al., các mô hình ngôn ngữ lớn có xu hướng ưu tiên tri thức được sử dụng để huấn luyện hơn, cụ thể bài báo nói “models that favor generating output with their parametric knowledge instead of the information from the input source can result in the hallucination of excess information in the output” [3]. Do đó, trong các hệ thống LLM dạng API/web, khi không có thông tin hoặc tài liệu nguồn cụ thể, nguy cơ ảo giác trở nên đặc biệt nghiêm trọng.

Thứ ba, vấn đề hộp đen: Khi tải tệp lên web (như ChatGPT Plus hỗ trợ tải tệp tin PDF), người dùng không kiểm soát được quy trình xử lý. Tệp tin được phân tích như thế nào? Dữ liệu được lưu trữ ra sao? Có bị sử dụng để huấn luyện thêm không? Điều này tạo ra rủi ro về quyền riêng tư và thiếu minh bạch.

Tóm lại, các hạn chế này làm giảm hiệu quả của LLM API/web trong môi trường thực tiễn, nơi cần độ chính xác cao và tích hợp dữ liệu riêng.

1.1.2 . Vấn đề bảo mật và tích hợp trong doanh nghiệp

Trong doanh nghiệp, bảo mật dữ liệu là yếu tố hàng đầu, theo Luật An ninh Mạng tại Việt Nam (Luật số 24/2018/QH14) và Nghị định 53/2022/NĐ-CP, các doanh nghiệp cung cấp hoặc sử dụng dịch vụ trên không gian mạng Việt Nam có trách nhiệm lưu trữ dữ liệu quan trọng, dữ liệu cá nhân và dữ liệu người dùng tại lãnh thổ Việt Nam. Việc sử dụng LLM dạng API/web mang lại rủi ro lớn về bảo mật.

Đầu tiên, **dữ liệu nhạy cảm**: Các tài liệu nội bộ như hợp đồng, quyết định kinh doanh hoặc dữ liệu tài chính không thể tùy ý tải lên các hệ thống máy chủ bên ngoài doanh

nghiệp. Các nghiên cứu học thuật đã chỉ ra rằng các hệ thống trí tuệ nhân tạo tiềm ẩn rủi ro nghiêm trọng về quyền riêng tư khi xử lý dữ liệu nhạy cảm. Cụ thể, Billiris et al. khẳng định rằng “Artificial Intelligence (AI) systems introduce unprecedented privacy challenges as they process increasingly sensitive data” [4], đồng thời ghi nhận nhiều trường hợp mô hình AI vô tình làm lộ thông tin nhạy cảm từ dữ liệu huấn luyện. Bên cạnh đó, các nghiên cứu về môi trường điện toán đám mây cho thấy rủi ro bảo mật gia tăng khi xử lý dữ liệu nhạy cảm trên hạ tầng cloud. Theo Soveizi et al., “security is a critical issue affecting the wide adoption of cloud technologies, especially for workflows that are mostly dealing with sensitive data and tasks” [5], và việc đưa quy trình xử lý lên cloud có thể làm hệ thống mất quyền kiểm soát dữ liệu, dẫn đến nguy cơ bị tấn công hoặc rò rỉ thông tin.

Thứ hai, phụ thuộc vào hạ tầng mạng và độ ổn định dịch vụ. Các hệ thống LLM dạng API/web yêu cầu kết nối Internet liên tục tới máy chủ đặt ở nước ngoài. Trong bối cảnh ở Việt Nam, sự cố đứt cáp quang biển mới nhất trong tháng 10/2025 theo tin tức trên trang web của Viettel cho thấy rủi ro thực tế và gián đoạn kết nối [6]. Theo tin tức khoa học công nghệ trên VnExpress [7], cũng bàn luận về sự phụ thuộc lớn của các hệ thống với hệ thống cáp quang toàn cầu hiện nay. Điều này làm giảm tính sẵn sàng, ổn định của hệ thống dựa trên API bên thứ ba.

Thứ ba, hạn chế trong tích hợp sâu với hệ thống nội bộ. Mặc dù các nền tảng như ChatGPT hay Gemini hỗ trợ tải lên tệp PDF và hỏi đáp trên giao diện web, chức năng này thực chất được triển khai thông qua một pipeline riêng do nhà cung cấp kiểm soát, không phải khả năng mặc định của API. Các API VLM, LLM hiện nay chủ yếu hỗ trợ mô hình tương tác dạng text-in/text-out, không cung cấp cơ chế ingest tài liệu, truy xuất tri thức theo tài liệu, hay liên kết trực tiếp với cơ sở dữ liệu nội bộ như ERP hoặc CRM, Yizheng Huang và cộng sự Jimmy Huang cũng khẳng định RAG là một phương pháp bổ sung, không phải tính năng gốc của LLM hay VLM [8]. Do đó, doanh nghiệp không thể tích hợp trực tiếp các giao diện SaaS này vào quy trình nghiệp vụ của mình. Ngược lại, pipeline RAG trong đề tài cho phép xây dựng luồng ingest, truy xuất, sinh câu trả lời hoàn toàn nội bộ, hỗ trợ truy xuất có kiểm soát theo tài liệu và dễ dàng tích hợp với các hệ thống hiện có.

1.2 Yêu cầu xây dựng hệ thống Chatbot RAG hoàn toàn local

Để xây dựng một hệ thống chatbot RAG hoạt động hoàn toàn local, cần xác định rõ các yêu cầu chức năng và phi chức năng của hệ thống. Các yêu cầu này được xây dựng dựa trên pipeline RAG thực tế, bao gồm các bước ingestion tài liệu PDF, chunking đa góc nhìn, lưu trữ lại, truy xuất hai giai đoạn (two-stage retrieval) và sinh câu trả lời bằng mô hình ngôn ngữ lớn chạy local. Việc xác định rõ yêu cầu ngay từ đầu giúp đảm bảo hệ thống đáp ứng được cả tính đúng đắn về mặt kỹ thuật lẫn khả năng triển khai trong bối cảnh thực tế.

1.2.1 . Yêu cầu chức năng

Về mặt chức năng, hệ thống cần hỗ trợ đầy đủ quy trình của một pipeline RAG cơ bản, đồng thời mở rộng để xử lý dữ liệu đa phương thức thay vì chỉ giới hạn ở văn bản thuần. Trước hết, hệ thống phải cho phép ingest tài liệu PDF một cách tự động, chuyển đổi tài liệu đầu vào thành dạng biểu diễn trung gian phù hợp cho việc truy xuất tri thức. Quá trình này bao gồm việc chuyển đổi PDF sang markdown, với mục tiêu bảo toàn cấu trúc của tài liệu như tiêu đề, đoạn văn và các thành phần nội dung quan trọng khác.

Bên cạnh văn bản, hệ thống cần hỗ trợ xử lý đa góc nhìn cho các loại dữ liệu thường xuất hiện trong tài liệu thực tế, bao gồm bảng biểu, hình ảnh và công thức toán học. Các bảng và hình ảnh cần được tách riêng để phục vụ truy xuất theo ngữ cảnh phù hợp, trong khi các công thức toán học phải được chuẩn hóa về dạng LaTeX nhằm đảm bảo khả năng biểu diễn và suy luận chính xác ở các bước downstream. Khả năng xử lý đa phương thức này là yêu cầu cốt lõi để hệ thống có thể áp dụng cho các tài liệu hành chính và bài báo khoa học có cấu trúc phức tạp.

Về truy xuất thông tin, hệ thống cần triển khai cơ chế truy xuất hai giai đoạn nhằm giảm nhiễu và tăng độ chính xác của ngữ cảnh được đưa vào mô hình sinh. Giai đoạn đầu tiên thực hiện tìm kiếm ở mức tài liệu, kết hợp giữa tìm kiếm ngữ nghĩa dựa trên biểu diễn vector ngữ nghĩa dạng dense và tìm kiếm theo từ khóa dựa trên BM25, sau đó áp dụng bước tái xếp hạng kết quả bằng mô hình BGE để chọn ra tập tài liệu phù hợp nhất. Giai đoạn thứ hai giới hạn phạm vi truy xuất trong các tài liệu đã được chọn, sử dụng kết hợp Qdrant và MySQL để truy xuất các chunk đa góc nhìn tương ứng. Cách tiếp cận này

giúp hệ thống tránh việc đưa quá nhiều thông tin không liên quan vào dữ liệu đầu vào phục vụ sinh văn bản (prompt), từ đó cải thiện chất lượng câu trả lời.

Cuối cùng, hệ thống chatbot cần hỗ trợ tương tác hội thoại bằng tiếng Việt, đảm bảo câu trả lời mạch lạc, dễ hiểu và có khả năng trích dẫn rõ ràng các nguồn thông tin đã được sử dụng trong quá trình trả lời. Việc cung cấp trích dẫn không chỉ giúp tăng độ tin cậy của hệ thống mà còn hỗ trợ người dùng kiểm tra và xác thực thông tin khi cần thiết.

1.2.2 . Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo khả năng vận hành ổn định trong môi trường local. Trước hết, yêu cầu về quyền riêng tư được đặt lên hàng đầu: toàn bộ hệ thống phải hoạt động hoàn toàn offline, không phụ thuộc vào kết nối Internet. Tất cả các thành phần, bao gồm bộ mã hóa cho văn bản, hình ảnh và công thức toán học, cũng như mô hình ngôn ngữ lớn, đều phải được triển khai và chạy local nhằm loại bỏ nguy cơ rò rỉ dữ liệu ra bên ngoài.

Về hiệu năng, hệ thống cần được tối ưu để vận hành trên máy tính cá nhân với cấu hình phần cứng giới hạn, cụ thể là GPU RTX 3080 với 10GB VRAM và 48GB RAM. Các kỹ thuật như lazy loading encoder, quản lý bộ nhớ thông qua QdrantManager và sử dụng các mô hình đã được lượng tử hoá (quantization) cho LLM và VLM cần được áp dụng để giảm tiêu thụ tài nguyên, đồng thời vẫn đảm bảo độ trễ chấp nhận được trong quá trình truy vấn và sinh câu trả lời.

Ngoài ra, hệ thống cần đảm bảo khả năng thêm dữ liệu, cho phép bổ sung tài liệu mới vào cơ sở tri thức mà không cần huấn luyện lại mô hình. Việc sử dụng cơ sở dữ liệu vector với cơ chế on-disk index như Qdrant giúp hệ thống có thể mở rộng quy mô dữ liệu lưu trữ và truy xuất, đồng thời vẫn phù hợp với hạ tầng local và tài nguyên phần cứng hạn chế.

1.3 Mục tiêu và phạm vi của đề tài

1.3.1 . Mục tiêu

Mục tiêu chính của đề tài là xây dựng một hệ thống Chatbot dựa trên RAG hoạt động hoàn toàn local, hướng tới xử lý dữ liệu đa phương thức và khắc phục các hạn chế của các pipeline RAG cơ bản thuần văn bản. Đề tài tập trung thiết kế và triển khai pipeline

RAG mã nguồn mở có khả năng vận hành độc lập, không phụ thuộc vào dịch vụ cloud hay API bên thứ ba, nhằm đáp ứng các yêu cầu về bảo mật dữ liệu, tính tự chủ và khả năng triển khai trong môi trường doanh nghiệp.

Trên phương diện kỹ thuật, đề tài hướng tới việc giải quyết bài toán mất mát cấu trúc tài liệu, đây một hạn chế phổ biến của các hệ thống RAG truyền thống. Thông qua quy trình chuyển đổi PDF sang markdown được bảo toàn cấu trúc, kết hợp với chiến lược chunking đa góc nhìn (multimodal chunking) để xử lý riêng biệt các loại nội dung như văn bản, bảng biểu, hình ảnh và công thức toán học. Bên cạnh đó, đề tài thiết kế cơ chế truy xuất hai giai đoạn nhằm giảm nhiễu cảnh trong quá trình truy xuất, qua đó cải thiện độ chính xác và tính trung thực (faithfulness) của câu trả lời được sinh ra.

Ngoài việc xây dựng hệ thống, đề tài còn đặt mục tiêu đánh giá toàn diện hiệu quả của pipeline RAG thông qua công cụ đánh giá tự động, qua đó cung cấp cái nhìn định lượng về chất lượng truy xuất và sinh câu trả lời của hệ thống trong các kịch bản thực tế.

1.3.2 . Phạm vi

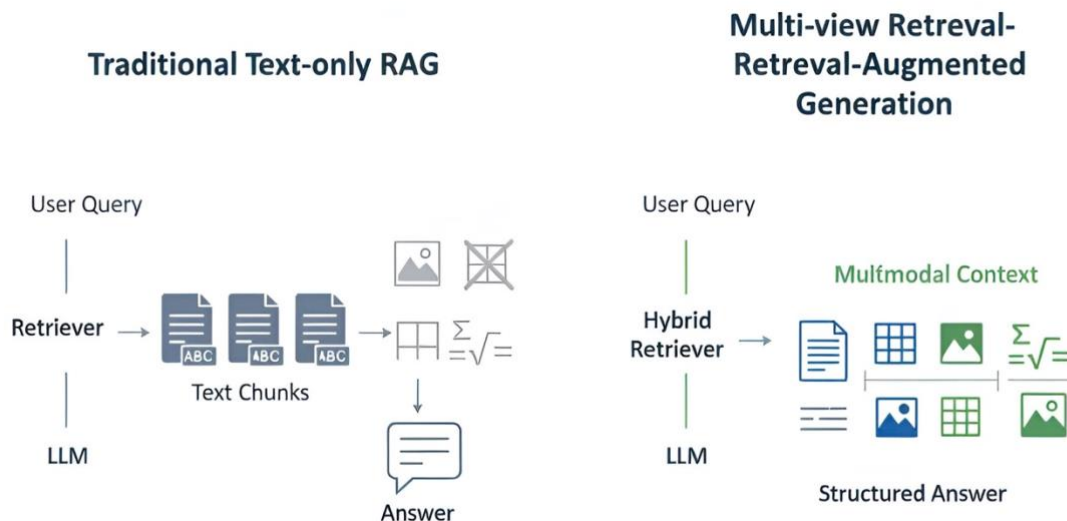
Phạm vi của đề tài được giới hạn ở các kịch bản ứng dụng có tính thực tiễn cao trong môi trường doanh nghiệp. Về dữ liệu, hệ thống được thử nghiệm chủ yếu trên các tài liệu hành chính nội bộ (bao gồm văn bản và bảng biểu) và các bài báo khoa học có cấu trúc phức tạp (chứa hình ảnh, bảng kết quả và công thức toán học) được sử dụng với mục đích kiểm thử tính năng vì tài liệu doanh nghiệp còn hạn chế thể loại công thức và hình ảnh, với ngôn ngữ xử lý chính là tiếng Việt. Điều này phản ánh đúng nhu cầu ứng dụng RAG trong bối cảnh Việt Nam, nơi dữ liệu tiếng Việt và tài liệu PDF có cấu trúc đa dạng chiếm tỷ lệ lớn.

Về hạ tầng triển khai, đề tài tập trung vào môi trường máy tính cá nhân với GPU tầm trung là RTX 3080, sử dụng các mô hình ngôn ngữ và mô hình ngôn ngữ thị giác chạy local thông qua các công cụ như LMStudio. Đề tài không xem xét bài toán huấn luyện mô hình từ đầu, mà tập trung vào việc khai thác và tích hợp các mô hình tiền huấn luyện trong một pipeline RAG hoàn chỉnh và có khả năng triển khai thực tế.

1.4 Đóng góp chính của đề tài

Đề tài đóng góp một số điểm mới cả về mặt kiến trúc hệ thống lẫn thực nghiệm triển khai. Trước hết, đề tài đề xuất kiến trúc RAG hoàn toàn local, kết hợp giữa chunking đa góc nhìn và lưu trữ lai nhằm nâng cao khả năng biểu diễn và truy xuất tri thức từ các tài liệu có cấu trúc phức tạp. Kiến trúc này cho phép xử lý đồng thời nhiều loại nội dung như văn bản, hình ảnh, công thức, bảng biểu trong cùng một pipeline, vượt ra ngoài giả định văn bản phẳng của các RAG cơ bản.

Bên cạnh đó, đề tài xây dựng một quy trình phân tích cú pháp và tiền xử lý thông minh, dựa trên việc chuyển đổi PDF sang markdown có bảo toàn cấu trúc, kết hợp với các chiến lược chunking tùy biến theo loại dữ liệu. Điều này giúp giảm đáng kể hiện tượng mất mát thông tin cấu trúc và cải thiện chất lượng truy xuất ở các bước downstream.



Hình 1.1 Tổng quan bài toán và mục tiêu dự án

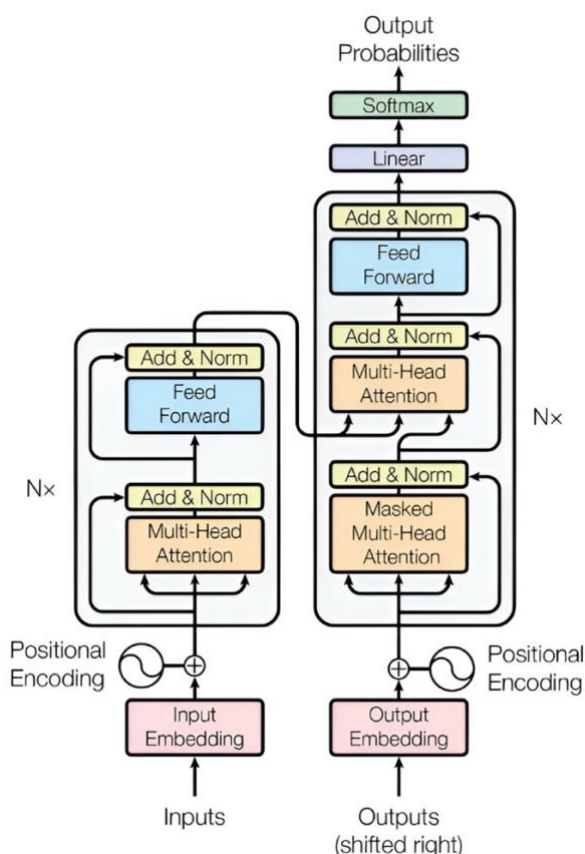
Ngoài ra, đề tài áp dụng cơ chế truy xuất hai giai đoạn theo phạm vi tài liệu được xác định, kết hợp giữa cơ sở dữ liệu vector và cơ sở dữ liệu quan hệ, nhằm giảm nhiễu cảnh và hạn chế hiện tượng ảo giác trong quá trình sinh câu trả lời. Cuối cùng, đề tài phát triển một công cụ đánh giá tự động cho hệ thống RAG đa phương thức, so sánh hiệu quả của pipeline với các hệ thống LLM dạng web trong các kịch bản hỏi đáp dựa trên tài liệu thực tế.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1 Mô hình ngôn ngữ lớn và giới hạn trong bài toán hỏi đáp tri thức

Sự phát triển nhanh chóng của các mô hình ngôn ngữ lớn dựa trên kiến trúc Transformer đã mang lại những bước tiến quan trọng cho xử lý ngôn ngữ tự nhiên, cho phép sinh văn bản mạch lạc, duy trì ngữ cảnh dài và hỗ trợ suy luận ở mức độ nhất định trong nhiều tác vụ hỏi đáp và tương tác người với máy. Tuy nhiên, trong các bài toán hỏi đáp dựa trên tri thức đòi hỏi độ chính xác, khả năng kiểm chứng và khai thác dữ liệu miền riêng, các mô hình thuần tham số bộc lộ những hạn chế mang tính bản chất, qua đó thúc đẩy sự ra đời của các kiến trúc kết hợp như RAG.

2.1.1 . Năng lực sinh văn bản và suy luận của LLM và VLM



Hình 2.1 Kiến trúc transformer/LLM trong QA³

³ LLMs and Transformers from Scratch: the Decoder, Luís Roque and Rafael Nardi, Jan. 10, 2024. [Online]. Available: <https://towardsdatascience.com/llms-and-transformers-from-scratch-the-decoder-d533008629c5/> [Accessed: Dec. 30, 2025].

Các mô hình ngôn ngữ lớn được xây dựng chủ yếu dựa trên kiến trúc Transformer, trong đó cơ chế self-attention cho phép mô hình học được các mối quan hệ ngữ nghĩa dài hạn trong chuỗi văn bản [9]. Thông qua quá trình tiền huấn luyện trên tập dữ liệu lớn gồm sách, bài báo, trang web và các nguồn văn bản đa dạng, LLMs học được các mẫu ngôn ngữ, cấu trúc cú pháp và mối liên hệ ngữ nghĩa ở nhiều mức độ trừu tượng khác nhau. Kết quả là các mô hình này thể hiện khả năng sinh văn bản trôi chảy, trả lời câu hỏi tự nhiên và thực hiện suy luận ngôn ngữ ở mức độ nhất định mà không cần thiết kế thủ công các luật hay đặc trưng.

Trong bài toán hỏi đáp, LLM có thể trực tiếp nhận câu hỏi đầu vào và sinh ra câu trả lời dựa trên tri thức đã được mã hóa trong tham số mô hình. Nghiên cứu của Brown et al [10] cho thấy LLM có thể đạt kết quả ấn tượng trên nhiều đánh giá hỏi đáp tổng quát, đặc biệt là các bài toán không yêu cầu truy xuất tri thức cụ thể hoặc chỉ dựa trên kiến thức phổ thông. Nhờ khả năng khái quát hóa mạnh, LLM có thể xử lý các câu hỏi được diễn đạt linh hoạt, không phụ thuộc vào cấu trúc câu cố định.

Tuy nhiên, năng lực của LLM không chỉ dừng lại ở việc ghi nhớ tri thức từ dữ liệu huấn luyện. Theo cứu của Wei et al [11], LLM có thể suy luận ở một mức độ nhất định, chẳng hạn như suy luận chuỗi (chain-of-thought), suy luận logic đơn giản hoặc kết hợp nhiều mảnh thông tin để tạo ra câu trả lời hợp lý. Khả năng này khiến LLM trở thành nền tảng hấp dẫn cho các hệ thống hỏi đáp tri thức, nơi câu trả lời không chỉ là việc truy xuất thông tin đơn thuần mà còn đòi hỏi diễn giải và tổng hợp.

Song song với sự phát triển của LLM, các mô hình đa phương thức, đặc biệt là Vision-Language Models (VLMs), đã mở rộng phạm vi xử lý của hệ thống AI sang các dữ liệu không chỉ giới hạn ở văn bản. VLM kết hợp biểu diễn thị giác và ngôn ngữ, cho phép mô hình hiểu và sinh câu trả lời dựa trên cả hình ảnh và văn bản [12]. Trong ngữ cảnh hỏi đáp, VLM có thể xử lý các câu hỏi liên quan đến hình ảnh, sơ đồ, biểu đồ hoặc các tài liệu chứa thành phần trực quan, điều mà LLM thuần văn bản không thể thực hiện.

Khả năng của VLM đặc biệt quan trọng trong các bài toán thực tế, nơi tri thức không chỉ tồn tại dưới dạng văn bản mà còn được biểu diễn thông qua hình ảnh, bảng biểu và công thức. Theo nghiên cứu của Radford et al [13], việc kết hợp thông tin thị giác giúp

cải thiện đáng kể khả năng hiểu ngữ cảnh và trả lời chính xác trong các tác vụ liên quan đến tài liệu đa phương thức. Tuy nhiên, mặc dù mở rộng phạm vi dữ liệu đầu vào, VLM về bản chất vẫn chia sẻ cùng một đặc điểm cốt lõi với LLM: tri thức được mã hóa trong tham số mô hình thông qua quá trình huấn luyện.

Tóm lại, cả LLM và VLM đều sở hữu năng lực sinh và suy luận mạnh mẽ, cho phép chúng trở thành công cụ trung tâm trong các hệ thống hỏi đáp hiện đại. Dù khác nhau về loại dữ liệu đầu vào, hai lớp mô hình này đều dựa trên tri thức tham số (parametric knowledge) được học từ dữ liệu huấn luyện, và chính đặc điểm này dẫn đến những giới hạn quan trọng khi áp dụng vào các bài toán hỏi đáp tri thức chuyên sâu.

2.1.2 . Giới hạn cốt lõi của mô hình thuần tham số

Mặc dù đạt được nhiều thành tựu ấn tượng, các mô hình LLM và VLM thuần tham số vẫn tồn tại những giới hạn mang tính bản chất trong các bài toán hỏi đáp tri thức. Giới hạn đầu tiên và dễ nhận thấy nhất là vấn đề cutoff tri thức. Tri thức của mô hình bị cố định tại thời điểm kết thúc quá trình huấn luyện, khiến LLM không thể tự động cập nhật các thông tin mới phát sinh sau đó [14]. Trong các lĩnh vực thay đổi nhanh như khoa học, công nghệ hoặc quy định pháp lý, hạn chế này làm giảm đáng kể tính hữu dụng của các hệ thống hỏi-đáp dựa thuần trên LLM.

Bên cạnh đó, việc cập nhật tri thức cho LLM thông qua fine-tuning hoặc huấn luyện lại là một quá trình tốn kém về tài nguyên tính toán và dữ liệu. Nghiên cứu của Wang et al [15] chỉ ra rằng việc fine-tune các mô hình lớn không chỉ đòi hỏi chi phí cao mà còn tiềm ẩn nguy cơ làm suy giảm khả năng tổng quát hóa của mô hình nếu không được thực hiện cẩn thận. Do đó, việc sử dụng LLM như một kho tri thức tĩnh không phải là giải pháp bền vững cho các hệ thống hỏi đáp yêu cầu cập nhật liên tục.

Một hạn chế nghiêm trọng khác của mô hình thuần tham số là hiện tượng ảo giác. Khi thiếu thông tin chính xác hoặc gặp các câu hỏi vượt ngoài tri thức đã học, LLM có xu hướng sinh ra các câu trả lời nghe có vẻ hợp lý nhưng không có căn cứ hoặc sai lệch so với thực tế [3]. Trong các môi trường yêu cầu độ tin cậy cao, chẳng hạn như doanh nghiệp, y tế hoặc pháp lý, hiện tượng này có thể dẫn đến hậu quả nghiêm trọng. Nghiên cứu của Maynez et al [16] đã chỉ ra rằng hallucination không phải là lỗi ngẫu nhiên mà

là hệ quả trực tiếp của việc mô hình phải suy đoán dựa trên tri thức tham số khi thiếu nguồn tham chiếu bên ngoài.

Ngoài ra, các câu trả lời do LLM sinh ra thường thiếu khả năng kiểm chứng và truy vết nguồn. Trong mô hình thuần tham số, không tồn tại mối liên kết rõ ràng giữa câu trả lời và các tài liệu hoặc bằng chứng cụ thể đã được sử dụng. Điều này gây khó khăn cho việc đánh giá độ tin cậy của kết quả, đặc biệt trong các kịch bản doanh nghiệp hoặc học thuật, nơi yêu cầu minh bạch nguồn thông tin là bắt buộc [17].

Mặc dù VLM có thể mở rộng khả năng xử lý dữ liệu sang các miền đa phương thức, chúng cũng không tránh khỏi các giới hạn nêu trên. Tri thức thị giác và ngôn ngữ vẫn được mã hóa trong tham số mô hình, và việc cập nhật hoặc kiểm chứng thông tin vẫn gặp những khó khăn tương tự như LLM thuần văn bản [18]. Do đó, việc bổ sung khả năng đa phương thức không giải quyết được bài toán cốt lõi về cập nhật tri thức, kiểm chứng và giảm ảo giác.

Nghiên cứu của Mohsin et al [19] gần đây cũng cho thấy rằng việc đơn thuần mở rộng kích thước mô hình hoặc tăng lượng dữ liệu huấn luyện không đủ để khắc phục các giới hạn này. Thay vào đó, cần một hướng tiếp cận kiến trúc cho phép kết hợp năng lực suy luận của mô hình ngôn ngữ với các nguồn tri thức bên ngoài, có thể cập nhật và truy xuất động tại thời điểm suy luận.

Những phân tích trên cho thấy rằng, mặc dù LLM và VLM đóng vai trò trung tâm trong các hệ thống hỏi đáp hiện đại, chúng không thể đáp ứng đầy đủ các yêu cầu của bài toán hỏi đáp tri thức khi hoạt động như một mô hình thuần tham số. Chính khoảng trống này đã thúc đẩy sự ra đời của các kiến trúc kết hợp, tiêu biểu là RAG, nhằm bổ sung tri thức ngoài mô hình và nâng cao độ tin cậy của hệ thống hỏi-đáp.

2.2 Bộ nhớ tham số và bộ nhớ phi tham số

Trong bối cảnh các hệ thống hỏi đáp dựa trên mô hình ngôn ngữ lớn, tri thức mà mô hình sử dụng để sinh câu trả lời có thể được phân chia thành hai thành phần khái niệm cốt lõi là tri thức tham số và tri thức phi tham số. Sự phân biệt này đóng vai trò nền tảng trong việc lý giải vì sao các mô hình ngôn ngữ thuần túy gặp hạn chế khi xử lý các tác

vụ yêu cầu tri thức chuyên biệt, cũng như động lực ra đời của các kiến trúc kết hợp như RAG.

Tri thức tham số đề cập đến tri thức được mã hóa trực tiếp trong các tham số (weights) của mô hình ngôn ngữ lớn thông qua quá trình huấn luyện trên tập dữ liệu quy mô lớn. Loại bộ nhớ này cho phép mô hình học được các quy luật ngôn ngữ, biểu diễn ngữ nghĩa và khả năng suy luận tổng quát, từ đó sinh ra văn bản trôi chảy và mạch lạc trong nhiều ngữ cảnh khác nhau. Tuy nhiên, tri thức được lưu trữ trong tri thức tham số mang tính tĩnh, bị giới hạn bởi thời điểm huấn luyện và khó cập nhật khi xuất hiện dữ liệu mới. Việc bổ sung hoặc điều chỉnh tri thức thường đòi hỏi quá trình fine-tuning tốn kém về chi phí tính toán và dữ liệu. Hơn nữa, do không được ràng buộc với nguồn tri thức cụ thể tại thời điểm suy luận, các mô hình chỉ dựa trên tri thức tham số có xu hướng xuất hiện hiện tượng ảo giác, đặc biệt trong các tác vụ yêu cầu độ chính xác cao và khả năng kiểm chứng thông tin [3].

Ngược lại, tri thức phi tham số đại diện cho tri thức được lưu trữ bên ngoài mô hình, dưới dạng tập văn bản, cơ sở dữ liệu có cấu trúc, hoặc các chỉ mục truy xuất như vector index. Tri thức phi tham số không bị cố định trong tham số mô hình và có thể được cập nhật linh hoạt mà không cần huấn luyện lại LLM. Nhờ cơ chế truy xuất theo truy vấn, tri thức phi tham số cho phép cung cấp các bằng chứng (evidence) cụ thể để hỗ trợ quá trình sinh câu trả lời, qua đó nâng cao tính minh bạch và độ tin cậy của hệ thống, đặc biệt trong các kịch bản sử dụng miền dữ liệu cụ thể hoặc môi trường doanh nghiệp. Tuy nhiên, hiệu quả của việc sử dụng tri thức phi tham số phụ thuộc mạnh vào chất lượng tiền xử lý dữ liệu, chiến lược chunking và cơ chế truy xuất, trong các tập dữ liệu văn bản lớn hoặc chứa nhiều nhiễu, việc truy xuất không chính xác có thể làm suy giảm chất lượng ngữ cảnh được đưa vào mô hình sinh [17].

2.3 Retrieval-Augmented Generation

2.3.1 Khái niệm

Retrieval-Augmented Generation (RAG) là một hướng tiếp cận kiến trúc kết hợp giữa mô hình sinh ngôn ngữ và cơ chế truy xuất tri thức bên ngoài, nhằm tăng cường khả năng trả lời các câu hỏi yêu cầu kiến thức chuyên biệt hoặc cập nhật. Thay vì chỉ dựa

vào tri thức đã được mã hóa trong tham số của mô hình ngôn ngữ lớn, RAG cho phép hệ thống truy xuất động thông tin liên quan từ một kho tri thức ngoài tại thời điểm suy luận, sau đó sử dụng thông tin này để sinh câu trả lời.

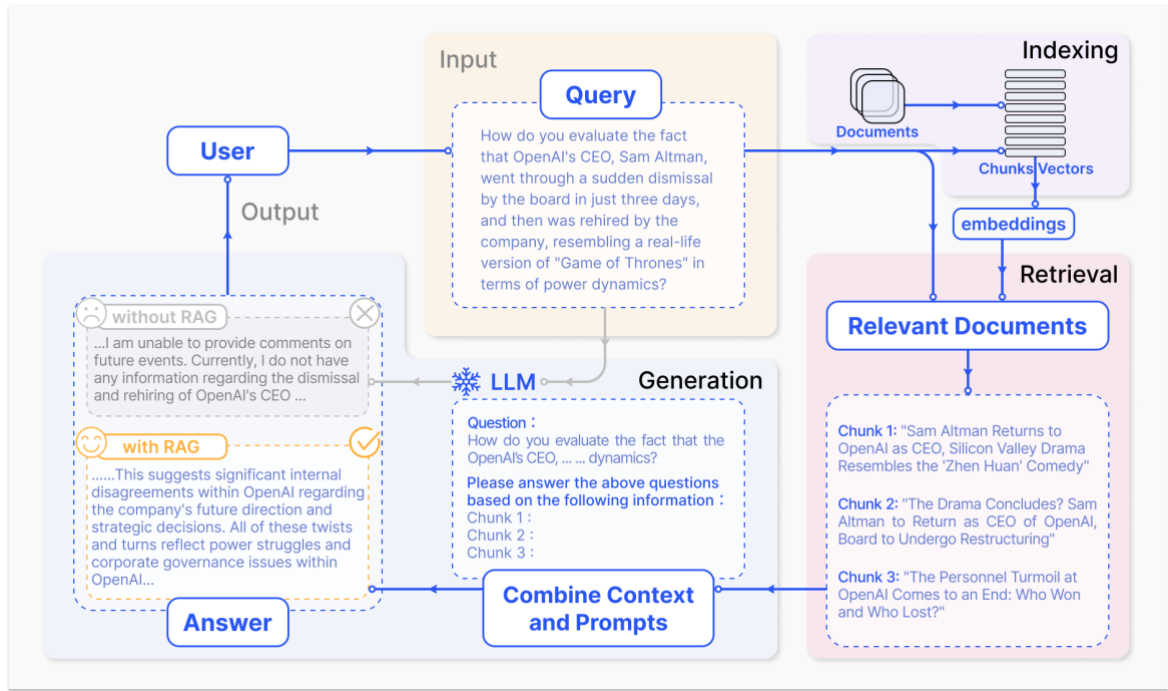
Khái niệm RAG được giới thiệu một cách có hệ thống trong công trình nền tảng của Lewis et al. [17], trong đó các tác giả chứng minh rằng việc tích hợp truy xuất tri thức vào quá trình sinh giúp cải thiện đáng kể hiệu quả trên các tác vụ hỏi đáp dựa tri thức so với các mô hình sinh thuần túy. RAG xem mô hình sinh là một thành phần suy luận, được cung cấp ngữ cảnh phù hợp thông qua truy xuất từ một tập dữ liệu ngoài.

Mục tiêu cốt lõi của RAG là khắc phục các hạn chế của LLM thuần tham số, bao gồm giới hạn tri thức theo thời điểm huấn luyện, hiện tượng ảo giác và thiếu khả năng gắn kết câu trả lời với nguồn tri thức cụ thể. Nhờ cơ chế truy xuất, RAG cho phép hệ thống sử dụng tri thức mới hoặc tri thức miền riêng mà không cần huấn luyện lại mô hình ngôn ngữ, đồng thời tạo tiền đề cho việc kiểm chứng và giải thích câu trả lời trong các ứng dụng đòi hỏi độ tin cậy cao.

Theo khảo sát của Huang et al [8] về RAG đều thống nhất rằng đây là một trong những hướng tiếp cận chủ đạo để triển khai LLM trong thực tiễn, đặc biệt trong các kịch bản sử dụng dữ liệu doanh nghiệp hoặc dữ liệu chuyên ngành. Trong bối cảnh đó, RAG không chỉ được xem như một kỹ thuật cải thiện hiệu năng, mà còn là một mô hình tư duy mới trong thiết kế hệ thống hỏi đáp dựa tri thức.

2.3.2 Kiến trúc cơ bản của RAG

Về mặt kiến trúc, RAG cơ bản được xây dựng dựa trên một pipeline tuần tự gồm bốn thành phần chính: truy vấn đầu vào (query), mô-đun truy xuất (retriever), ngữ cảnh được truy xuất (context) và mô hình sinh câu trả lời (generator). Pipeline này thể hiện cách thức cơ bản mà RAG kết hợp tri thức phi tham số với khả năng sinh ngôn ngữ của LLM.



Hình 2.2 Kiến trúc tổng quát của RAG baseline [20]

Trong pipeline này, người dùng cung cấp một truy vấn đầu vào dưới dạng câu hỏi hoặc mệnh đề ngôn ngữ tự nhiên. Truy vấn này được chuyển tới mô-đun truy xuất, nơi hệ thống thực hiện tìm kiếm các đơn vị tri thức liên quan trong kho dữ liệu ngoài, thường là một tập văn bản được biểu diễn vector ngữ nghĩa dạng dense, sparse hoặc kết hợp cả hai. Kết quả của bước truy xuất là một tập ngữ cảnh, thường bao gồm các đoạn văn bản được đánh giá là liên quan nhất đến truy vấn.

Ngữ cảnh truy xuất sau đó được đưa vào mô hình sinh cùng với truy vấn ban đầu để tạo thành đầu vào cho quá trình sinh câu trả lời. Cách tiếp cận này cho phép mô hình sinh tận dụng tri thức ngoài một cách trực tiếp trong quá trình suy luận, thay vì phải nhớ tri thức đó trong tham số mô hình.

Quá trình sinh trong RAG baseline có thể được biểu diễn bởi công thức (2-1).

$$p(y | x) = \sum_{z \in Z} p(z | x) p(y | x, z) \quad (2-1)$$

Trong đó x là truy vấn đầu vào, z là các đơn vị tri thức được truy xuất từ kho dữ liệu ngoài, và y là câu trả lời được sinh. Công thức này nhấn mạnh vai trò trung tâm của truy

xuất trong việc điều kiện hóa phân phối sinh câu trả lời, và cho thấy RAG về bản chất là sự kết hợp xác suất giữa mô hình truy xuất và mô hình sinh [17].

Kiến trúc cơ sở này đã trở thành nền tảng cho nhiều biến thể RAG sau này, từ các hệ thống hỏi đáp đơn giản cho đến các pipeline phức tạp hơn trong môi trường doanh nghiệp. Các nghiên cứu tiếp theo như nghiên cứu của Huang et al. [8] hay Gao et al. [20] chủ yếu mở rộng hoặc tinh chỉnh từng thành phần trong pipeline, chẳng hạn như cải tiến mô hình truy xuất, chiến lược chọn ngữ cảnh, hoặc cách kết hợp ngữ cảnh vào mô hình sinh. Tuy nhiên, về mặt khái niệm, pipeline cơ bản vẫn là cấu trúc cốt lõi của hầu hết các hệ thống RAG hiện đại.

2.4 Tiền xử lý dữ liệu

Trong kiến trúc RAG, tiền xử lý dữ liệu giữ vai trò then chốt trong việc đảm bảo hiệu quả của toàn bộ pipeline, từ lưu trữ tri thức, truy xuất thông tin cho đến sinh câu trả lời. Khác với các hệ thống hỏi đáp truyền thống dựa hoàn toàn vào tri thức tham số của mô hình ngôn ngữ, RAG yêu cầu xây dựng một kho tri thức ngoài có khả năng truy xuất động tại thời điểm suy luận. Điều này đòi hỏi dữ liệu đầu vào phải được chuyển đổi từ các tài liệu thô sang dạng biểu diễn có cấu trúc, phù hợp cho việc lập chỉ mục và tìm kiếm.

Về bản chất, tiền xử lý dữ liệu trong RAG không chỉ là một bước chuẩn bị kỹ thuật, mà là một bài toán thiết kế mang tính lý thuyết. Quá trình này liên quan trực tiếp đến cách phân mảnh tri thức, cách bảo toàn ngữ nghĩa trong từng đơn vị dữ liệu, cũng như cách cân bằng giữa độ chính xác của truy xuất và độ đầy đủ của ngữ cảnh được đưa vào mô hình sinh. Trong số các thao tác tiền xử lý, việc phân chia tài liệu thành các đơn vị nhỏ hơn được xem là thành phần trung tâm, có ảnh hưởng sâu rộng đến chất lượng của hệ thống RAG.

2.4.1 Văn bản đầu vào

Trong hệ thống RAG, văn bản đầu vào không được hiểu đơn thuần là chuỗi ký tự dùng làm dữ liệu đầu vào phục vụ sinh văn bản cho mô hình ngôn ngữ, mà là tập hợp các tài liệu nguồn đóng vai trò như kho tri thức ngoài. Không giống kiến trúc LLM thuần tham

số, RAG xem các tài liệu đầu vào như một bộ nhớ động, có thể cập nhật, mở rộng hoặc thay thế mà không cần huấn luyện lại mô hình sinh [17].

Từ góc độ hình thức, mỗi tài liệu đầu vào thường có độ dài vượt xa context length mà mô hình ngôn ngữ có thể xử lý trong một lần suy luận. Một báo cáo kỹ thuật hoặc bài báo khoa học có thể chứa hàng nghìn đến hàng chục nghìn token, trong khi hầu hết các LLM hiện đại chỉ cho phép xử lý một số lượng token hữu hạn. Do đó, việc đưa toàn bộ tài liệu để làm dữ liệu đầu vào phục vụ sinh văn bản là không khả thi, cả về mặt kỹ thuật lẫn hiệu quả suy luận.

Về mặt lý thuyết, tập tài liệu đầu vào trong RAG cần được lập chỉ mục trước khi truy xuất. Quá trình lập chỉ mục thường bao gồm các bước như chuẩn hóa văn bản, phân mảnh dữ liệu và ánh xạ các đơn vị văn bản sang không gian biểu diễn [20]. Trong các bước này, phân mảnh dữ liệu đóng vai trò trung tâm, quyết định trực tiếp đến chất lượng biểu diễn tri thức và khả năng truy xuất chính xác của hệ thống.

2.4.2 Phân mảnh dữ liệu

Phân mảnh dữ liệu được định nghĩa là quá trình chia một tài liệu đầu vào thành tập hợp các đơn vị con có kích thước và cấu trúc phù hợp cho việc lưu trữ và truy xuất. Quá trình phân mảnh dữ liệu có thể được mô hình hóa bằng công thức (2-2):

$$D_{\text{chunking}} = \{c_1, c_2, \dots, c_n\} \quad (2-2)$$

trong đó D là tài liệu gốc và c_i là các chunk sau khi phân mảnh. Mỗi chunk đại diện cho một phần tri thức cục bộ của tài liệu và được xem là đơn vị cơ bản trong giai đoạn lập chỉ mục.

Nhu cầu phân mảnh dữ liệu xuất phát từ hai nguyên nhân cốt lõi. Thứ nhất, giới hạn context length của mô hình ngôn ngữ khiến việc xử lý toàn bộ tài liệu trong một lần suy luận là bất khả thi. Thứ hai, ngay cả trong trường hợp có thể đưa một lượng lớn văn bản vào ngữ cảnh, việc cung cấp toàn bộ tài liệu cho mô hình sinh thường làm tăng nhiễu và làm giảm khả năng tập trung vào thông tin liên quan, dẫn đến suy giảm chất lượng câu trả lời. Các nghiên cứu về hành vi của LLM trong ngữ cảnh dài cho thấy rằng mô hình

thường gặp khó khăn trong việc sử dụng hiệu quả thông tin nằm ở phần giữa hoặc cuối ngữ cảnh, hiện tượng này được mô tả trong nghiên cứu của Liu et al. [2].

Do đó, phân mảnh dữ liệu không đơn thuần là thao tác cắt nhỏ văn bản, mà là một quyết định thiết kế nhằm tối ưu hóa sự đánh đổi giữa độ chi tiết của tri thức và khả năng sử dụng ngữ cảnh của mô hình sinh. Khảo sát của Gao et al. [20] chỉ ra rằng chunking là một thành phần cốt lõi của giai đoạn tiền xử lý dữ liệu trong pipeline RAG và chất lượng của bước này có ảnh hưởng đáng kể đến hiệu quả toàn hệ thống.

2.4.3 Nhóm phương pháp chunking không có cấu trúc

Nhóm phương pháp phân mảnh không có cấu trúc bao gồm các kỹ thuật chia văn bản dựa trên các quy tắc đơn giản về độ dài hoặc ký tự phân tách, không yêu cầu hiểu biết sâu về ngữ nghĩa. Theo khảo của Gao et al. [20], đây là nhóm phương pháp được sử dụng phổ biến nhất trong các hệ thống RAG baseline và thường xuất hiện trong các triển khai “naive RAG”.

Phương pháp đơn giản nhất trong nhóm này là fixed-size chunking. Nếu gọi D là tài liệu gốc, tài liệu được biểu diễn như một chuỗi token $D = (t_1, t_2, \dots, t_N)$ và được chia thành các đoạn có độ dài cố định L . Mỗi chunk được xác định vị trí bởi công thức (2-3).

$$c_i = (t_{(i-1)L+1}, \dots, t_{iL}), \quad i = 1, \dots, \left\lfloor \frac{N}{L} \right\rfloor \quad (2-3)$$

Trong đó D là tài liệu gốc, t_i là token thứ i trong tài liệu, N là tổng số token của tài liệu, L là độ dài cố định của mỗi chunk, $\left\lfloor \frac{N}{L} \right\rfloor$ là số lượng chunk thu được.

Ưu điểm của cách tiếp cận này là đơn giản, dễ triển khai và có chi phí tính toán thấp. Tuy nhiên, do không xét đến ranh giới ngữ nghĩa, fixed-size chunking thường cắt ngang câu hoặc đoạn văn, dẫn đến hiện tượng phân mảnh ngữ cảnh (context fragmentation). Các chunk thu được có thể thiếu tính mạch lạc, làm giảm chất lượng embedding và kéo theo sự suy giảm hiệu quả truy xuất.

Để giảm thiểu hiện tượng này, sliding window chunking được đề xuất, trong đó các chunk liên tiếp có một phần chồng lấp (overlap). Gọi L là kích thước chunk và O là số

token chồng lấp, bước trượt $s = L - O$. Vị trí các token tạo lên mỗi chunk được xác định bởi công thức (2-4).

$$c_i = (t_{(i-1)s+1}, \dots, t_{(i-1)s+L}) \quad (2-4)$$

Cách tiếp cận này giúp duy trì sự liên kết ngữ nghĩa giữa các chunk kề nhau, đảm bảo rằng thông tin quan trọng không bị mất nếu nó nằm ở ranh giới cắt. Theo [20], sliding window chunking có thể cải thiện recall trong truy xuất, đặc biệt đối với các câu hỏi yêu cầu thông tin trải dài qua nhiều câu hoặc đoạn văn. Tuy nhiên, phương pháp này làm tăng số lượng chunk và chi phí lưu trữ, đồng thời dẫn đến sự trùng lặp thông tin trong ngữ cảnh được đưa vào mô hình sinh.

Một biến thể khác là recursive character splitting, trong đó hệ thống sử dụng một tập hợp các ký tự phân tách được sắp xếp theo thứ tự ưu tiên, chẳng hạn như ngắt đoạn, xuống dòng hoặc dấu câu để chia tài liệu sao cho mỗi chunk giữ được cấu trúc tự nhiên của văn bản trong giới hạn kích thước cho phép. Mặc dù giúp giảm hiện tượng cắt ngang câu, phương pháp này vẫn không đảm bảo rằng mỗi chunk tương ứng với một đơn vị tri thức hoàn chỉnh về mặt ngữ nghĩa.

2.4.4 Nhóm chunking theo ngữ nghĩa

Nhóm chunking theo đơn vị ngữ nghĩa tập trung vào việc xác định ranh giới phân mảnh dựa trên nội dung và ý nghĩa của văn bản. Một hướng nghiên cứu tiêu biểu là chunking dựa trên độ mịn của đơn vị truy xuất, được nghiên cứu và thảo luận bởi Chen et al. [21].

Phân mảnh theo ngữ nghĩa được đề xuất như một kỹ thuật tiên tiến, trong đó văn bản được phân tách thành các mệnh đề, những đơn vị thông tin nhỏ nhất, tự chứa đựng một sự thật cụ thể và có cấu trúc ngôn ngữ độc lập. Quá trình này có thể được mô hình hóa bởi công thức (2-5).

$$p_i = f_{LLM}(s_i) \quad (2-5)$$

trong đó s_i là một đoạn văn gốc và f_{LLM} là hàm trích xuất mệnh đề dựa trên mô hình ngôn ngữ. Kết quả là tập hợp các mệnh đề $\{p_1, p_2, \dots, p_m\}$ đại diện cho các đơn vị tri thức có mật độ thông tin cao.

Các thực nghiệm của Chen et al. [21] cho thấy rằng việc truy xuất dựa trên các đơn vị mệnh đề có thể cải thiện đáng kể độ chính xác khi trả lời các câu hỏi chi tiết, đặc biệt trong các tập dữ liệu tri thức dày đặc. Tuy nhiên, chi phí tính toán cao và sự phụ thuộc mạnh vào chất lượng của mô hình ngôn ngữ khiến các kỹ thuật semantic chunking hiện nay chủ yếu xuất hiện trong bối cảnh nghiên cứu, chưa được triển khai rộng rãi trong các hệ thống RAG sản xuất.

2.4.5 Nhóm chunking nâng cao

Bên cạnh hai nhóm trên, chunking nâng cao thường tận dụng cấu trúc nội tại của tài liệu. Một hướng tiếp cận tiêu biểu là small-to-big, trong đó tài liệu được chia thành các đoạn nhỏ để phục vụ truy xuất chính xác, nhưng ngữ cảnh được đưa vào mô hình sinh là một đoạn lớn hơn gồm nhóm các đoạn nhỏ được truy xuất. Về mặt hình thức, đoạn lớn được truy xuất này có thể được biểu diễn như hợp của các đoạn lân cận (2-6).

$$C_i = \bigcup_{j \in N(i)} c_j \quad (2-6)$$

trong đó $N(i)$ là tập các chunk lân cận của c_i . Cách tiếp cận này cho phép cân bằng giữa độ chính xác trong truy xuất và độ đầy đủ ngữ cảnh trong sinh văn bản, một bài toán đánh đổi được nhấn mạnh trong nhiều nghiên cứu về RAG [20].

Ngoài ra, layout-aware indexing xử lý các tài liệu có cấu trúc phân cấp rõ ràng và các cấu trúc như bảng biểu hoặc danh sách. Thay vì coi bảng là văn bản phẳng, hệ thống chunking dựa trên hàng, cột hoặc chuyển đổi bảng sang các định dạng có cấu trúc như Markdown hoặc JSON nhằm bảo toàn mối quan hệ giữa các thực thể dữ liệu. Theo khảo sát của Gao et al. [20], việc bảo toàn cấu trúc bố cục giúp cải thiện đáng kể khả năng truy xuất và suy luận trên các tài liệu kỹ thuật và khoa học.

2.4.6 Giới hạn độ dài ngữ cảnh

Giới hạn độ dài ngữ cảnh của mô hình ngôn ngữ là một ràng buộc mang tính hệ thống, ảnh hưởng trực tiếp đến mọi quyết định thiết kế trong pipeline RAG. Quá trình sinh câu trả lời có thể được mô hình hóa bởi công thức (2-7).

$$P(a \mid q, C) = P(a \mid q, c_1, c_2, \dots, c_k) \quad (2-7)$$

Trong đó q là truy vấn, c_i là các chunk được truy xuất, và k bị giới hạn bởi tổng số token mà mô hình có thể xử lý trong một lần suy luận, $P(a \mid q, C)$ là xác suất sinh câu trả lời a của mô hình ngôn ngữ [17]. Khi kích thước hoặc số lượng chunk tăng lên, hệ thống buộc phải giảm k hoặc cắt bớt ngữ cảnh, làm tăng nguy cơ mất thông tin quan trọng.

Nghiên cứu của Chen et al. [21] chỉ ra rằng việc sử dụng các chunk quá lớn làm giảm khả năng truy xuất chính xác, trong khi các chunk quá nhỏ lại làm mất ngữ cảnh cần thiết cho sinh câu trả lời. Do đó, thiết kế chunking luôn là một bài toán phụ thuộc vào nhu cầu thực tế, đồng thời là động lực cho các nghiên cứu về truy xuất đa giai đoạn và các chiến lược chunking nâng cao trong các chương tiếp theo.

2.4.7 So sánh các chiến lược chunking

Bảng 2.1 Tổng hợp các chiến lược chunking phổ biến theo các trục đánh giá có ý nghĩa trực tiếp đối với hiệu quả của RAG baseline, bao gồm độ mịn (granularity), mức độ mạch lạc của ngữ cảnh (context coherence), khả năng cải thiện recall trong truy xuất, mức độ nhiễu ngữ cảnh và chi phí lập chỉ mục.

Phương pháp	Granularity	Context coherence	Retrieval recall	Context noise	Chi phí indexing
Fixed-size	Thấp	Thấp	Thấp	Cao	Thấp
Sliding window	Trung Bình	Trung bình	Cao	Trung bình	Trung bình
Recursive splitting	Trung bình	Trung bình	Trung bình	Trung bình	Thấp - trung bình
Small-to-Big	Cao	Cao	Cao	Thấp	Cao
Layout-aware	Trung bình - cao	Cao	Trung bình	Thấp	Trung bình - cao

Mặc dù các phương pháp chunking khác nhau đều nhằm mục tiêu phân mảnh tài liệu để phục vụ truy xuất tri thức, chúng thể hiện những đặc điểm và đánh đổi khác nhau. Do đó, việc so sánh các chiến lược chunking không nên chỉ dừng ở mô tả kỹ thuật, mà cần được xem xét dưới góc độ ảnh hưởng của chúng đến các yếu tố cốt lõi của RAG, bao gồm độ mịn của đơn vị tri thức, khả năng bảo toàn ngữ cảnh, mức độ nhiễu trong truy xuất và chi phí triển khai.

Từ Bảng 2.1 có thể nhận thấy rằng các phương pháp phân mảnh không có cấu trúc như fixed-size hoặc sliding window có ưu điểm về tính đơn giản và khả năng triển khai, do đó thường được sử dụng trong các hệ thống RAG cơ bản. Tuy nhiên, các phương pháp này bộc lộ hạn chế rõ rệt về khả năng bảo toàn ngữ nghĩa và kiểm soát nhiễu ngữ cảnh, đặc biệt khi quy mô corpus tăng lên. Ngược lại, các phương pháp chunking theo đơn vị ngữ nghĩa hoặc chunking có cấu trúc cho thấy tiềm năng cải thiện đáng kể chất lượng truy xuất và giảm nhiễu, nhưng đi kèm với chi phí tính toán cao và độ phức tạp trong thiết kế hệ thống.

Những đánh đổi này cho thấy rằng không tồn tại một chiến lược chunking tối ưu tuyệt đối cho mọi kịch bản. Thay vào đó, việc lựa chọn phương pháp chunking trong RAG cơ bản thường ưu tiên tính khả thi và chi phí triển khai, trong khi các chiến lược nâng cao chỉ phù hợp khi yêu cầu độ chính xác cao hơn hoặc khi tập văn bản có cấu trúc phức tạp. Chính các hạn chế này của chunking trong RAG cơ bản đã đặt nền tảng cho các hướng nghiên cứu và cải tiến pipeline, sẽ được phân tích chi tiết trong chương tiếp theo.

2.5 Lưu trữ và biểu diễn tri thức

Trong các hệ thống RAG, biểu diễn và lưu trữ tri thức đóng vai trò trung tâm, quyết định trực tiếp đến hiệu quả của giai đoạn truy xuất và gián tiếp ảnh hưởng đến chất lượng câu trả lời được sinh ra. Khác với các mô hình ngôn ngữ thuần tham số, nơi tri thức được mã hóa ngầm trong trọng số của mô hình, RAG yêu cầu một cơ chế biểu diễn tri thức tường minh, có thể truy cập, cập nhật và kiểm soát được tại thời điểm suy luận.

Về mặt khái niệm, biểu diễn tri thức trong RAG có thể được chia thành hai hướng chính: biểu diễn ngữ nghĩa liên tục thông qua vector dense và biểu diễn thưa dựa trên từ khóa (sparse representation). Hai hướng này tương ứng với hai cách nhìn khác nhau về sự liên quan (relevance): một bên nhấn mạnh tương đồng ngữ nghĩa tổng quát, bên còn lại tập trung vào sự trùng khớp chính xác của thuật ngữ.

2.5.1 Biểu diễn tri thức bằng biểu diễn vector dày

Dense embedding tức biểu diễn vector dày là phương pháp biểu diễn văn bản dưới dạng các vector liên tục trong không gian có số chiều cố định, thường được sinh ra bởi các mô hình mã hoá dựa trên Transformer như BERT, Sentence-BERT hoặc các mô hình

biểu diễn vector chuyên dụng. Ý tưởng cốt lõi của biểu diễn vector dạng dense là ánh xạ các đơn vị văn bản như câu, đoạn, hoặc chunk vào một không gian vector sao cho các đơn vị có ý nghĩa ngữ nghĩa gần nhau sẽ nằm gần nhau trong không gian vector [22], [23].

Về mặt hình thức, một đoạn văn bản x được ánh xạ thành một vector dense $e_x \in \mathbb{R}^d$ thông qua một hàm biểu diễn vector $f()$:

$$e_x = f(x) \in \mathbb{R}^d \quad (2-8)$$

Trong đó, d là số chiều của không gian biểu diễn vector và f thường là một mô hình được huấn luyện để tối ưu hóa sự tương đồng ngữ nghĩa. Khi truy vấn q được đưa vào hệ thống, nó cũng được ánh xạ thành một vector e_q , và mức độ liên quan giữa truy vấn và một đơn vị tri thức được xác định thông qua một hàm đo độ tương đồng, phổ biến nhất là độ tương đồng cosine (2-9).

$$\text{sim}(e_q, e_x) = \frac{e_q \cdot e_x}{\|e_q\| \|e_x\|} \quad (2-9)$$

Trong đó $e_q \cdot e_x$ là tích vô hướng của vector đoạn văn bản được lưu trong cơ sở dữ liệu và vector câu hỏi đầu vào, $\|e_q\| \|e_x\|$ là chuẩn Euclid của các vector dùng để chuẩn hoá độ dài.

Cách tiếp cận này cho phép hệ thống RAG truy xuất các đoạn văn có nội dung ngữ nghĩa gần với truy vấn, ngay cả khi không có sự trùng khớp trực tiếp về mặt từ vựng. Đây là một ưu điểm quan trọng trong các kịch bản hỏi đáp sử dụng ngôn ngữ tự nhiên, người dùng có thể diễn đạt cùng một ý theo nhiều cách khác nhau. Theo Lewis et al. [20] trong công trình nền tảng về RAG, dense retrieval giúp cải thiện đáng kể hiệu quả trên các tác vụ hỏi đáp dựa tri thức so với các phương pháp truy xuất dựa trên từ khóa thuần túy, đặc biệt khi tập văn bản có quy mô lớn và nội dung đa dạng.

Tuy nhiên, biểu diễn vector dạng dense cũng mang những đặc điểm cần được cân nhắc. Việc biểu diễn tri thức dưới dạng vector liên tục khiến hệ thống phụ thuộc mạnh vào chất lượng của mô hình biểu diễn vector và dữ liệu huấn luyện của mô hình. Ngoài ra, biểu diễn vector dense thường hoạt động tốt với các truy vấn mang tính khái quát hoặc

diễn giải, nhưng có thể gặp khó khăn trong các câu hỏi yêu cầu khớp chính xác thuật ngữ, mã số hoặc tên riêng hiếm gặp [24]

2.5.2 Biểu diễn tri thức bằng biểu diễn vector thưa

Song song với biểu diễn vector dense, biểu diễn thưa tức dense embedding dựa trên việc biểu diễn văn bản thông qua các đặc trưng rời rạc, điển hình là các từ khóa hoặc n-gram. Một trong những mô hình sparse phổ biến nhất trong truy xuất thông tin là BM25, được xem là một chuẩn mực nhờ tính đơn giản và hiệu quả [25].

Trong BM25, mức độ liên quan giữa truy vấn q và tài liệu d được tính dựa trên tần suất xuất hiện của các thuật ngữ trong tài liệu, độ dài tài liệu và các tham số điều chỉnh. Công thức tổng quát của BM25 có thể được biểu diễn bởi công thức (2-10):

$$BM25(q, d) = \sum_{t \in q} IDF(t) \cdot \left(\frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b \frac{|d|}{avgdl})} \right) \quad (2-10)$$

Trong đó $f(t, d)$ là tần suất của thuật ngữ t trong tài liệu d , $IDF(t)$ là trọng số nghịch đảo tần suất tài liệu, và k_1, b là các tham số điều chỉnh. Cách tiếp cận này đặc biệt hiệu quả với các truy vấn yêu cầu độ chính xác cao về từ khóa, chẳng hạn như tên văn bản pháp lý, mã điều khoản, hoặc các thuật ngữ chuyên ngành. Theo khảo sát của Gao et al. [20], sparse retrieval giữ vai trò quan trọng trong hệ thống RAG thực tế, đặc biệt khi corpus chứa nhiều thuật ngữ chuyên biệt.

Tuy nhiên, do phụ thuộc vào sự trùng khớp của từ vựng, biểu diễn thưa khó nắm bắt được các đoạn văn liên quan nếu chúng được diễn đạt bằng từ ngữ khác [26].

2.5.3 Vector database

Cơ sở dữ liệu vector (vector database) là một lớp hạ tầng lưu trữ và truy xuất dữ liệu được thiết kế để làm việc trực tiếp với các vector trong không gian nhiều chiều. Trong đó, mỗi đối tượng dữ liệu như văn bản, hình ảnh, hoặc đặc trưng trích xuất được biểu diễn dưới dạng một vector số thực $e \in \mathbb{R}^d$, và các phép truy vấn được thực hiện dựa trên độ tương đồng hoặc khoảng cách giữa các vector. Về bản chất, vector database tập trung giải quyết bài toán tìm kiếm lân cận gần nhất (nearest neighbor search) trong

không gian vector, một bài toán đã được nghiên cứu từ lâu trong các lĩnh vực như Information Retrieval, Computer Vision và Multimedia Retrieval [27].

Khác với các hệ quản trị cơ sở dữ liệu truyền thống vốn tối ưu cho truy vấn chính xác theo khóa hoặc theo điều kiện logic, vector database được xây dựng xoay quanh các cấu trúc chỉ mục chuyên dụng cho dữ liệu vector, cho phép truy xuất hiệu quả trong không gian có số chiều lớn. Các kỹ thuật index phổ biến như KD-tree, IVF hay Hierarchical Navigable Small World (HNSW) được đề xuất nhằm giảm đáng kể chi phí tính toán so với tìm kiếm tuyến tính, bằng cách tổ chức không gian vector theo những cấu trúc cho phép loại bỏ nhanh các vùng không liên quan trong quá trình truy vấn.

Trong bối cảnh RAG, tri thức ngoài mô hình được biểu diễn dưới dạng biểu diễn vector dense để nắm bắt tương đồng ngữ nghĩa giữa truy vấn và các đoạn văn bản. Khi đó, tập tri thức của hệ thống RAG về bản chất trở thành một tập lớn các vector trong không gian $\mathbb{R}^d$, và bài toán truy xuất thông tin được chuyển hóa thành bài toán tìm kiếm các vector gần nhất với vector truy vấn. Với quy mô dữ liệu thực tế, việc so sánh truy vấn với toàn bộ tập vector là không khả thi về mặt hiệu năng, đặc biệt khi truy xuất diễn ra tại thời điểm suy luận và ảnh hưởng trực tiếp đến độ trễ phản hồi của hệ thống. Chính vì lý do đó, việc sử dụng vector database trong RAG là một lựa chọn hợp lý và tất yếu, xuất phát từ bản chất của cách biểu diễn tri thức bằng biểu diễn vector dense [26], [20].

2.5.4 So sánh biểu diễn dày và biểu diễn thưa

Các nghiên cứu của Gao et al. và Lin et al. [20], [24] đều thống nhất rằng biểu diễn dày tức dense embedding và biểu diễn thưa tức sparse representation phục vụ cho các loại truy vấn khác nhau và mang tính bổ sung hơn là thay thế lẫn nhau. Dense embedding phù hợp với các truy vấn diễn giải và câu hỏi ngôn ngữ tự nhiên, trong khi sparse representation hiệu quả với các truy vấn yêu cầu độ chính xác cao về thuật ngữ. Bảng 2.2 trình bày sự so sánh tổng quát giữa hai hướng biểu diễn tri thức trong hệ thống RAG.

Bảng 2.2 So sánh dense embedding và sparse representation trong hệ thống RAG..

Đặc điểm	Biểu diễn dày	Biểu diễn thưa
Cơ sở biểu diễn	Vector liên tục	Từ khoá rời rạc
Khả năng nắm bắt ngữ nghĩa	Cao	Hạn chế
Phù hợp với truy vấn	Diễn giải, tự nhiên	Thuật ngữ, chính xác
Phụ thuộc mô hình	Cần encoder	Không cần mô hình học sâu
Khả năng mở rộng	Phụ thuộc ANN index	Tốt với inverted index

2.6 Truy xuất thông tin

Trong kiến trúc RAG, truy xuất thông tin không tương đương với bài toán tìm kiếm truyền thống. Nếu như tìm kiếm trong các hệ thống Information Retrieval (IR) cổ điển chủ yếu nhằm mục tiêu xếp hạng và trả về những đoạn văn bản có mức độ tương đồng cao nhất với truy vấn, thì truy xuất trong RAG mang một mục tiêu căn bản khác: lựa chọn một tập ngữ cảnh đủ thông tin để mô hình ngôn ngữ lớn có thể suy luận và sinh ra câu trả lời đúng. Do đó, chất lượng của bước truy xuất trong RAG cần được đánh giá không chỉ bằng độ tương đồng giữa truy vấn và tài liệu, mà còn bằng mức độ hỗ trợ suy luận của tập ngữ cảnh được đưa vào mô hình sinh.

Về mặt hình thức, giả sử kho tri thức của hệ thống được ký hiệu là D , bao gồm toàn bộ các đơn vị có thể truy xuất (đoạn văn, chunk hoặc tài liệu). Do giới hạn độ dài ngữ cảnh của LLM, tại thời điểm suy luận chỉ có thể chọn ra một tập con nhỏ $C \subset D$ để làm ngữ cảnh đầu vào được biểu diễn công thức (2-11).

$$C = \{c_1, c_2, \dots, c_k\}, \quad k \ll |D| \quad (2-11)$$

Ở đây, mục tiêu của truy xuất không phải là tối ưu hóa độ tương đồng từng phần tử c_i với truy vấn q , mà là tối ưu hóa khả năng suy luận của mô hình khi được điều kiện hóa trên toàn bộ tập C . Cách nhìn này phù hợp với mô hình hóa xác suất của RAG, trong đó các đoạn truy xuất được xem là biến ẩn điều kiện hóa phân phối sinh của mô hình ngôn ngữ [17].

2.6.1 Các chiến lược truy xuất

Trong các hệ thống RAG cơ bản, truy xuất thường được triển khai theo mô hình một giai đoạn (single-stage retrieval), nghĩa là truy vấn được so khớp trực tiếp với toàn bộ kho chỉ mục và hệ thống trả về top- k đơn vị có điểm xếp hạng cao nhất. Khảo sát của Gao et al. [20] tổng quan về RAG chỉ ra rằng ba họ phương pháp truy xuất phổ biến nhất trong giai đoạn baseline là sparse retrieval, dense retrieval và hybrid retrieval.

Sparse retrieval đại diện cho các phương pháp dựa trên từ khóa, tiêu biểu là BM25. Trong cách tiếp cận này, văn bản được biểu diễn dưới dạng bag-of-words và mức độ liên quan được ước lượng dựa trên thống kê tần suất từ. BM25 đã được chứng minh là hiệu quả và ổn định trong nhiều thập kỷ nghiên cứu IR, đặc biệt với các truy vấn chứa thuật ngữ chính xác, ký hiệu hoặc thực thể hiếm [25]. Tuy nhiên, do không mô hình hóa ngữ nghĩa ở mức sâu, sparse retrieval gặp hạn chế khi truy vấn được diễn đạt linh hoạt hoặc khi thông tin cần thiết không chia sẻ trực tiếp từ khóa với câu hỏi.

Dense retrieval chuyển bài toán truy xuất sang không gian vector liên tục bằng cách sử dụng mô hình embedding để ánh xạ truy vấn và các đơn vị tri thức vào cùng một không gian ngữ nghĩa. Độ tương đồng thường được tính bằng cosine similarity. Nghiên cứu của Karpukhin [26] về Dense Passage Retrieval cho thấy phương pháp này vượt trội hơn các mô hình sparse trong nhiều bài toán hỏi đáp mở, nhờ khả năng nắm bắt quan hệ ngữ nghĩa và paraphrase. Tuy nhiên, dense retrieval cũng bộc lộ hạn chế quan trọng: các đoạn văn tương đồng nghĩa với nhau trong không gian véc-tơ có thể chỉ cùng chủ đề chung mà không chứa thông tin trọng tâm cần thiết cho việc trả lời, dẫn đến hiện tượng lệch ngữ nghĩa (semantic drift).

Hybrid retrieval được đề xuất nhằm kết hợp ưu điểm của hai hướng tiếp cận trên. Bằng cách kết hợp tín hiệu từ sparse và dense retrieval, hệ thống có thể vừa đảm bảo độ chính xác từ khóa, vừa duy trì khả năng bao phủ ngữ nghĩa. Khảo sát của Gao et al. [20] xem hybrid retrieval là phương pháp phổ biến trong RAG thực tế, đặc biệt khi kho dữ liệu lớn và truy vấn đa dạng.

2.6.2 Đơn vị truy xuất và mối liên hệ với chunking

Hiệu quả của truy xuất trong RAG phụ thuộc chặt chẽ vào đơn vị được lập chỉ mục. Nếu kho tri thức được chia thành các chunk có ranh giới không phù hợp, truy xuất sẽ phải đối mặt với sự đánh đổi giữa độ đầy đủ ngữ cảnh và mức độ nhiễu. Nghiên cứu của Chen et al. [21] về độ mịn truy xuất chỉ ra rằng việc lựa chọn đơn vị truy xuất đoạn, câu hay mệnh đề có ảnh hưởng đáng kể đến chất lượng truy xuất và hiệu năng hỏi đáp downstream. Xem chi tiết trong Bảng 2.1.

2.6.3 Nhiễu ngữ cảnh

Một vấn đề cốt lõi của truy xuất trong RAG là hiện tượng nhiễu ngữ cảnh (context noise). Context noise có thể được hiểu là phần thông tin trong tập ngữ cảnh được truy xuất không cần thiết hoặc gây cản trở cho việc suy luận đúng, mặc dù về mặt hình thức các đoạn này có thể được xem là liên quan đến truy vấn. Khảo sát của Gao et al. [20] cho thấy nhiễu ngữ cảnh không phải là hiện tượng ngoại lệ, mà là hệ quả tự nhiên của việc truy xuất trong kho dữ liệu lớn với năng lực truy xuất hữu hạn.

Về bản chất, context noise có thể xuất hiện dưới nhiều dạng. Nhiễu ngữ nghĩa phát sinh khi quá trình truy xuất chọn các đoạn cùng chủ đề nhưng khác trọng tâm lập luận, đặc biệt phổ biến trong dense retrieval do semantic drift. Nhiễu cấu trúc xuất hiện khi chunk bị cắt sai ranh giới, làm mất mối liên hệ logic nội tại của thông tin. Nhiễu dư thừa xảy ra khi nhiều đoạn lặp lại cùng một ý, làm tăng độ dài ngữ cảnh mà không bổ sung tri thức mới. Ngoài ra, trong các kịch bản đa tài liệu, nhiễu liên tài liệu có thể làm mô hình trộn lẫn thông tin từ các nguồn khác nhau, dẫn đến câu trả lời không nhất quán.

Nghiên cứu của Liu et al. [2] về hành vi sử dụng ngữ cảnh dài của LLM cho thấy mô hình không khai thác ngữ cảnh một cách đồng đều, hiệu năng suy luận có thể giảm đáng kể khi bằng chứng nằm ở giữa chuỗi ngữ cảnh dài, ngay cả khi thông tin đúng đã được cung cấp. Điều này làm cho context noise trở nên đặc biệt nguy hiểm trong RAG, bởi nhiễu không chỉ chiếm chỗ token mà còn làm suy giảm khả năng mô hình tập trung vào bằng chứng quan trọng.

2.6.4 Đánh đổi giữa recall và precision trong truy xuất

Trong truy xuất thông tin, chất lượng kết quả thường được đánh giá thông qua hai chỉ số cơ bản là recall và precision. Gọi R là tập các tài liệu thực sự liên quan đến truy vấn, và A là tập các tài liệu được hệ thống truy xuất, precision và recall lần lượt được biểu diễn bởi công thức (2-12) (2-13).

$$Precision = \frac{|A \cap R|}{|A|} \quad (2-12)$$

$$Recall = \frac{|A \cap R|}{|R|} \quad (2-13)$$

Trong đó, precision phản ánh tỷ lệ các tài liệu liên quan trong tập kết quả được truy xuất, còn recall đo lường mức độ hệ thống có khả năng bao phủ đầy đủ các tài liệu liên quan tồn tại trong corpus.

Nguồn gốc lý thuyết của context noise trong RAG baseline có thể được giải thích thông qua đánh đổi kinh điển giữa recall và precision trong IR. Khi tăng k để mở rộng tập C , khả năng bao phủ đầy đủ các mảnh bằng chứng cần thiết sẽ tăng tức recall cao hơn, nhưng đồng thời lượng thông tin không liên quan cũng tăng, làm giảm precision và gia tăng nhiễu. Ngược lại, khi giảm k , precision có thể được cải thiện nhưng rủi ro bỏ sót thông tin quan trọng lại tăng, đặc biệt với các câu hỏi yêu cầu nhiều điều kiện hoặc suy luận đa bước [25], [20], [2].

Trong bối cảnh RAG, đánh đổi này trở nên phức tạp hơn vì hệ quả của nhiễu không chỉ là xếp hạng sai, mà còn ảnh hưởng trực tiếp đến hành vi sinh câu trả lời của LLM. Khảo sát về ảo giác của Ji et al. [3] chỉ ra rằng nhiều trường hợp sinh thông tin sai không xuất phát từ bản thân mô hình, mà từ việc mô hình được điều kiện hóa trên ngữ cảnh không phù hợp hoặc thiếu chặt chẽ. Do đó, không tồn tại một giá trị k tối ưu cho mọi truy vấn, đặc biệt khi kho dữ liệu lớn và đa dạng, và context noise trở thành một đặc trưng gần như không tránh khỏi của RAG baseline.

2.7 Sinh câu trả lời

2.7.1 Vai trò của thành phần sinh câu trả lời trong kiến trúc RAG

Trong kiến trúc RAG, thành phần sinh câu trả lời (Generator) đảm nhiệm vai trò trung tâm trong việc chuyển hóa truy vấn của người dùng và ngữ cảnh được truy xuất thành một câu trả lời tự nhiên, mạch lạc và có vẻ thuyết phục. Về mặt khái niệm, Generator thường là một mô hình ngôn ngữ lớn hoặc mô hình sinh chuỗi (sequence-to-sequence) đã được tiền huấn luyện, có khả năng sinh văn bản dựa trên xác suất có điều kiện. Điểm khác biệt cốt lõi giữa RAG và các hệ thống hỏi-đáp thuần LLM là quá trình sinh không chỉ phụ thuộc vào tri thức tham số của mô hình, mà còn được điều kiện hóa bởi tri thức ngoài thông qua ngữ cảnh truy xuất tại thời điểm suy luận.

Tuy nhiên, cần nhấn mạnh rằng Generator trong RAG không phải là một mô-đun suy luận có khả năng kiểm chứng hay đánh giá độ đúng sai của ngữ cảnh đầu vào. Về bản chất, LLM vẫn hoạt động như một mô hình sinh xác suất, học cách tối ưu hóa phân phối sinh văn bản dựa trên chuỗi đầu vào, chứ không sở hữu cơ chế logic hoặc kiểm tra tính xác thực của tri thức được cung cấp. Điều này dẫn đến một thực tế quan trọng trong các hệ thống RAG hiện đại đó là chất lượng câu trả lời không chỉ phụ thuộc vào khả năng truy xuất, mà còn phụ thuộc mạnh vào cách ngữ cảnh được đưa vào mô hình sinh.

Nghiên cứu của Lewis [17] chỉ ra rằng việc tích hợp tri thức ngoài vào quá trình sinh có thể cải thiện đáng kể hiệu năng trên các bài toán hỏi đáp dựa trên tri thức, so với các mô hình sinh thuần túy. Tuy nhiên, khảo sát của Gao et al. [20] hoặc Ji et al. [3] cũng cho thấy rằng ngay cả khi đã cung cấp ngữ cảnh liên quan, mô hình sinh vẫn có thể tạo ra câu trả lời sai lệch hoặc không trung thực nếu ngữ cảnh không được tích hợp một cách phù hợp hoặc vượt quá khả năng xử lý của mô hình. Do đó, để hiểu rõ cơ chế sinh câu trả lời trong RAG, cần phân tích chi tiết cách thức xây dựng prompt và cách ngữ cảnh cung cấp cho mô hình sinh.

2.7.2 Chiến lược xây dựng prompt

Chiến lược xây dựng prompt trong RAG bao gồm truy vấn ban đầu của người dùng, ngữ cảnh truy xuất từ kho tri thức và các chỉ dẫn (instructions) bổ sung nhằm định hướng hành vi sinh của mô hình. Không giống như các hệ thống LLM tương tác thông thường,

chủ yếu đóng vai trò kích hoạt hành vi hội thoại, prompt trong RAG mang tính cấu trúc cao hơn và có vai trò quyết định trong việc mô hình có sử dụng đúng tri thức được truy xuất hay không.

Về mặt hình thức, prompt trong RAG thường được xây dựng theo dạng nối chuỗi, trong đó truy vấn và các đoạn ngữ cảnh được đặt trong cùng một không gian đầu vào của mô hình sinh. Một biểu diễn tổng quát cho quá trình sinh trong RAG có thể được biểu diễn bởi công thức (2-14).

$$P(y \mid q, C) = \prod_{t=1}^T P(y_t \mid y_{<t}, q, C) \quad (2-14)$$

Trong đó q là truy vấn đầu vào, $C = \{c_1, c_2, \dots, c_k\}$ là tập các đoạn ngữ cảnh được truy xuất, và y là câu trả lời được sinh. Công thức này cho thấy rằng ngữ cảnh truy xuất chỉ đóng vai trò điều kiện hóa phân phối sinh, chứ không phải là ràng buộc logic bắt buộc đối với kết quả đầu ra.

Nghiên cứu của Min et al. [28] đã chỉ ra rằng việc thiết kế prompt có cấu trúc rõ ràng, tách biệt giữa phần ngữ cảnh và phần câu hỏi, giúp mô hình sinh sử dụng tri thức ngoài một cách hiệu quả hơn. Ngược lại, các prompt không có cấu trúc hoặc trộn lẫn ngữ cảnh với truy vấn có thể khiến mô hình bỏ qua thông tin quan trọng, hoặc thậm chí dựa chủ yếu vào tri thức tham số của chính nó để sinh câu trả lời.

2.7.3 Chèn ngữ cảnh

Chèn ngữ cảnh hay context injection đề cập đến cách thức đưa các đoạn tri thức được truy xuất vào prompt của mô hình sinh. Trong các hệ thống RAG, context injection thường được thực hiện bằng cách nối trực tiếp các đoạn văn bản truy xuất vào prompt, theo một thứ tự nhất định, ví dụ: theo độ liên quan giảm dần. Cách tiếp cận này có ưu điểm là đơn giản và dễ triển khai, nhưng đồng thời cũng bộc lộ nhiều giới hạn khi quy mô ngữ cảnh tăng lên.

Một trong những giới hạn quan trọng nhất là giới hạn độ dài ngữ cảnh của mô hình sinh. Mặc dù các LLM hiện đại đã mở rộng đáng kể khả năng xử lý chuỗi dài, các nghiên cứu thực nghiệm cho thấy mô hình không sử dụng thông tin trong ngữ cảnh một cách đồng

đều. Công trình nghiên cứu của Liu et al. [2] chỉ ra rằng LLM có xu hướng chú ý nhiều hơn đến phần đầu và phần cuối của chuỗi ngữ cảnh, trong khi các đoạn nằm ở giữa dễ bị bỏ qua hoặc sử dụng kém hiệu quả. Điều này dẫn đến hiện tượng mà trong hệ thống RAG, dù đã truy xuất đúng thông tin, mô hình sinh vẫn có thể bỏ sót hoặc không khai thác được các bằng chứng quan trọng.

Ngoài ra, việc tiêm quá nhiều ngữ cảnh vào prompt có thể gây ra hiện tượng nhiễu thông tin, làm giảm khả năng tập trung của mô hình sinh và dẫn đến câu trả lời mơ hồ hoặc sai lệch. Các khảo sát của Gao et al. [20] nhấn mạnh rằng nhiều ngữ cảnh hơn không đồng nghĩa với câu trả lời tốt hơn, và việc lựa chọn cũng như tổ chức ngữ cảnh có vai trò quyết định trong chất lượng sinh.

2.7.4 Các chiến lược prompt

Trong bối cảnh RAG, hai khái niệm thường được nhắc đến khi nói về cách sử dụng ngữ cảnh trong prompt là prompt stuffing và prompt grounding. Prompt stuffing đề cập đến chiến lược đưa càng nhiều ngữ cảnh càng tốt vào prompt, với kỳ vọng rằng mô hình sinh sẽ tự động chọn ra thông tin phù hợp để trả lời câu hỏi. Cách tiếp cận này thường xuất hiện trong các hệ thống RAG baseline hoặc các triển khai đơn giản, nơi mục tiêu chính là tối đa hóa recall của tri thức được cung cấp cho mô hình.

Tuy nhiên, prompt stuffing bộc lộ nhiều hạn chế trong thực tế. Khi prompt chứa quá nhiều thông tin không liên quan trực tiếp đến truy vấn, mô hình sinh có thể bị phân tán chú ý, dẫn đến câu trả lời dài dòng, thiếu trọng tâm hoặc thậm chí mâu thuẫn. Nghiên cứu thực nghiệm của Press et al. [29] cho thấy rằng prompt stuffing có thể làm tăng nguy cơ ảo giác, do mô hình cố gắng kết nối các mảnh thông tin rời rạc thành một câu trả lời có vẻ hợp lý.

Ngược lại, prompt grounding tập trung vào việc đưa vào prompt chỉ những ngữ cảnh thực sự cần thiết, đồng thời sử dụng các chỉ dẫn rõ ràng để yêu cầu mô hình sinh trả lời dựa trên các bằng chứng được cung cấp. Prompt grounding không nhằm mục tiêu cung cấp nhiều tri thức nhất, mà nhằm đảm bảo rằng mô hình sinh được neo (grounded) vào một tập thông tin cụ thể, có liên quan chặt chẽ đến truy vấn. Công trình của Gao et al.

[20] cho thấy rằng prompt grounding giúp cải thiện đáng kể độ trung thực của câu trả lời, đặc biệt trong các hệ thống hỏi đáp dựa trên tài liệu.

Bảng 2.3 Tóm tắt chiến lược prompt stuffing và prompt grounding.

Tiêu chí	Prompt stuffing	Prompt grounding
Mục tiêu chính	Tối đa hoá lượng ngữ cảnh	Tối đa hoá mức độ liên quan
Cách đưa ngữ cảnh	Nói nhiều về đoạn văn bản	Chọn lọc và cấu trúc
Nguy cơ nhiễu	Cao	Thấp
Khả năng giảm hallucination	Thấp	Cao
Phù hợp với	RAG baseline, corpus nhỏ	Hệ thống RAG nâng cao

2.7.5 Hallucination trong giai đoạn sinh

Một quan niệm phổ biến nhưng chưa hoàn toàn chính xác là việc sử dụng RAG sẽ loại bỏ hoàn toàn hiện tượng hallucination. Trên thực tế, nghiên cứu của Ji et al. [3] đã chỉ ra rằng hallucination vẫn có thể xảy ra trong giai đoạn sinh, ngay cả khi mô hình đã được cung cấp ngữ cảnh liên quan. Nguyên nhân cốt lõi của hiện tượng này không nằm ở truy xuất, mà nằm ở bản chất của mô hình sinh.

LLM không sở hữu cơ chế nội tại để kiểm chứng tính đúng sai của thông tin trong ngữ cảnh. Thay vào đó, mô hình học cách sinh văn bản dựa trên các mẫu thống kê trong dữ liệu huấn luyện, với mục tiêu tối ưu hóa xác suất sinh chuỗi hợp lý về mặt ngôn ngữ. Khi ngữ cảnh được cung cấp không đầy đủ, mâu thuẫn hoặc vượt quá khả năng xử lý, mô hình có thể sinh câu trả lời bằng tri thức tham số của chính nó, dẫn đến các câu trả lời nghe có vẻ hợp lý nhưng không được hỗ trợ bởi bằng chứng trong ngữ cảnh.

Khảo sát về ảo giác của Li et al. [30] trong sinh ngôn ngữ tự nhiên nhấn mạnh rằng đây là một vấn đề mang tính hệ thống, xuất phát từ việc mô hình sinh không phân biệt được đâu là tri thức được cung cấp và đâu là tri thức đã học. Trong RAG, ngữ cảnh chỉ đóng vai trò là điều kiện đầu vào, chứ không phải là ràng buộc bắt buộc đối với kết quả sinh.

Do đó, ngay cả khi truy xuất hoạt động chính xác, mô-đun sinh vẫn có thể tạo ra thông tin sai lệch nếu prompt không được thiết kế để ràng buộc chặt chẽ quá trình sinh vào các bằng chứng được cung cấp.

2.8 Đánh giá hệ thống RAG

Trong các hệ thống hỏi đáp truyền thống, chất lượng mô hình thường được đánh giá chủ yếu dựa trên mức độ trùng khớp giữa câu trả lời dự đoán và đáp án tham chiếu thông qua các chỉ số như Exact Match (EM) hoặc F1-score. Các chỉ số này tỏ ra hiệu quả trong những bài toán hỏi đáp trích xuất (extractive QA), nơi câu trả lời thường là một đoạn văn bản ngắn xuất hiện trực tiếp trong ngữ cảnh. Tuy nhiên, khi chuyển sang kiến trúc RAG, các phương pháp đánh giá này bộc lộ những hạn chế mang tính bản chất.

RAG là một hệ thống lai, kết hợp giữa mô-đun truy xuất tri thức từ bộ nhớ ngoài và mô hình sinh ngôn ngữ tự do. Do đó, một câu trả lời đúng về mặt nội dung chưa đủ để khẳng định hệ thống hoạt động tốt, nếu câu trả lời đó không thực sự dựa trên các bằng chứng đã được truy xuất. Nghiên cứu của Lewis et al. [17] hay nghiên cứu của Ji et al. [3] đã chỉ ra rằng các mô hình ngôn ngữ lớn có thể sinh ra câu trả lời chính xác nhờ tri thức tham số được học trong quá trình huấn luyện, ngay cả khi ngữ cảnh truy xuất không liên quan hoặc thậm chí bị bỏ qua hoàn toàn, dẫn đến hiện tượng ảo giác khó phát hiện nếu chỉ dựa trên EM hoặc F1.

Chính vì vậy, khảo sát của Gao et al. [20] nhấn mạnh rằng việc đánh giá cần vượt ra khỏi phạm vi từ khoá chính xác tuyệt đối và chuyển sang đánh giá mối quan hệ giữa câu trả lời, ngữ cảnh truy xuất và nguồn tri thức bên ngoài. Trong bối cảnh này, một số khái niệm đánh giá đặc thù đã được đề xuất và dần trở thành chuẩn mực trong nghiên cứu RAG, bao gồm faithfulness, completeness, context relevance và citation grounding.

Faithfulness (tính trung thực) phản ánh mức độ mà câu trả lời được sinh ra chỉ dựa trên thông tin có trong ngữ cảnh truy xuất. Một câu trả lời được coi là faithful nếu mọi khẳng định trong đó đều có thể được suy ra trực tiếp từ các đoạn tri thức đã được cung cấp cho mô hình sinh. Ngược lại, khi mô hình bổ sung các thông tin không tồn tại trong ngữ cảnh hoặc dựa vào tri thức tham số nội tại để suy đoán, câu trả lời bị xem là không faithful. Khái niệm này có mối liên hệ chặt chẽ với vấn đề ảo giác trong mô hình ngôn

ngữ lớn, vốn được xác định là một trong những rào cản lớn nhất khi triển khai LLM trong các ứng dụng đòi hỏi độ tin cậy cao [3], [16]. Trong hệ thống RAG, faithfulness không chỉ phản ánh chất lượng của mô hình sinh, mà còn gián tiếp phản ánh hiệu quả của mô-đun truy xuất và cách thức kết hợp ngữ cảnh vào prompt.

Bên cạnh tính trung thực, completeness (tính đầy đủ) là một tiêu chí quan trọng khác trong đánh giá RAG. Completeness mô tả mức độ mà câu trả lời bao quát được toàn bộ các khía cạnh quan trọng của câu hỏi, dựa trên tri thức sẵn có trong kho dữ liệu. Một câu trả lời có thể faithful nhưng vẫn không complete nếu nó chỉ phản ánh một phần thông tin, bỏ sót các chi tiết cần thiết để người dùng hiểu đầy đủ vấn đề. Trong thực tế, completeness chịu ảnh hưởng mạnh từ chiến lược truy xuất, đặc biệt là số lượng và mức độ đa dạng của các đoạn ngữ cảnh được đưa vào mô hình sinh. Khảo sát của Gao et al. [20] cho thấy rằng việc cân bằng giữa completeness và các tiêu chí khác, như giảm nhiễu ngữ cảnh, là một thách thức cốt lõi trong thiết kế pipeline RAG.

Context relevance (độ liên quan của ngữ cảnh) tập trung vào việc đánh giá chất lượng của mô-đun truy xuất. Tiêu chí này đo lường mức độ mà các đoạn tri thức được truy xuất thực sự liên quan đến truy vấn đầu vào. Trong các tập văn bản lớn, sự xuất hiện của các đoạn không liên quan hoặc chỉ liên quan yếu là hiện tượng phổ biến và có tác động tiêu cực đến toàn bộ pipeline RAG. Nghiên cứu của Lewis et al. [17] và Liu et al. [2] đã chỉ ra rằng ngữ cảnh kém liên quan không chỉ làm giảm chất lượng câu trả lời, mà còn làm tăng nguy cơ ảo giác do mô hình phải suy luận trong một không gian thông tin nhiễu. Vì vậy, context relevance được xem là một chỉ báo quan trọng phản ánh hiệu quả của chiến lược indexing và retrieval.

Một tiêu chí khác ngày càng được quan tâm trong các hệ thống RAG là citation grounding, tức khả năng gắn kết câu trả lời với các nguồn tri thức cụ thể. Khác với mô hình ngôn ngữ thuần tham số, RAG cho phép hệ thống chỉ ra các đoạn văn bản hoặc tài liệu nguồn đã được sử dụng để tạo ra câu trả lời. Khả năng truy vết nguồn thông tin này đặc biệt quan trọng trong các ứng dụng doanh nghiệp, học thuật và pháp lý, nơi yêu cầu minh bạch và khả năng kiểm chứng là bắt buộc. Nghiên cứu của Liu et al. [2] coi citation grounding là một yếu tố then chốt để nâng cao mức độ tin cậy và khả năng chấp nhận của các hệ thống hỏi đáp dựa trên RAG trong thực tiễn.

Các tiêu chí này không được định nghĩa bằng các công thức đóng đơn giản, mà thường được đánh giá thông qua quy trình gán nhãn thủ công, đánh giá dựa trên suy luận ngữ nghĩa, hoặc sử dụng mô hình ngôn ngữ làm bộ đánh giá, phản ánh đúng bản chất phức tạp của bài toán RAG.

Bảng 2.4 Tóm tắt các phương pháp đánh giá phổ biến.

Tiêu chí	Thành phần phản ánh	Vai trò trong RAG
Failthfulness	Câu trả lời và ngữ cảnh	Đánh giá hallucination
Completeness	Câu trả lời	Đảm bảo bao quát thông tin
Context relevance	Ngữ cảnh truy xuất	Đánh giá chất lượng retrieval
Citation grounding	Câu trả lời và nguồn	Hỗ trợ kiểm chứng, minh bạch

CHƯƠNG 3 PHÂN TÍCH HẠN CHẾ RAG TRUYỀN THỐNG VÀ ĐỊNH HƯỚNG CẢI TIẾN

Chương này phân tích các hạn chế chính của kiến trúc RAG truyền thống, tập trung vào cách tiếp cận text-only và single-stage retrieval. Từ đó, chương đề xuất định hướng cải tiến, nhấn mạnh tính cấp thiết của kiến trúc multimodal (xử lý đa phương thức) và truy xuất hai giai đoạn. Đồng thời, chương cũng làm rõ các thách thức trong việc triển khai các cải tiến này, đặc biệt trong môi trường hoàn toàn local.



Hình 3.1 Failure point của text-only one-stage RAG

3.1 Hạn chế của RAG Text-Only truyền thống

RAG truyền thống, thường được gọi là Naive RAG trong các khảo sát gần đây của Gao et al. [20], chủ yếu được thiết kế cho dữ liệu văn bản thuần. Cách tiếp cận này hoạt động hiệu quả đối với các tập tài liệu đơn giản, nhưng bộc lộ nhiều hạn chế khi áp dụng cho các tài liệu có cấu trúc phức tạp, bao gồm bảng biểu, hình ảnh và công thức toán học, những thành phần này xuất hiện phổ biến trong tài liệu khoa học, kỹ thuật và tài liệu doanh nghiệp.

Một trong những hạn chế phổ biến nhất của RAG text-only là mất mát thông tin bảng biểu. Trong pipeline RAG cơ bản, bảng thường bị chuyển đổi thành chuỗi văn bản phẳng, làm mất cấu trúc hàng cột cũng như các mối quan hệ ngữ nghĩa giữa các ô dữ liệu. Hệ quả là các giá trị số hoặc công thức trong bảng có thể bị tách rời khỏi ngữ cảnh, khiến mô hình ngôn ngữ khó suy luận chính xác. Theo phân tích thực nghiệm của Barnett et al. [31], việc xử lý bảng trong quy trình nạp dữ liệu là một thách thức lớn, thường dẫn đến việc thiếu hụt nội dung quan trọng khi bảng không được cấu trúc hoá đúng cách gây ra thiếu hụt nội dung quan trọng khi bảng không được cấu trúc hoá đúng cách, gây thất bại ở bước truy xuất. Khảo sát tổng hợp của Gupta et al. [32] cũng chỉ ra rằng pipeline

cơ bản của RAG gặp nhiều khó khăn trong việc bảo toàn cấu trúc bảng phức tạp, đặc biệt khi bảng chứa dữ liệu số hoặc biểu thức toán học, từ đó làm gia tăng nguy cơ ảo giác trong quá trình sinh câu trả lời.

Bên cạnh đó, mất ngữ cảnh hình ảnh là một hạn chế quan trọng khác của RAG truyền thống. Trong các hệ thống text-only, hình ảnh như biểu đồ, sơ đồ hoặc hình minh họa thường không được lập chỉ mục và biểu diễn vector một cách độc lập, mà chỉ được gián tiếp phản ánh thông qua phần văn bản mô tả xung quanh (nếu có). Do đó, khi truy vấn yêu cầu thông tin trực tiếp từ hình ảnh, hệ thống không thể cung cấp ngữ cảnh phù hợp, buộc mô hình sinh phải dựa nhiều vào tri thức tham số sẵn có. Theo khảo sát của Gao et al. về Multimodal RAG [33], cách tiếp cận này có thể bỏ lỡ một phần đáng kể thông tin thị giác trong tài liệu khoa học, nơi hình ảnh thường mang hàm lượng thông tin cao, và làm suy giảm hiệu năng tổng thể của hệ thống trong các nhiệm vụ hỏi-đáp dựa trên tài liệu đa phương thức.

Ngoài ra, việc xử lý công thức toán học cũng đặt ra thách thức lớn đối với RAG text-only. Trong quá trình parse tài liệu PDF, các công thức thường bị mất ký hiệu đặc thù, sai lệch định dạng hoặc bị chuyển thành văn bản không còn giữ được ý nghĩa toán học ban đầu. Điều này khiến mô hình ngôn ngữ gặp khó khăn trong việc hiểu vai trò của các biến và mối quan hệ toán học, đặc biệt trong các lĩnh vực khoa học và kỹ thuật.

Những hạn chế nêu trên không chỉ mang tính lý thuyết mà còn được phản ánh qua các benchmark thực nghiệm. Nghiên cứu của Gao et al. [20] và Mei et al. [33] cho thấy các hệ thống RAG thuần văn bản thường đạt kết quả thấp hơn so với các biến thể có xét đến đa phương thức trong các bộ dữ liệu hỏi đáp phổ biến, đặc biệt khi câu hỏi yêu cầu thông tin không chỉ nằm trong văn bản liên tục. Tuy nhiên, như sẽ được phân tích ở các mục tiếp theo, việc triển khai các kiến trúc này, đặc biệt trong môi trường hoàn toàn local, vẫn đặt ra nhiều thách thức đáng kể.

3.2 Vấn đề nhiễu ngữ cảnh trong truy xuất một giai đoạn

Single-stage retrieval hay truy xuất một giai đoạn là cách tiếp cận phổ biến trong các hệ thống RAG truyền thống, trong đó mô-đun truy xuất thực hiện tìm kiếm trực tiếp trên toàn bộ knowledge base để lựa chọn một tập các đoạn văn có độ tương đồng cao nhất

với truy vấn đầu vào [20]. Cách tiếp cận này có ưu điểm về tính đơn giản và dễ triển khai, tuy nhiên khi quy mô tập dữ liệu tăng lên, nó bộc lộ hạn chế đáng kể liên quan đến hiện tượng nhiễu ngữ cảnh.

Nguyên nhân chính của nhiễu ngữ cảnh xuất phát từ việc truy xuất trên toàn bộ tập tài liệu mà không có cơ chế khoanh vùng theo tài liệu hoặc chủ đề. Trong các tập dữ liệu lớn, đặc biệt là dữ liệu doanh nghiệp hoặc tài liệu kỹ thuật, nhiều đoạn văn có thể chia sẻ các từ khóa hoặc biểu thức ngôn ngữ tương tự nhưng mang ý nghĩa khác nhau về mặt ngữ cảnh. Khi sử dụng các mô hình dense embedding, dễ bị chi phối bởi các cách diễn đạt khác nhau và sự tương đồng ở mức từ ngữ, mô-đun truy xuất có xu hướng trả về các đoạn có vẻ liên quan về mặt ngôn từ nhưng không thực sự phù hợp với câu truy vấn. Điều này dẫn đến việc tập ngữ cảnh được đưa vào mô hình sinh chứa nhiều thông tin không cần thiết hoặc sai lệch.

Vấn đề này trở nên nghiêm trọng hơn khi các chunk truy xuất được ghép lại thành một ngữ cảnh dài để đưa vào mô hình ngôn ngữ lớn. Nghiên cứu của Liu et al. [2] chỉ ra rằng LLM thường khai thác hiệu quả nhất thông tin nằm ở phần đầu và phần cuối của ngữ cảnh dài, trong khi các đoạn ở giữa có xu hướng bị bỏ qua. Trong bối cảnh RAG một giai đoạn truy xuất, ngữ cảnh thường bao gồm nhiều chunk không đồng nhất về mức độ liên quan, hiện tượng này làm suy giảm khả năng sử dụng thông tin thực sự quan trọng, ngay cả khi retriever đã truy xuất đúng một phần nội dung liên quan.

Hệ quả trực tiếp của nhiễu ngữ cảnh là sự suy giảm độ chính xác và độ tin cậy của câu trả lời được sinh ra. Khi ngữ cảnh đầu vào chứa các đoạn văn không phù hợp, mô hình ngôn ngữ có thể dựa trên các thông tin sai lệch để suy luận, từ đó làm gia tăng hiện tượng ảo giác và giảm mức độ gắn kết giữa câu trả lời và nguồn tri thức gốc. Khảo sát của Gao et al. [20] về độ tin cậy của hệ thống RAG cũng chỉ ra rằng single-stage retrieval gặp nhiều khó khăn trong việc duy trì faithfulness khi áp dụng cho các kịch bản hỏi đáp trên tập tài liệu lớn và đa dạng.

Những hạn chế này càng bộc lộ rõ rệt trong các hệ thống RAG đa phương thức. Do không phân biệt rõ giữa các loại dữ liệu khác nhau, single-stage retrieval thường ưu tiên truy xuất các đoạn văn bản thuần, trong khi bỏ qua hoặc không khai thác hiệu quả các

thành phần như hình ảnh, bảng biểu hoặc công thức liên quan. Theo khảo sát của Gao et al. [20], các kiến trúc RAG áp dụng truy xuất đa giai đoạn hoặc có bước khoanh vùng trước thường cho thấy khả năng kiểm soát nhiễu ngữ cảnh tốt hơn so với cách tiếp cận single-stage, đặc biệt trong các kịch bản xử lý tài liệu phức tạp.

3.3 Thách thức khi xây dựng pipeline hoàn toàn Local

Triển khai RAG hoàn toàn local mang lại lợi ích rõ rệt về quyền riêng tư, kiểm soát dữ liệu và khả năng triển khai trong môi trường nội bộ. Tuy nhiên, từ nghiên cứu đến triển khai thực tế trên workstation phổ thông, hệ thống thường đối mặt đồng thời hai nhóm ràng buộc: ràng buộc hạ tầng, hiệu năng và ràng buộc mô hình, dữ liệu, đặc biệt khi xử lý tài liệu đa phương thức. Khảo sát về tối ưu suy luận LLM của Zhou et al. [34] cho thấy chi phí suy luận không chỉ nằm ở trọng số mô hình mà còn ở KV cache và cơ chế tập trung, khiến việc chạy ổn định trên tài nguyên hạn chế trở thành một nút thắt quan trọng trong các hệ thống RAG local.

3.3.1 Hạn chế phần cứng

Một trong những ràng buộc cốt lõi của triển khai RAG local là giới hạn về bộ nhớ và hiệu năng tính toán. Khảo sát của Zhou et al. [34] về suy luận mô hình ngôn ngữ lớn chỉ ra rằng chi phí bộ nhớ không chỉ đến từ trọng số mô hình mà còn từ bộ nhớ dành cho cơ chế tập trung, đặc biệt là key-value cache, vốn tăng tuyến tính theo độ dài ngữ cảnh và số lớp của mô hình. Trong bối cảnh RAG, nơi mô hình thường phải xử lý ngữ cảnh dài bao gồm nhiều đoạn văn được truy xuất, áp lực bộ nhớ trở nên đặc biệt nghiêm trọng trên các GPU phổ thông.

Để đáp ứng ràng buộc phần cứng, nhiều hệ thống local buộc phải sử dụng các kỹ thuật nén và lượng tử hóa mô hình. Tuy nhiên, các kết quả thực nghiệm gần đây cho thấy rằng lượng tử hóa, đặc biệt ở mức 4-bit, có thể gây suy giảm đáng kể chất lượng suy luận trong các tác vụ yêu cầu xử lý ngữ cảnh dài. Nghiên cứu của Mekala et al. [35] cho thấy hiệu năng của các mô hình lượng tử hóa giảm rõ rệt khi độ dài context tăng, ảnh hưởng trực tiếp đến khả năng tận dụng ngữ cảnh trong các hệ thống RAG. Điều này đặt ra một bài toán đánh đổi giữa khả năng triển khai trên phần cứng hạn chế và độ chính xác của câu trả lời sinh ra.

Bên cạnh bộ nhớ, độ trễ end-to-end của pipeline RAG local cũng là một thách thức đáng kể. Không giống các hệ thống chỉ thực hiện suy luận đơn lẻ, RAG bao gồm nhiều bước liên tiếp như tiền xử lý tài liệu, biểu diễn vector, truy xuất, reranking và sinh câu trả lời. Các khảo sát về hạ tầng phục vụ LLM cho thấy rằng việc kết hợp nhiều bước tính toán nặng trong một pipeline làm gia tăng độ trễ và gây khó khăn trong việc đạt được trải nghiệm tương tác thời gian thực trên hạ tầng local [36].

3.3.2 Thách thức dữ liệu đa phương thức

Khi tài liệu đầu vào chứa các thành phần không thuần văn bản như bảng biểu, hình ảnh, bố cục trang và công thức toán học, độ phức tạp của pipeline RAG tăng lên đáng kể. Khảo sát của Abootorabi et al. [37] hay khảo sát của Mei et al. [33] về Multimodal Retrieval-Augmented Generation (MRAG) đều thống nhất rằng việc mở rộng RAG sang đa phương thức không chỉ là vấn đề bổ sung thêm embedding cho các loại dữ liệu khác nhau, mà còn liên quan đến bài toán liên kết chéo giữa các modality, bảo toàn cấu trúc và kiểm soát sai số tích lũy trong toàn bộ pipeline.

Một khó khăn quan trọng là hiện tượng lan truyền sai số từ các bước tiền xử lý. Khảo sát MRAG của Abootorabi et al. [37] chỉ ra rằng lỗi trong OCR, nhận dạng bảng hoặc trích xuất bố cục có thể dẫn đến sai lệch trong biểu diễn ngữ nghĩa, từ đó làm giảm chất lượng truy xuất và sinh câu trả lời ở các bước sau. Vấn đề này đặc biệt nghiêm trọng trong môi trường local, nơi việc sử dụng các mô hình lớn, được tinh chỉnh chuyên sâu cho từng modality, thường bị hạn chế bởi tài nguyên phần cứng.

Ngoài ra, việc xử lý công thức toán học và bảng biểu vẫn được xem là bài toán mở trong cộng đồng nghiên cứu RAG. Các khảo sát của Mei et al. [33] và Gao et al. [38] cho thấy chưa tồn tại một phương pháp biểu diễn thống nhất cho các thành phần này có thể hoạt động ổn định trên nhiều loại tài liệu khác nhau, đặc biệt khi yêu cầu vừa chính xác về ngữ nghĩa vừa hiệu quả về tính toán. Do đó, ngay cả trong các hệ thống MRAG được đề xuất gần đây, hiệu quả xử lý dữ liệu đa phương thức vẫn phụ thuộc mạnh vào đặc điểm của tập dữ liệu và thiết kế pipeline cụ thể.

3.3.3 Định hướng giải pháp

Trước các hạn chế nêu trên, hai hướng cải tiến chính là kiến trúc đa góc nhìn và truy xuất đa giai đoạn. Tuy nhiên, khảo sát Abootorabi et al. [37] và Gao et al. [38], đều nhấn mạnh rằng cả hai hướng này vẫn đang trong quá trình hoàn thiện và chưa đạt đến mức độ chuẩn hóa cho triển khai rộng rãi.

Đối với multimodal RAG, việc tách riêng và xử lý độc lập các loại dữ liệu như văn bản, bảng biểu và hình ảnh giúp giảm mất mát thông tin so với cách tiếp cận thuần văn bản. Mặc dù vậy, nghiên cứu của Abootorabi et al. [37] và Gao et al. [38] chỉ ra rằng hiệu quả của phương pháp này phụ thuộc lớn vào chất lượng liên kết giữa các modality và chiến lược hợp nhất thông tin, vốn vẫn là những vấn đề nghiên cứu đang được tiếp tục khai thác.

Song song với đó, truy xuất đa giai đoạn được đề xuất nhằm giảm nhiễu cảnh và cải thiện độ chính xác của thông tin được đưa vào mô hình sinh. Công trình của Ren et al. [39] và Zhu et al. [40] cho thấy rằng việc bổ sung các bước truy xuất hoặc lọc nhiễu cảnh trung gian có thể cải thiện đáng kể chất lượng truy xuất và hiệu năng sinh câu trả lời so với các hệ thống single-stage truyền thống. Các kết quả này cung cấp bằng chứng thực nghiệm cho thấy multi-stage retrieval là một hướng tiếp cận tiềm năng, đặc biệt trong các hệ thống RAG xử lý tập tài liệu lớn và phức tạp.

Trong phạm vi đề án này, việc triển khai hệ thống được thực hiện với mục tiêu kết hợp xử lý đa góc nhìn và truy xuất đa giai đoạn trong môi trường hoàn toàn local, đồng thời phân tích các đánh đổi về độ chính xác, độ trễ và độ tin cậy khi áp dụng các kỹ thuật này trên hạ tầng phần cứng giới hạn.

CHƯƠNG 4 TRIỂN KHAI XÂY DỰNG HỆ THỐNG RAG HOÀN TOÀN LOCAL

4.1 Tổng quan kiến trúc hệ thống

4.1.1 Mục tiêu thiết kế kiến trúc

Kiến trúc của hệ thống được thiết kế nhằm đáp ứng các yêu cầu triển khai RAG trong bối cảnh hoàn toàn local, nơi tài nguyên phần cứng và hạ tầng bị giới hạn nhưng vẫn đòi

hỏi khả năng xử lý tài liệu phức tạp và đảm bảo tính kiểm chứng của kết quả sinh. Do đó, mục tiêu thiết kế không tập trung vào việc tối đa hóa hiệu năng tuyệt đối, mà hướng đến việc cân bằng giữa tính khả thi khi triển khai, độ tin cậy của truy xuất và khả năng mở rộng về mặt biểu diễn tri thức.

Mục tiêu đầu tiên của kiến trúc là đảm bảo nguyên tắc local-first, theo đó toàn bộ các thành phần cốt lõi của hệ thống, bao gồm mô hình sinh ngôn ngữ, mô hình embedding, cơ chế truy xuất và các kho lưu trữ dữ liệu, đều được vận hành trong hạ tầng nội bộ. Cách tiếp cận này nhằm đáp ứng các kịch bản sử dụng yêu cầu kiểm soát dữ liệu và hạn chế phụ thuộc vào dịch vụ cloud bên ngoài, đồng thời phản ánh các ràng buộc thực tế trong môi trường triển khai tại doanh nghiệp hoặc tổ chức nghiên cứu.

Mục tiêu thứ hai là xây dựng một kiến trúc biểu diễn tri thức đa góc nhìn, trong đó các thành phần tri thức khác nhau của tài liệu, bao gồm văn bản, bảng biểu, hình ảnh và công thức toán học, được xử lý và lưu trữ như các thực thể độc lập thay vì bị ép về dạng văn bản thuần. Thiết kế này xuất phát từ thực tế rằng các tài liệu khoa học và doanh nghiệp thường chứa thông tin quan trọng ngoài văn bản liên tục, và việc mất cấu trúc ở giai đoạn tiền xử lý có thể làm suy giảm đáng kể hiệu quả truy xuất cũng như độ trung thực của câu trả lời sinh ra.

Mục tiêu thứ ba của kiến trúc là giảm thiểu nhiễu cảnh trong quá trình truy xuất bằng cách áp dụng chiến lược truy xuất hai giai đoạn. Thay vì truy xuất trực tiếp trên toàn bộ tập dữ liệu, hệ thống được thiết kế để thực hiện một bước định tuyến tri thức ở cấp tài liệu trước, sau đó giới hạn phạm vi truy xuất chi tiết trong tài liệu đã được lựa chọn. Cách tổ chức này phù hợp với đặc thù của bài toán hỏi đáp trên tập tài liệu lớn, nơi nhiều tài liệu có nội dung và thuật ngữ tương đồng, và cho phép kiểm soát tốt hơn không gian tìm kiếm cũng như độ dài ngữ cảnh đưa vào mô hình sinh.

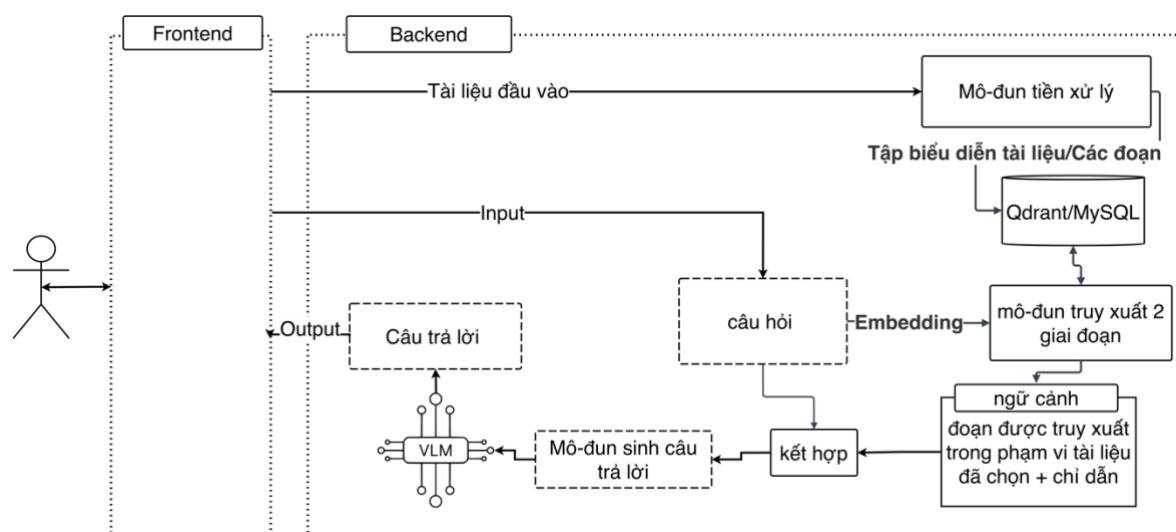
Cuối cùng, kiến trúc được thiết kế theo hướng mô-đun và có khả năng cấu hình, cho phép thay thế hoặc điều chỉnh từng thành phần như mô hình embedding, cơ chế truy xuất, hoặc mô hình sinh mà không ảnh hưởng đến toàn bộ pipeline. Mục tiêu này nhằm tạo điều kiện cho việc thực nghiệm, so sánh và phân tích các lựa chọn thiết kế khác nhau

trong khuôn khổ đề án, đồng thời đảm bảo rằng hệ thống có thể được mở rộng hoặc tinh chỉnh trong các nghiên cứu tiếp theo.

Bảng 4.1 Mục tiêu thiết kế và thành phần đáp ứng

Mục tiêu thiết kế	Thành phần kiến trúc đáp ứng
Local-first	• VLM với LMStudio • Qdrant • MySQL • MongoDB • Qdrant/bm25 • BAAI/bge-small-en-v1.5 • clip-ViT-B-32 • jinaai/jina-embeddings-v3 • BAAI/bge-reranker-v2-m3
Multi-view	Đoạn văn, hình ảnh, bảng, công thức
Giảm nhiễu ngữ cảnh	Truy xuất hai giai đoạn
Các mô-đun	Bộ tiền xử lý/ bộ truy xuất/ bộ sinh câu trả lời

4.1.2 Sơ đồ kiến trúc tổng quát



Hình 4.1 Minh họa kiến trúc tổng quát của hệ thống

Ở mức tổng thể, hệ thống được chia thành hai luồng chính: luồng nạp tri thức (ingest) và luồng hỏi đáp. Hai luồng này cùng chia sẻ các kho lưu trữ tri thức nhưng được thiết kế tách biệt nhằm đảm bảo tính mô-đun và khả năng mở rộng của hệ thống.

Trong luồng ingest, tài liệu do người dùng cung cấp thông qua frontend được chuyển đến backend và đưa vào mô-đun tiền xử lý. Mô-đun này chịu trách nhiệm phân tích cấu trúc tài liệu, trích xuất nội dung và xây dựng các biểu diễn tri thức ở nhiều mức độ khác nhau. Kết quả của quá trình này là tập các đơn vị tri thức đã được chuẩn hóa, bao gồm tập biểu diễn tài liệu và các đoạn tri thức ở cấp chi tiết. Các biểu diễn này sau đó được lưu trữ kết hợp trong các kho dữ liệu bên dưới, bao gồm cơ sở dữ liệu vector và cơ sở dữ liệu quan hệ, tạo nền tảng cho các bước truy xuất ở pha hỏi đáp. Thiết kế này cho phép hệ thống tách biệt rõ ràng giữa giai đoạn xây dựng tri thức và giai đoạn khai thác tri thức, đồng thời tránh việc phải xử lý lại tài liệu gốc trong mỗi truy vấn.

Trong luồng hỏi, câu hỏi của người dùng được gửi từ frontend đến backend và được biểu diễn dưới dạng truy vấn ngữ nghĩa. Truy vấn này được đưa vào mô-đun truy xuất hai giai đoạn. Ở giai đoạn thứ nhất, hệ thống thực hiện định tuyến tri thức ở cấp tài liệu nhằm xác định tài liệu hợp nhất với câu hỏi. Ở giai đoạn thứ hai, quá trình truy xuất chi tiết được thực hiện trong phạm vi tài liệu đã được lựa chọn, giúp giới hạn không gian tìm kiếm và giảm nhiễu ngữ cảnh. Kết quả của bước này là một tập các đoạn tri thức liên quan, được coi là ngữ cảnh truy xuất cho bước sinh câu trả lời.

Tập ngữ cảnh truy xuất sau đó được chuyển đến mô-đun xây dựng ngữ cảnh và sinh đầu vào, nơi các đoạn tri thức được kết hợp và định dạng phù hợp để đưa vào mô hình sinh. Mô hình sinh dự án sử dụng là mô hình ngôn ngữ thị giác, thực hiện sinh câu trả lời dựa trên ngữ cảnh đã được cung cấp. Kết quả sinh được trả về backend và chuyển tiếp đến frontend để hiển thị cho người dùng. Đồng thời, hệ thống có thể lưu trữ thông tin hội thoại và kết quả truy vấn phục vụ cho các phiên tương tác tiếp theo.

Về mặt lưu trữ, kiến trúc sử dụng cách tiếp cận lưu trữ lai, trong đó cơ sở dữ liệu vector được sử dụng để lưu trữ các embedding phục vụ truy xuất ngữ nghĩa với dữ liệu văn bản và hình ảnh, còn cơ sở dữ liệu quan hệ được sử dụng để tìm kiếm chính xác với văn bản, hình ảnh và bảng biểu. Cách tổ chức này cho phép kết hợp ưu điểm của truy xuất ngữ

nghĩa và truy xuất dựa trên cấu trúc, đồng thời tạo điều kiện cho việc lọc, phân loại và kiểm soát phạm vi truy xuất ở các bước sau.

4.2 Dữ liệu và phạm vi thí nghiệm

Dữ liệu đóng vai trò trung tâm trong việc đánh giá hiệu quả của các hệ thống RAG, đặc biệt trong bối cảnh xử lý tri thức miền riêng và tài liệu có cấu trúc phức tạp. Trong dự án này, tập dữ liệu được lựa chọn có cấu trúc phức tạp nhưng tương đối rõ ràng.

Nhóm thứ nhất bao gồm các tài liệu nội bộ doanh nghiệp, đại diện cho dữ liệu thực tế trong môi trường triển khai nội bộ. Nhóm thứ hai bao gồm các tài liệu khoa học và tập dữ liệu thử nghiệm, được sử dụng nhằm kiểm chứng khả năng xử lý đa phương thức và suy luận trên các tài liệu có cấu trúc học thuật phức tạp.

4.2.1 Nhóm dữ liệu nội bộ doanh nghiệp

Nhóm dữ liệu nội bộ trong dự án bao gồm các tài liệu PDF là các thông báo nội bộ, quyết định điều hành, văn bản liên quan đến hoạt động kinh doanh và dịch vụ của công ty Cổ Phần Giải Pháp Công Nghệ Thông Tin Quốc Tế (InterITS). Các tài liệu này được phát hành dưới dạng văn bản chính thức, có bố cục và cấu trúc tương đối rõ ràng, phản ánh đặc trưng của tài liệu hành chính và nghiệp vụ trong thực tế.

Về mặt cấu trúc, các tài liệu nội bộ thường bao gồm tiêu đề, phần mở đầu, các mục và tiểu mục được phân tách rõ ràng, cùng với nội dung trình bày theo dạng đoạn văn, danh sách hoặc bảng biểu. Một số tài liệu có kèm theo bảng, điều khoản hoặc thông tin định lượng, trong khi các tài liệu khác có thể chứa chữ ký, con dấu hoặc hình ảnh scan. Những đặc điểm này khiến dữ liệu doanh nghiệp không hoàn toàn là văn bản thuần túy, mà mang tính bán cấu trúc, đòi hỏi hệ thống RAG phải bảo toàn được ngữ cảnh và mối quan hệ giữa các thành phần trong tài liệu.

Việc lựa chọn nhóm dữ liệu này nhằm phản ánh đúng kịch bản ứng dụng mục tiêu của hệ thống, nơi yêu cầu đặt ra không chỉ là trả lời câu hỏi dựa trên nội dung văn bản, mà còn phải đảm bảo tính chính xác, khả năng kiểm chứng và khả năng truy vết lại nguồn thông tin trong các tài liệu chính thức. Đồng thời, các tài liệu doanh nghiệp thường có nội dung tương đối tương đồng về mặt từ vựng và chủ đề, tạo ra thách thức đáng kể cho

các hệ thống RAG truyền thống trong việc giảm nhiễu cảnh và định tuyến đúng tài liệu liên quan.

4.2.2 Dữ liệu thử nghiệm

Bên cạnh dữ liệu doanh nghiệp, dự án sử dụng thêm một tập dữ liệu thử nghiệm bao gồm các bài báo khoa học và tài liệu học thuật dưới dạng PDF. Nhóm dữ liệu này được lựa chọn nhằm đánh giá khả năng của hệ thống trong việc xử lý các tài liệu có cấu trúc phức tạp và đa phương thức, vốn là thách thức lớn đối với các hệ thống RAG truyền thống.

Các bài báo khoa học thường có cấu trúc chặt chẽ với nhiều cấp độ tiêu đề. Ngoài nội dung văn bản, các tài liệu này còn chứa bảng kết quả thực nghiệm, hình vẽ minh họa, sơ đồ kiến trúc và công thức toán. Sự kết hợp của nhiều loại modality trong cùng một tài liệu khiến việc xử lý theo hướng text-only dễ dẫn đến mất mát thông tin hoặc sai lệch nhiễu cảnh khi truy xuất.

Việc sử dụng nhóm dữ liệu khoa học trong dự án đóng vai trò như một tập kiểm thử nội bộ để đánh giá khả năng hỏi đáp với nhiều loại dữ liệu của hệ thống. Thông qua các tài liệu này, hệ thống được kiểm chứng về khả năng phân tách và lưu trữ tri thức theo nhiều dạng biểu diễn khác nhau, cũng như khả năng truy xuất và kết hợp các nguồn thông tin để sinh câu trả lời.

Sự kết hợp giữa dữ liệu doanh nghiệp và dữ liệu khoa học cho phép dự án đánh giá hệ thống, đảm bảo rằng các kết quả trình bày ở các mục đánh giá sau phản ánh được năng lực tổng quát của hệ thống, thay vì chỉ tối ưu cho một loại dữ liệu cụ thể.

4.3 Cấu hình phần cứng

Trong dự án này triển khai và thử nghiệm trên máy tính cá nhân sử dụng phần cứng phổ thông ở phân khúc người dùng cá nhân và nghiên cứu độc lập. Việc lựa chọn môi trường triển khai này nhằm phản ánh kịch bản triển khai hoàn toàn local, không phụ thuộc vào hạ tầng điện toán đám mây hay các tài nguyên chuyên dụng từ bên thứ ba.

Máy tính sử dụng bộ xử lý trung tâm đa nhân, bộ nhớ hệ thống dung lượng lớn và bộ xử lý đồ họa rời hỗ trợ tăng tốc tính toán. Hệ thống lưu trữ sử dụng ổ cứng thể rắn chuẩn

NVMe nhằm đảm bảo tốc độ truy xuất dữ liệu trong quá trình tiền xử lý tài liệu và truy xuất tri thức. Toàn bộ môi trường phần mềm được vận hành trên hệ điều hành Linux, phù hợp với các công cụ mã nguồn mở và các framework được sử dụng trong dự án.

Bảng 4.2 Cấu hình phần cứng được sử dụng cho dự án.

Thành phần	Thông số
Loại thiết bị	Máy tính cá nhân
CPU	AMD ryzen 7 5800x
RAM	48GB
GPU	NVIDIA RTX 3080
Dung lượng VRAM	10GB
Ổ cứng	NVMe SSD 1T
Hệ điều hành	Linux Mint
Môi trường triển khai	Local (on-premise)

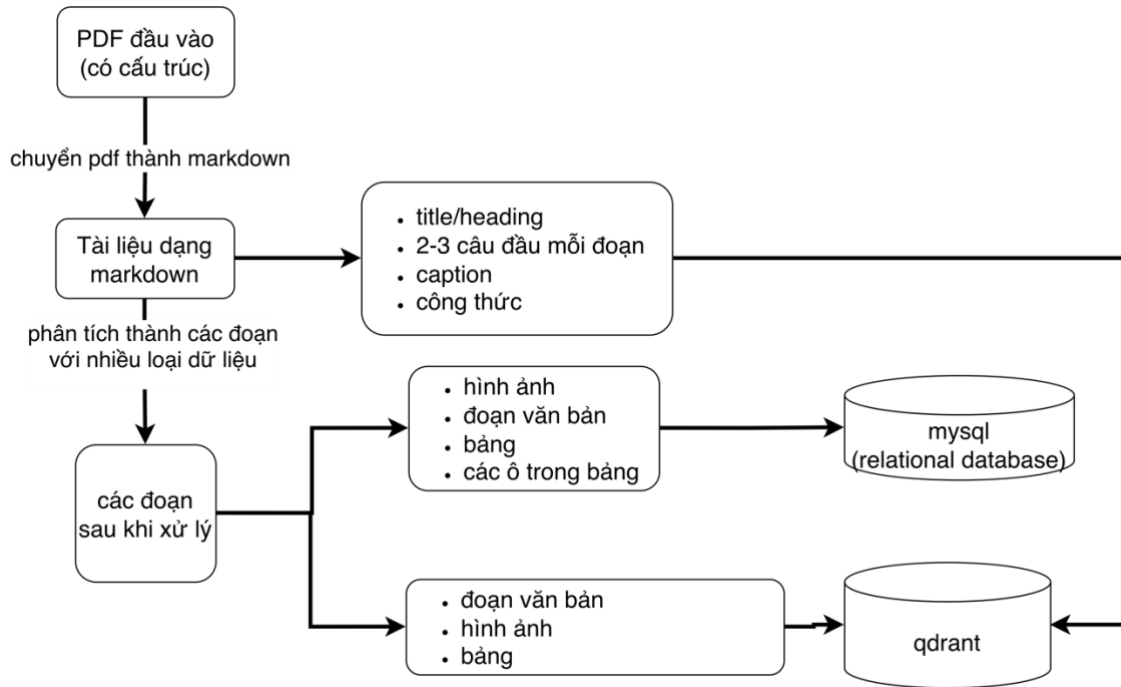
Cấu hình phần cứng trên được sử dụng xuyên suốt quá trình xây dựng, triển khai và kiểm thử hệ thống, bao gồm các giai đoạn tiền xử lý dữ liệu, lưu trữ tri thức, truy xuất và sinh câu trả lời. Các đặc điểm và giới hạn liên quan đến cấu hình này sẽ được phân tích và thảo luận cụ thể hơn trong các phần đánh giá và thảo luận kết quả ở các mục sau.

4.4 Tiền xử lý dữ liệu

Tiền xử lý dữ liệu đóng vai trò then chốt trong toàn bộ pipeline của hệ thống, bởi chất lượng của các bước truy xuất và sinh câu trả lời phụ thuộc trực tiếp vào mức độ bảo toàn cấu trúc và ngữ nghĩa của tài liệu đầu vào. Trong các hệ thống RAG truyền thống, việc xử lý tài liệu theo hướng text-only hoặc cắt ngẫu nhiên theo số token thường dẫn đến mất mát thông tin, phá vỡ mối quan hệ ngữ nghĩa giữa các thành phần trong tài liệu, từ đó làm gia tăng nhiễu ngữ cảnh ngay từ giai đoạn đầu.

Xuất phát từ đặc thù dữ liệu của dự án, bao gồm các tài liệu doanh nghiệp và bài báo khoa học có cấu trúc rõ ràng, chứa bảng biểu, hình ảnh và công thức toán học, hệ thống tập trung thiết kế pipeline tiền xử lý với mục tiêu giữ nguyên cấu trúc logic của tài liệu

càng nhiều càng tốt. Thay vì coi tài liệu chỉ là chuỗi văn bản phẳng, pipeline tiền xử lý được xây dựng nhằm chuyển đổi tài liệu sang một biểu diễn trung gian có cấu trúc, từ đó phục vụ cho các bước xây dựng tri thức, chunking và truy xuất ở các giai đoạn sau.



Hình 4.2 Minh họa luồng tiền xử lý dữ liệu.

Toàn bộ quy trình tiền xử lý được tổ chức thành các bước tuần tự, bắt đầu từ việc phân tích tài liệu PDF và chuyển đổi sang Markdown, tiếp theo là xây dựng tập biểu diễn tài liệu ở cấp tài liệu, thực hiện phân tích theo cấu trúc, và cuối cùng là tách các đơn vị tri thức theo nhiều dạng dữ liệu. Cách tiếp cận này cho phép hệ thống giảm nhiễu từ sớm, đồng thời tạo nền tảng cho chiến lược truy xuất hai giai đoạn.

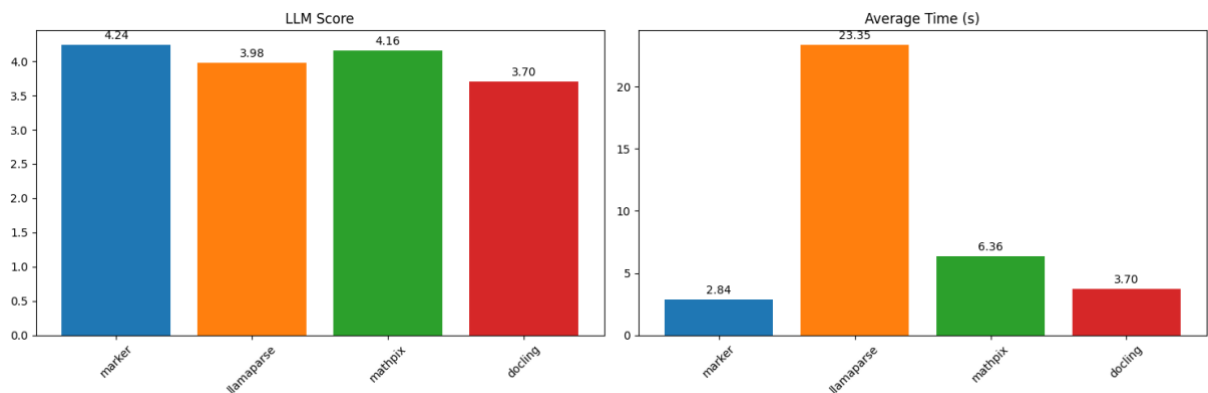
4.4.1 Chuyển đổi PDF thành markdown với Marker

Bước đầu tiên trong pipeline tiền xử lý là chuyển đổi tài liệu PDF đầu vào sang dạng Markdown có cấu trúc. Mục tiêu của bước này không chỉ là trích xuất nội dung văn bản, mà còn là bảo toàn tối đa bố cục, tiêu đề, đoạn văn, bảng biểu, hình ảnh và công thức toán học xuất hiện trong tài liệu gốc.

Trong hệ thống, quá trình chuyển đổi PDF được thực hiện thông qua một bộ chuyển đổi độc lập, được backend điều phối bằng cơ chế gọi tiến trình con. Việc tách mô-đun chuyển đổi thành một tiến trình độc lập giúp hệ thống linh hoạt trong cấu hình, dễ dàng

mở rộng hoặc thay thế công cụ phân tích PDF khi cần thiết, mà không ảnh hưởng đến logic chính của backend.

Trong các công cụ chuyển đổi PDF sang Markdown hiện nay, Marker đã được đánh giá và cho thấy hiệu năng xử lý vượt trội so với các công cụ khác như docling, mathpix, llamaparse, đồng thời giữ nguyên cấu trúc bảng, công thức toán học, hình ảnh hơn các công cụ khác [41]. Marker hỗ trợ OCR cho tài liệu scan và đa ngôn ngữ, cho phép xử lý tốt các tài liệu học thuật phức tạp chứa nhiều thành phần phi văn bản. Những đặc điểm này làm cho Marker phù hợp làm công cụ chuyển đổi tài liệu chính trong mô-đun tiền xử lý tài liệu cho hệ thống. Các tham số quan trọng được sử dụng trong quá trình chuyển đổi PDF thành markdown bao gồm tùy chọn sử dụng mô hình ngôn ngữ để cải thiện chất lượng trích xuất bố cục và định dạng, cơ chế ép buộc OCR đối với các tài liệu dạng scan, xử lý lại công thức toán học dạng inline nhằm giảm lỗi định dạng, chỉ định sử dụng tiếng Việt cho OCR và xuất metadata hình ảnh dưới dạng cấu trúc riêng biệt. Những tham số này cho phép pipeline thích ứng với nhiều loại PDF khác nhau, từ tài liệu văn bản thuần cho đến tài liệu scan hoặc tài liệu học thuật chứa nhiều công thức và hình vẽ.



Hình 4.3 Hiệu năng của marker so với các framework khác

Kết quả của bước chuyển đổi là tệp Markdown chính chứa nội dung đã được chuẩn hóa, thư mục hình ảnh kèm theo metadata phục vụ cho việc xây dựng view hình ảnh, và trong một số cấu hình pipeline, các tệp mô tả cấu trúc hoặc tiêu đề của tài liệu. Các kết quả này được tổ chức theo từng tài liệu, đóng vai trò là đầu vào trực tiếp cho bước xây dựng cấu trúc tri thức và chunking ở các mục tiếp theo.

Việc sử dụng Markdown làm biểu diễn trung gian giúp hệ thống duy trì được mối quan hệ phân cấp giữa các thành phần trong tài liệu, đồng thời tạo điều kiện thuận lợi cho

việc phân tách tri thức theo cấu trúc thay vì theo số token cố định. Đây là nền tảng quan trọng cho các chiến lược phân tích và xử lý đa kiểu dữ liệu được trình bày trong các mục sau.

4.4.2 Xây dựng tập biểu diễn tài liệu

Sau khi tài liệu được chuyển đổi sang dạng Markdown có cấu trúc, hệ thống tiến hành xây dựng biểu diễn tri thức ở cấp tài liệu nhằm phục vụ cho giai đoạn truy xuất đầu tiên trong pipeline hai giai đoạn. Mục tiêu của bước này là tạo ra một tập dữ liệu đại diện cho từng tài liệu, đóng vai trò như một điểm neo ở cấp độ tài liệu cho quá trình định tuyến truy vấn.

Biểu diễn tri thức ở cấp tài liệu bao gồm các thành phần sau. Thứ nhất, hệ thống lưu trữ cấu trúc tiêu đề của tài liệu. Thông tin này cho phép truy vấn ở giai đoạn đầu nắm bắt được chủ đề tổng thể và phạm vi nội dung của tài liệu, thay vì chỉ dựa trên các đoạn văn rời rạc. Thứ hai, hệ thống sinh và lưu trữ tóm tắt ngắn cho các đoạn, thường bao gồm một đến hai câu đầu của mỗi đơn vị tri thức, nhằm tạo ra biểu diễn cô đọng cho nội dung chi tiết phía dưới. Các tóm tắt này giúp tăng khả năng phân biệt giữa các tài liệu có từ vựng tương đồng nhưng khác về ngữ cảnh sử dụng.

Bên cạnh đó, tập dữ liệu đại diện cho văn bản này còn lưu các thông tin mô tả liên quan đến các thành phần phi văn bản trong tài liệu. Đối với các công thức toán học, hệ thống ghi nhận các biểu thức hoặc ký hiệu xuất hiện trong tài liệu nhằm phản ánh mức độ liên quan đến các nội dung kỹ thuật hoặc học thuật. Đối với hình ảnh và bảng biểu, hệ thống lưu trữ chú thích tương ứng, đóng vai trò như một biểu diễn ngữ nghĩa ngắn gọn cho nội dung thị giác hoặc dữ liệu có cấu trúc. Việc tích hợp các thông tin này giúp biểu diễn tri thức ở cấp tài liệu không bị giới hạn trong văn bản thuần.

4.4.3 Phân tích theo cấu trúc

Sau khi tài liệu được chuyển đổi sang dạng Markdown có cấu trúc và xây dựng biểu diễn tri thức ở cấp tài liệu, hệ thống tiến hành phân tách nội dung thành các đơn vị tri thức nhỏ hơn thông qua bước chunking. Khác với các phương pháp chunking truyền thống dựa trên việc cắt văn bản theo số lượng token cố định, hệ thống áp dụng chiến

lược chunking theo cấu trúc, nhằm bảo toàn ngữ nghĩa và mối quan hệ logic giữa các thành phần trong tài liệu.

Cụ thể, quá trình chunking được thực hiện dựa trên cấu trúc Markdown của tài liệu, trong đó các tiêu đề và tiểu mục đóng vai trò là các mốc phân chia tự nhiên. Hệ thống ưu tiên phân tách nội dung theo đường dẫn tiêu đề, phản ánh đầy đủ ngữ cảnh phân cấp của mỗi đoạn văn trong tài liệu. Nhờ đó, mỗi chunk không chỉ chứa nội dung văn bản thuần mà còn gắn liền với vị trí logic của nó trong cấu trúc tổng thể của tài liệu.

Bên cạnh việc tôn trọng cấu trúc tiêu đề, hệ thống giữ nguyên các đoạn văn ở mức độ hợp lý nhằm tránh làm gãy các định nghĩa, điều khoản hoặc lập luận liên tục. Trong cấu hình môi trường cho phép cấu hình ngưỡng token tối đa cho 1 chunk và cấu hình số token overlap để tránh mất ngữ nghĩa. Cách tiếp cận này đặc biệt quan trọng đối với các tài liệu doanh nghiệp và tài liệu học thuật, nơi ý nghĩa của một đoạn nội dung thường phụ thuộc mạnh vào ngữ cảnh trước và sau. Việc chia nhỏ văn bản một cách cơ học theo số token có thể dẫn đến mất mạch logic, từ đó làm giảm hiệu quả truy xuất và gia tăng nguy cơ sinh câu trả lời thiếu chính xác.

Việc áp dụng chunking theo cấu trúc mang lại nhiều lợi ích so với phương pháp chunking theo cửa sổ cố định. Thứ nhất, các chunk giữ được tính toàn vẹn về mặt ngữ nghĩa, giúp mô hình sinh dễ dàng khai thác thông tin liên quan mà không cần suy đoán thêm ngữ cảnh bị thiếu. Thứ hai, lượng nhiễu trong kết quả truy xuất được giảm đáng kể, do mỗi chunk đại diện cho một đơn vị nội dung có ý nghĩa rõ ràng.

Trên cơ sở các chunk đã được phân tách theo cấu trúc, hệ thống tiếp tục mở rộng bước tiền xử lý bằng cách phân loại và xử lý các đơn vị tri thức theo nhiều dạng biểu diễn khác nhau. Mỗi chunk được gán một loại dữ liệu tương ứng, phản ánh bản chất của nội dung mà nó đại diện, bao gồm văn bản, bảng biểu, hình ảnh hoặc công thức toán học. Cách tổ chức này cho phép hệ thống khai thác hiệu quả các nguồn thông tin dị thể trong các bước truy xuất và sinh câu trả lời ở giai đoạn sau.

4.5 Lưu trữ dữ liệu lại

Trong các hệ thống RAG, thiết kế kho lưu trữ dữ liệu có ảnh hưởng trực tiếp đến hiệu quả truy xuất, khả năng mở rộng và mức độ kiểm soát ngữ cảnh. Thay vì sử dụng một

cơ sở dữ liệu duy nhất cho toàn bộ pipeline, hệ thống áp dụng cách tiếp cận lưu trữ lai, kết hợp giữa cơ sở dữ liệu quan hệ, cơ sở dữ liệu vector. Cách tổ chức này cho phép khai thác ưu điểm riêng của từng loại kho lưu trữ.

4.5.1 Vai trò từng kho

Cơ sở dữ liệu MySQL được sử dụng để lưu trữ các chunk văn bản có cấu trúc cùng với metadata tương ứng. Nhờ khả năng hỗ trợ truy vấn có điều kiện và full-text search, MySQL đóng vai trò chính trong các thao tác truy xuất dựa trên từ khóa, lọc theo định danh tài liệu và loại dữ liệu.

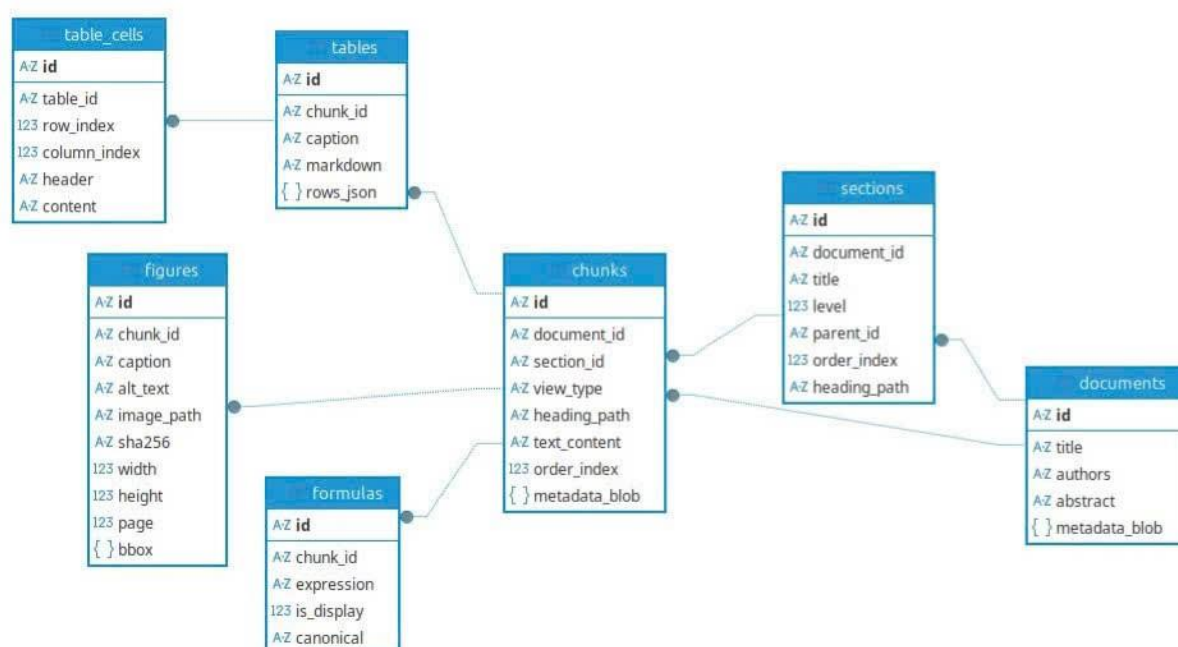
Qdrant được lựa chọn làm vector database do hiệu năng truy xuất cao dựa trên chỉ mục HNSW, cùng khả năng hỗ trợ tìm kiếm theo metadata ngay trong quá trình tìm kiếm vector [42], [43], [44]. Thiết kế này đặc biệt phù hợp với kiến trúc truy xuất hai giai đoạn, trong đó truy xuất chi tiết giai đoạn hai được giới hạn trong phạm vi tài liệu đã được định tuyến, giúp giảm đáng kể nhiễu ngữ cảnh và chi phí suy luận. Cơ sở dữ liệu Qdrant được sử dụng làm kho lưu trữ vector, phục vụ cho các phép truy xuất ngữ nghĩa và truy xuất từ khóa dựa trên biểu diễn vector. Qdrant lưu trữ embedding của các chunk ở cấp độ chi tiết, đồng thời lưu trữ embedding đại diện cho tri thức ở cấp tài liệu. Nhờ đó, Qdrant hỗ trợ trực tiếp cho cả hai giai đoạn truy xuất: giai đoạn định tuyến tài liệu và giai đoạn truy xuất chi tiết trong phạm vi tài liệu đã chọn.

Bên cạnh hai kho lưu trữ tri thức chính, hệ thống sử dụng MongoDB để lưu trữ dữ liệu hội thoại và lịch sử tương tác với người dùng. MongoDB được lựa chọn do khả năng linh hoạt trong việc lưu trữ dữ liệu dạng tài liệu, cũng như hỗ trợ chỉ mục theo thời gian và theo định danh hội thoại. Việc tách dữ liệu hội thoại ra khỏi các kho tri thức giúp hệ thống duy trì sự rõ ràng về mặt kiến trúc và tránh làm nhiễu các thao tác truy xuất tri thức.

Bảng 4.3 Tóm tắt vai trò từng kho lưu trữ

Kho lưu trữ	Lưu trữ	Phục vụ cho
MySQL	Chunks và metadata	Sparse retrieval
Qdrant	Embeddings	Stage-1 và Stage-2
MongoDB	Conversation và lịch sử hội thoại	Interaction

4.5.2 Schema MySQL



Hình 4.4 Schema MySQL sử dụng trong dự án

Trong hệ thống, MySQL được sử dụng để lưu trữ các đơn vị tri thức ở cấp chunk cùng với metadata cần thiết cho truy xuất và hiển thị. Lược đồ dữ liệu được thiết kế xoay quanh các thực thể chính như tài liệu, các đoạn đã được phân tách theo cấu trúc.

Trong quá trình truy xuất, hệ thống thực hiện truy vấn MySQL để lấy các trường thông tin như định danh đoạn, loại dữ liệu, đường dẫn tiêu đề, nội dung văn bản và metadata đi kèm. Các truy vấn này thường kết hợp giữa điều kiện lọc theo định danh tài liệu và

phép tìm kiếm toàn bộ nội dung trên nội dung văn bản, cho phép hệ thống thực hiện truy xuất theo từ khoá một cách hiệu quả.

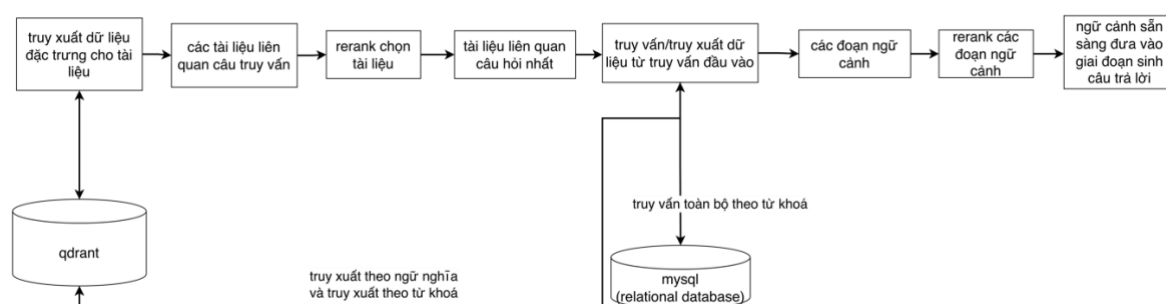
4.5.3 Cấu trúc kho lưu trữ Qdrant

Đối với kho lưu trữ vector, hệ thống tổ chức dữ liệu trong Qdrant thành hai nhóm bộ dữ liệu tương ứng với hai mức độ truy xuất trong pipeline.

Nhóm thứ nhất là bộ dữ liệu lưu trữ các tập biểu diễn tài liệu, bộ dữ liệu này được sử dụng trong giai đoạn truy xuất đầu tiên để xác định tài liệu phù hợp nhất với truy vấn đầu vào. Mỗi phần tử trong bộ dữ liệu lưu trữ vector đại diện cho tài liệu cùng với các metadata định danh tài liệu, chú thích của ảnh, chú thích của bảng, đường dẫn và tiêu đề.

Nhóm thứ hai là bộ dữ liệu lưu trữ các đoạn, trong đó mỗi phần tử tương ứng với một đoạn đã được phân tách theo cấu trúc. Bộ dữ liệu lưu trữ các đoạn này được sử dụng trong giai đoạn truy xuất thứ hai nhằm tìm kiếm các đơn vị tri thức chi tiết nhất trong phạm vi tài liệu đã được chọn khớp với truy vấn đầu vào của pipeline. Metadata đi kèm cho phép hệ thống lọc theo định danh tài liệu, loại dữ liệu.

4.6 Truy xuất dữ liệu



Hình 4.5 Two-stage retrieval pipeline

Truy xuất dữ liệu là bước trung tâm của hệ thống, quyết định trực tiếp chất lượng ngữ cảnh được cung cấp cho mô hình sinh. Nhằm khắc phục các hạn chế của truy xuất một giai đoạn trong các hệ thống RAG truyền thống, hệ thống áp dụng chiến lược truy xuất hai giai đoạn kết hợp với truy xuất lai dense/sparse và các cơ chế kiểm soát nhiễu ở cả trước và sau quá trình sinh câu trả lời. Thiết kế này cho phép thu hẹp không gian tìm kiếm từ sớm, giảm nhiễu ngữ cảnh và tăng độ chính xác của kết quả truy xuất.

4.6.1 Giai đoạn thứ nhất

Giai đoạn đầu tiên của quá trình truy xuất nhằm xác định tài liệu phù hợp nhất với truy vấn đầu vào. Hệ thống gọi hàm truy xuất trên kho vector của tri thức cấp tài liệu để thu được một tập các tài liệu liên quan, nếu có nhiều hơn một tài liệu, hệ thống thực hiện reranking để sắp xếp lại các tài liệu theo mức độ liên quan với truy vấn và chọn tài liệu liên quan nhất.

Kết quả của giai đoạn này là một tài liệu được chọn, kèm theo các định danh cần thiết để thiết lập phạm vi truy xuất cho giai đoạn tiếp theo. Việc tách riêng giai đoạn lựa chọn tài liệu giúp hệ thống tránh phải quét toàn bộ tập các đoạn của mọi tài liệu, từ đó giảm đáng kể không gian tìm kiếm và hạn chế nhiễu ngữ cảnh ở giai đoạn truy xuất chi tiết.

4.6.2 Giai đoạn thứ hai

Sau khi đã xác định được tài liệu phù hợp ở giai đoạn đầu, hệ thống tiến hành truy xuất chi tiết các đơn vị tri thức trong phạm vi tài liệu đó. Ở giai đoạn này, truy xuất được thực hiện trong phạm vi tài liệu đã được chọn trước đó, trong đó cả truy xuất sparse và truy xuất dense đều bị giới hạn bởi định danh tài liệu đã được chọn.

Đối với truy xuất sparse, hệ thống sử dụng cơ sở dữ liệu quan hệ để thực hiện tìm kiếm full-text trên các chunk văn bản song song với tìm kiếm vector theo từ khoá chính xác với bm25, đồng thời tự động áp dụng điều kiện lọc theo định danh tài liệu. Điều này đảm bảo rằng các kết quả truy xuất chỉ đến từ tài liệu đã được chọn ở giai đoạn trước. Song song với đó, truy xuất dense được thực hiện trên kho vector, trong đó các phép tìm kiếm vector cũng được lọc theo metadata của tài liệu tương ứng.

Để kết hợp hai nguồn truy xuất này, hệ thống sử dụng một lớp truy xuất lai được giới hạn phạm vi truy vấn, chịu trách nhiệm hợp nhất kết quả từ truy xuất sparse và dense, loại bỏ các kết quả trùng lặp và áp dụng các điều kiện lọc cần thiết. Cơ chế này cho phép hệ thống tận dụng đồng thời ưu điểm của tìm kiếm theo từ khóa và tìm kiếm ngữ nghĩa, trong khi vẫn giữ phạm vi truy xuất ở mức hẹp và có kiểm soát.

4.6.3 Chiến lược hợp nhất truy xuất dày và truy xuất thưa

Trong pipeline truy xuất thực tế, hệ thống thực hiện quá trình truy xuất lai theo ba bước chính. Trước hết, truy xuất sparse được thực hiện trên cơ sở dữ liệu quan hệ và cơ sở dữ

liệu vector để thu được các chunk có mức độ khớp cao về mặt từ khóa. Tiếp theo, truy xuất dense được thực hiện trên kho vector để tìm các chunk có độ tương đồng ngữ nghĩa cao với truy vấn. Cuối cùng, hai tập kết quả này được hợp nhất thông qua một cơ chế loại bỏ các đoạ trùng lặp.

Trong bước hợp nhất, điểm số từ các nguồn truy xuất khác nhau được chuẩn hóa về cùng một miền giá trị trước khi gộp theo định danh chunk, quá trình chuẩn hoá về cùng miền giá trị này được biểu diễn bởi công thức

$$S_{weighted} = S_{hit} \times w_{source} \quad (4-1)$$

Trong đó s_{hit} là điểm số gốc do mô-đun truy xuất trả về và w_{source} là trọng số tương ứng với nguồn truy xuất. Các kết quả sau khi trọng số hóa được hợp nhất theo định danh đoạn, trong trường hợp một chunk xuất hiện ở nhiều nguồn, hệ thống giữ lại giá trị điểm cao nhất nhằm tránh khuếch đại điểm số không kiểm soát. Sau khi tập kết quả truy xuất ban đầu được hình thành, hệ thống có thể áp dụng bước reranking nhằm sắp xếp lại các chunk theo mức độ liên quan với truy vấn.

4.7 Sinh câu trả lời

Sau khi hoàn tất quá trình truy xuất và xây dựng tập ngữ cảnh liên quan, hệ thống tiến hành sinh câu trả lời dựa trên các đơn vị tri thức đã được lựa chọn. Giai đoạn sinh đóng vai trò kết nối trực tiếp giữa kết quả truy xuất và đầu ra cuối cùng của hệ thống, do đó yêu cầu vừa đảm bảo tính mạch lạc của câu trả lời, vừa duy trì khả năng kiểm chứng thông tin dựa trên các bằng chứng đã được cung cấp.

Một điểm khác biệt quan trọng của hệ thống trong đề án tốt nghiệp này so với các hệ thống RAG text-only là khả năng tích hợp trực tiếp thông tin hình ảnh vào quá trình sinh câu trả lời, thay vì chỉ dựa trên mô tả văn bản của hình ảnh. Khi trong tập ngữ cảnh truy xuất xuất hiện các đơn vị tri thức thuộc loại hình ảnh, hệ thống tiến hành xử lý và tích hợp chúng vào đầu vào của mô hình sinh theo cơ chế đa phương thức.

Cụ thể, đối với mỗi đơn vị tri thức hình ảnh, hệ thống xác định đường dẫn ảnh tương ứng từ metadata đã được lưu trữ trong quá trình tiền xử lý. Hình ảnh sau đó được mã hóa dưới dạng base64 và được đưa vào đầu vào của mô hình sinh dưới dạng một thành

phần hình ảnh, tuân theo chuẩn định dạng phổ biến của các giao diện mô hình đa phương thức. Việc tích hợp này cho phép mô hình sinh tiếp cận trực tiếp dữ liệu hình ảnh của tài liệu, thay vì chỉ suy luận gián tiếp thông qua chú thích hoặc mô tả văn bản.

Nhờ cơ chế này, quá trình sinh câu trả lời trong không chỉ dừng lại ở việc kết hợp văn bản truy xuất, mà còn mở rộng sang việc khai thác trực tiếp các nguồn thông tin thị giác khi cần thiết. Đây là một bước tiến quan trọng trong việc nâng cao khả năng biểu đạt và độ đầy đủ của câu trả lời đối với các truy vấn liên quan đến biểu đồ, sơ đồ hoặc hình minh họa trong tài liệu.

Bảng 4.4 Cấu trúc prompt

Thành phần	Mô tả vai trò	Mục đích trong grounded generation
Chỉ dẫn hệ thống	Định nghĩa vai trò, ngôn ngữ suy luận và yêu cầu trích dẫn bằng chứng	Thiết lập vai trò và hành vi tổng quát của mô hình sinh
Ràng buộc suy luận	Yêu cầu suy luận từng bước dựa trên ngữ cảnh được cung cấp	Ngăn mô hình suy diễn ngoài phạm vi tri thức truy xuất
Ngữ cảnh đầu vào	Tập ngữ cảnh có cấu trúc được trích xuất từ PDF, bao gồm các đoạn văn bản đã được định danh	Cung cấp bằng chứng đầu vào cho quá trình sinh câu trả lời
Ràng buộc các ngữ cảnh đầu vào	Chỉ cho phép sử dụng các phần ngữ cảnh liên quan trực tiếp tới câu hỏi	Giảm nhiễu và tránh lồng ghép thông tin không liên quan
Xử lý khi ngữ cảnh không liên quan	Quy định cách phản hồi khi toàn bộ ngữ cảnh không phù hợp với câu hỏi	Tránh hallucination và trả lời sai trọng tâm
Kiểm soát trùng lặp	Yêu cầu loại bỏ thông tin thừa khỏi phần trả lời chính	Đảm bảo câu trả lời ngắn gọn và tập trung
Quy tắc xử lý bảng	Hướng dẫn tóm tắt hàng hoặc ô liên quan, không bịa số liệu	Đảm bảo tính trung thực khi suy luận từ bảng biểu
Quy tắc xử lý hình ảnh	Quy định mô tả ngắn gọn nội dung hình ảnh và nhắc nguồn	Hạn chế suy đoán thị giác ngoài ngữ cảnh
Quy tắc xử lý công thức	Giữ nguyên ký hiệu toán học, giải thích biến, tự chỉnh LaTeX nếu cần	Bảo toàn ý nghĩa toán học và định dạng học thuật
Ràng buộc định dạng đầu ra	Bắt buộc trả lời bằng Markdown thuần, không dùng code fence	Đảm bảo tính nhất quán và dễ hiển thị trong báo cáo
Quy tắc định dạng công thức	Quy định chặt chẽ cách hiển thị công thức block và inline	Tránh lỗi định dạng và trùng lặp công thức
Tránh bịa thông tin	Cấm lồng ghép thông tin không có trong ngữ cảnh	Tăng độ trung thực (faithfulness) của câu trả lời
Yêu cầu phong cách trả lời	Yêu cầu trả lời ngắn gọn, đầy đủ và nhấn mạnh chi tiết quan trọng	Đảm bảo câu trả lời đúng trọng tâm truy vấn

4.8 Đánh giá hệ thống

Đánh giá hệ thống là bước cần thiết nhằm kiểm chứng mức độ hiệu quả của kiến trúc và các lựa chọn thiết kế đã được trình bày trong các mục trước. Trong dự án này, quá trình đánh giá được thực hiện theo hai hướng hỗ trợ cho nhau đó là đánh giá chất lượng truy xuất tri thức và đánh giá chất lượng câu trả lời được sinh ra. Cách tiếp cận này cho phép phân tách rõ ràng ảnh hưởng của từng giai đoạn trong pipeline RAG, đồng thời phản ánh sát hơn hành vi thực tế của hệ thống trong các kịch bản hỏi đáp dựa trên tài liệu.

Toàn bộ quy trình đánh giá được triển khai thông qua một script độc lập, cho phép nạp tập dữ liệu đánh giá, thực hiện truy xuất, sinh câu trả lời và tổng hợp các chỉ số đánh giá. Thiết kế này giúp đảm bảo tính tái lập của thí nghiệm và cho phép phân tích kết quả theo nhiều góc độ khác nhau.

4.8.1 Bộ chỉ số đánh giá

Đánh giá truy xuất tập trung vào việc đo lường mức độ hệ thống có khả năng đưa vào ngữ cảnh các đơn vị tri thức thực sự liên quan đến câu hỏi hay không. Đối với mỗi truy vấn trong tập đánh giá, hệ thống so sánh tập các đơn vị tri thức được truy xuất với tập bằng chứng tham chiếu (evidence) đã được xác định trước.

Các chỉ số precision và recall được tính toán dựa trên giao giữa tập các đơn vị tri thức được truy xuất và tập bằng chứng tham chiếu. Precision phản ánh tỷ lệ các đơn vị tri thức truy xuất là thực sự liên quan, trong khi recall phản ánh mức độ bao phủ của hệ thống đối với các bằng chứng cần thiết để trả lời câu hỏi.

4.8.2 Sử dụng VLM làm giám khảo

Bên cạnh đánh giá truy xuất, hệ thống được đánh giá về chất lượng câu trả lời thông qua cơ chế sử dụng VLM làm giám khảo hay VLM-as-a-Judge. Thay vì chỉ dựa trên các chỉ số tự động, cách tiếp cận này sử dụng một mô hình ngôn ngữ lớn làm giám khảo để chấm điểm câu trả lời dựa trên nhiều tiêu chí khác nhau, phản ánh sát hơn các yêu cầu trong bài toán hỏi-đáp dựa trên tài liệu.

Đối với mỗi câu hỏi, mô hình giám khảo được yêu cầu trả về một cấu trúc JSON bao gồm các điểm số thành phần và điểm tổng hợp. Các tiêu chí đánh giá bao gồm: độ trung thực của câu trả lời đối với các bằng chứng đã được cung cấp; mức độ đầy đủ của nội

dung trả lời so với câu trả lời tham chiếu; khả năng sử dụng đúng loại modality khi câu hỏi yêu cầu; và mức độ giữ đúng định dạng mong muốn đối với các bảng biểu hoặc công thức toán học.

Trong bối cảnh hệ thống, mỗi tiêu chí mang một ý nghĩa cụ thể. Độ trung thực phản ánh việc câu trả lời có thực sự dựa trên các đơn vị tri thức đã được truy xuất hay không, đặc biệt đối với các thông tin định lượng hoặc bảng số liệu. Độ đầy đủ phản ánh mức độ hệ thống bao quát được các ý chính cần thiết để trả lời câu hỏi. Chỉ số modality coverage đánh giá khả năng hệ thống sử dụng đúng loại nguồn thông tin (ví dụ hình ảnh, bảng biểu hoặc công thức) khi câu hỏi đòi hỏi. Cuối cùng, format fidelity phản ánh mức độ hệ thống giữ được định dạng mong muốn của đầu ra, chẳng hạn như cấu trúc bảng hoặc biểu diễn công thức toán học.

4.8.3 Prompt cho nhiều loại dữ liệu

Kết quả đánh giá cho thấy hiệu năng của hệ thống có sự khác biệt rõ rệt giữa các loại dữ liệu. Đối với nội dung văn bản thuần, hệ thống đạt kết quả tốt nhất cả về truy xuất và sinh câu trả lời, nhờ vào cấu trúc chunking theo tiêu đề và chiến lược truy xuất lai dense/sparse. Đối với bảng biểu, hiệu quả của hệ thống phụ thuộc mạnh vào chất lượng parse bảng và khả năng giữ lại biểu diễn Markdown thô, đặc biệt trong các trường hợp bảng có cấu trúc phức tạp hoặc nhiều cột.

Đối với hình ảnh, kết quả đánh giá cho thấy chất lượng câu trả lời chịu ảnh hưởng đáng kể bởi độ chính xác của chú thích và khả năng hỗ trợ đầu vào đa phương thức của mô hình sinh. Trong các trường hợp mô hình sinh không hỗ trợ trực tiếp hình ảnh, hệ thống phải dựa vào biểu diễn văn bản thay thế, dẫn đến suy giảm chất lượng đối với các câu hỏi yêu cầu thông tin thị giác. Đối với công thức toán học, hiệu quả phụ thuộc vào ngữ cảnh văn bản xung quanh công thức và các bước chuẩn hóa đầu ra, đặc biệt trong các truy vấn yêu cầu biểu diễn chính xác ký hiệu toán học.

Để minh họa rõ hơn về thành công đã đạt được, một số hình ảnh bên dưới trình bày kết quả của pipeline RAG xử lý nhiều loại dữ liệu đã đề ra.

show hình 10 trong bài báo neusym-rag, nói chi tiết nội dung hình 10

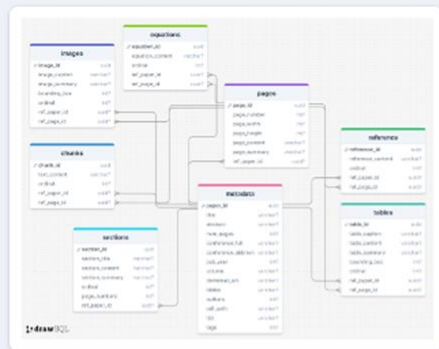
Hình 10 trong bài báo NeuSym-RAG là sơ đồ cơ sở dữ liệu (schema) dùng để lưu trữ các phần tử được phân tích từ mỗi file PDF. Sơ đồ này mô tả mối quan hệ giữa các bảng dữ liệu chính:

- **metadata:** chứa thông tin tổng quan về bài báo (title, abstract, authors, tags, etc.).
- **sections:** lưu trữ các phần (section) trong bài, bao gồm tiêu đề, nội dung, tóm tắt, và trang số.
- **chunks:** lưu trữ các mảnh văn bản (chunk) được chia theo nhiều view.
- **pages:** lưu trữ thông tin về từng trang (số trang, chiều rộng, chiều cao, nội dung trang).
- **images:** lưu trữ thông tin về hình ảnh (caption, bounding box, trang liên quan).
- **tables:** lưu trữ thông tin về bảng (caption, nội dung, tóm tắt, bounding box, trang liên quan).
- **equations:** lưu trữ thông tin về phương trình (nội dung, trang liên quan).
- **reference:** lưu trữ thông tin về tài liệu tham khảo (nội dung, trang liên quan).

Các bảng này được liên kết với nhau thông qua các trường `'ref_paper_id'` và `'ref_page_id'`, cho phép truy vấn và trích xuất thông tin một cách có cấu trúc. Lưu ý: trong sơ đồ, các cột như `'bounding_box'`, `'authors'`, `'tags'`, `'page_numbers'` có kiểu dữ liệu thực tế là `'INT[4]'`, `'VARCHAR[]'`, `'VARCHAR[]'`, `'INT[]'` (do công cụ drawSQL không hiển thị kiểu `'ARRAY'`).

Trích xuất hình ảnh

[1] root > Database Schema and Encodable Columns in Vectorstore



Nguồn trích dẫn

- [1] root > bstract
- [2] root > Supplementary Experiments and Settings
- [3] root > Experiment > Experiment Settings
- [4] root > Experiment > Main Experiment
- [5] root > Conclusion
- [6] root > Introduction
- [7] root > Related Work
- [8] root > Supplementary Experiments and Settings
- [9] root > Experiment > Experiment Settings
- [10] root > or Hybrid-RAG > Task Prompt
- [11] root > Database Schema and Encodable Columns in Vectorstore

Hình 4.6 Output với câu hỏi hình ảnh

công thức rag-sequence model trong bài báo Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks đầy đủ là gì

Công thức của mô hình RAG-Sequence trong bài báo là:

$$p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\theta}(y|x, z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})$$

Giải thích chi tiết:

- $p_{\eta}(z|x)$: xác suất truy xuất tài liệu z từ đầu vào x thông qua retriever.
- $p_{\theta}(y_i|x, z, y_{1:i-1})$: xác suất phát sinh token thứ i từ generator, dựa trên đầu vào x , tài liệu z và các token trước đó $y_{1:i-1}$.
- Tổng hợp qua tất cả K tài liệu top được truy xuất, và tích các xác suất token theo từng vị trí trong chuỗi.
- Mô hình này giả định **cùng một tài liệu được dùng để sinh ra toàn bộ chuỗi**, phù hợp với các tác vụ yêu cầu tính nhất quán trong ngữ cảnh.

Bạn có muốn biết thêm về RAG-Token model không?

 Nguồn trích dẫn

- [1] root > bstract
- [2] root > Methods
- [3] root > Methods > Models
- [4] root > Introduction
- [5] root > Methods > Generator: BART
- [6] root > etr

Ask Briq

Hình 4.7 Output với câu hỏi công thức

hiệu suất của các retrieval options khác nhau của Classic RAG baseline trên tập airqa-real, đưa cho tôi bảng đầy đủ đi

Bảng hiệu suất của các retrieval options khác nhau của Classic RAG baseline trên tập AIRQA-REAL được trình bày trong **Table 4**:

Retrieved Column	text	table	image	formula	Overall (%)
chunk_content	13.95	15.00	22.22	28.57	18
section_content	18.60	10.00	5.56	14.29	12
table_content	13.95	15.00	0.00	9.52	9
image_summary	4.65	0.00	16.67	4.76	6
equation_content	6.98	0.00	5.56	33.33	8

Ghi chú:

- Các cột "text", "table", "image", "formula" thể hiện hiệu suất (%) theo từng loại nội dung.
- Cột "Overall (%)" là tổng hiệu suất trung bình của từng loại retrieval option.
- **chunk_content** đạt hiệu suất cao nhất (18%), trong khi **image_summary** đạt thấp nhất (6%).
- **equation_content** có hiệu suất cao nhất trong nhóm "formula" (33.33%), nhưng thấp nhất trong tổng thể (8%).

Bạn có muốn biết thêm về cách các retrieval option này được thiết kế hay so sánh với NeuSym-RAG không?

Nguồn trích dẫn

- [1] root > Experiment > Ablation Study

Hình 4.8 Output với câu hỏi công thức

CHƯƠNG 5 ĐÁNH GIÁ, KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết các kết quả chính của dự án và đối chiếu trực tiếp với các mục tiêu đã đặt ra. Trên cơ sở các kết quả thực nghiệm, chương phân tích rõ những điểm hệ thống làm tốt, những hạn chế còn tồn tại, đồng thời đề xuất các hướng phát triển theo mức độ ưu tiên và tính khả thi trong bối cảnh triển khai hoàn toàn nội bộ (local-first).

5.1 Tổng kết kết quả đạt được

5.1.1 Xây dựng hệ thống RAG hoàn toàn nội bộ và khả năng triển khai trong môi trường doanh nghiệp

Mục tiêu cốt lõi của dự án là triển khai một hệ thống hỏi đáp dựa trên kiến trúc RAG có thể vận hành hoàn toàn trong hạ tầng nội bộ, không phụ thuộc dịch vụ SaaS hay API bên thứ ba trong quy trình nạp dữ liệu và truy vấn. Ở mức kiến trúc và triển khai, dự án đã hiện thực hoá một pipeline khép kín bao gồm nạp tài liệu, tiền xử lý, lưu trữ tri thức, truy xuất và sinh câu trả lời. Thành phần mô hình sinh được vận hành theo cơ chế local gateway cụ thể là LMStudio, trong khi tri thức và trạng thái hội thoại được quản lý bởi các kho dữ liệu nội bộ gồm MySQL, Qdrant, và MongoDB. Với thiết kế này, toàn bộ vòng đời dữ liệu từ tệp tài liệu đầu vào đến các chỉ mục và ngữ cảnh truy xuất được kiểm soát trong phạm vi hệ thống nội bộ, phù hợp với yêu cầu bảo mật và kiểm soát dữ liệu thường gặp trong môi trường doanh nghiệp.

Điểm quan trọng không chỉ nằm ở việc chạy được local, mà ở tính khả thi triển khai khi hệ thống được đóng gói như một dịch vụ backend có khả năng quản lý trạng thái pipeline, tách biệt rõ tri thức với hội thoại. Điều này giúp giảm độ phức tạp khi tích hợp vào các kênh sử dụng thực tế như intranet, portal nội bộ hoặc trợ lý quy trình.

5.1.2 Hiệu quả với tài liệu có cấu trúc

Trong phạm vi dữ liệu mục tiêu của đề tài, các văn bản hành chính tiếng Việt như thông báo, quy định, quyết định, công văn thì hệ thống thể hiện chất lượng trả lời ổn định. Kết quả này có liên hệ trực tiếp với triết lý data-centric RAG, trong đó trọng tâm tối ưu hoá nằm ở giai đoạn tiền xử lý và tổ chức tri thức, thay vì chỉ dựa vào prompt hoặc năng lực tham số của mô hình. Cụ thể, việc chuyển đổi PDF sang Markdown nhằm bảo toàn cấu trúc tài liệu tạo điều kiện cho chunking theo ngữ nghĩa của đoạn, tiểu mục, đồng thời

cho phép gắn nhãn đường dẫn tiêu đề để truy xuất và trích dẫn. Các đơn vị ngữ nghĩa đầy đủ giúp cả sparse retrieval và dense retrieval hoạt động ổn định hơn.

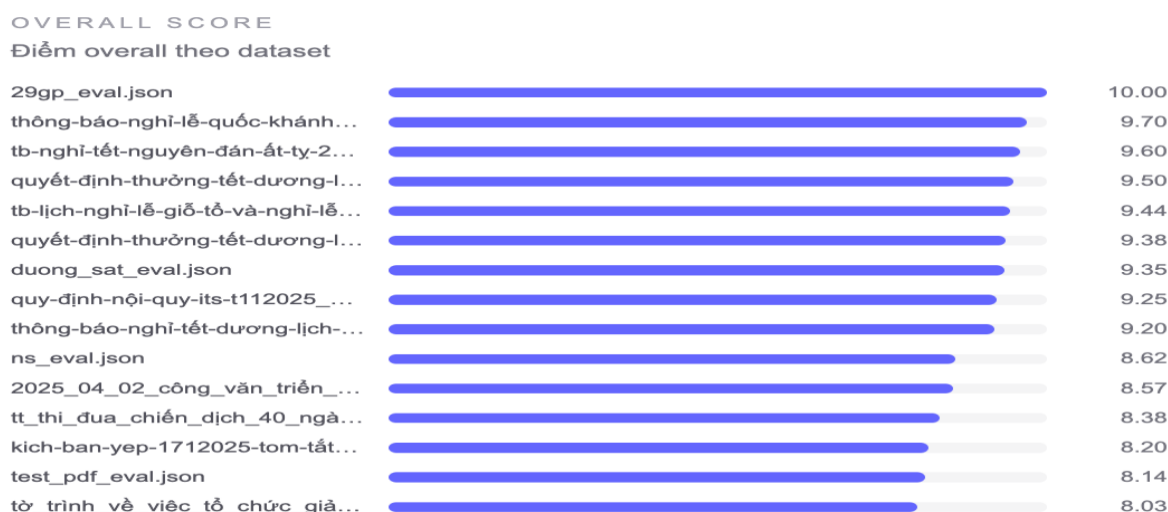
5.1.3 Khả năng hoạt động trên phần cứng được sử dụng

Một đóng góp thực tiễn của dự án là chứng minh hệ thống RAG local-first có thể vận hành mức thử nghiệm nhỏ trên phần cứng tầm trung, ví dụ GPU tầm trung như RTX 3080. Hệ thống được tổ chức theo dạng pipeline: các bước parse/ingest có thể thực hiện offline, trong khi luồng query được tối ưu cho phản hồi theo lượt. API được thiết kế rõ ràng để dễ tích hợp. Thực tế này giúp tăng tính ứng dụng của hệ thống: một bộ phận nhỏ trong doanh nghiệp có thể thử nghiệm và triển khai trong phạm vi kiểm soát được trước khi mở rộng quy mô.

5.2 Đánh giá kết quả thực nghiệm

Phần này tổng hợp kết quả đo lường theo từng dataset nhằm phản ánh khách quan hiệu quả của hệ thống ở hai mức là hiệu quả truy xuất thông qua Recall và Precision và chất lượng end-to-end thông qua điểm Overall theo bộ tiêu chí chấm (Faithfulness, Completeness, Modality coverage, Format fidelity). Các kết quả được tổng hợp theo dataset vì mỗi dataset đại diện cho một tài liệu cụ thể, qua đó phản ánh mức độ ổn định của hệ thống trên các nhóm tài liệu khác nhau.

5.2.1 Kết quả chất lượng end-to-end



Hình 5.1 Kết quả chất lượng end-to-end (overall)

Theo số liệu thực nghiệm, điểm Overall dao động từ 8.03 đến 10.00, cho thấy hệ thống duy trì chất lượng cao trên phần lớn dataset. Nhiều bộ dữ liệu đạt mức gần tối đa, tiêu biểu như 29gp_eval.json đạt 10.00, thông-báo-nghi-lễ-quốc-khánh-292025_eval.json đạt 9.70, tb-nghi-tết-nguyên-đán-ất-tỵ-2025_eval.json đạt 9.60 và quyết-định-thường-tết-dương-lịch-2026_eval.json đạt 9.50. Ngay cả các dataset có mức điểm thấp hơn tương đối vẫn duy trì trên 8.0, chẳng hạn tờ_trình_về_việc_tổ_chức_giải_billiards... đạt 8.03 và test_pdf_eval.json đạt 8.14. Mức phân bố này gợi ý rằng hệ thống không chỉ có một vài trường hợp điểm cao bất thường, mà có tính ổn định tương đối trên tập dữ liệu đánh giá hiện có.

5.2.2 Kết quả chất lượng retrieval

RETRIEVAL

Recall vs Precision theo dataset



Hình 5.2 Recall và Precision theo từng dataset

Về truy xuất, Recall thể hiện xu hướng rất cao khi đa số dataset đạt Recall = 1.000, một số dataset khác đạt mức 0.967 hoặc 0.950, và trường hợp thấp hơn đáng chú ý là ns_eval.json chỉ đạt 0.783. Nhìn tổng thể, Recall cao cho thấy hệ thống có khả năng không bỏ sót bằng chứng cần thiết trong phần lớn truy vấn thuộc tập đánh giá, đây là thuộc tính quan trọng đối với RAG, vì thiếu bằng chứng thường dẫn đến suy giảm độ đầy đủ và dễ phát sinh suy đoán.

Precision quan sát được ở mức trung bình và phân hoá theo dataset, một nhóm lớn có Precision khoảng 0.50, trong khi một số dataset nằm ở vùng 0.13-0.17, và có trường hợp đạt 1.00. Cách hiểu hợp lý cho hiện tượng này là hệ thống đang vận hành theo chiến lược retrieval thiên về coverage-oriented ưu tiên đảm bảo có đủ bằng chứng đúng trong top-K hơn là tối thiểu hoá tuyệt đối số chunk dư. Với tài liệu doanh nghiệp, nhiều văn bản có cấu trúc tương tự và trùng lặp thuật ngữ thì chiến lược này giúp giảm rủi ro bỏ sót điều khoản then chốt, đặc biệt khi người dùng hỏi theo cách diễn đạt không trùng từ khoá.

5.2.3 Precision trung bình nhưng Overall vẫn cao



Hình 5.3 Quan hệ trade-off giữa Recall, Precision và Overall với Recall (x), Overall (y) và Precision (màu)

Kết hợp quan sát từ Hình 5.1 và Hình 5.2 cho thấy một đặc điểm quan trọng là mặc dù Precision không luôn cao, điểm Overall vẫn duy trì ở mức cao trên nhiều dataset. Điều này gợi ý rằng lượng ngữ cảnh dư ở bước retrieval chưa trở thành yếu tố chi phối làm giảm chất lượng end-to-end trong tập đánh giá hiện tại. Cơ chế sau truy xuất đã giúp

giảm nhiễu và giữ câu trả lời bám bằng chứng, việc truy xuất theo phạm vi tài liệu, kết hợp dense và sparse để tăng tính kháng nhiễu và sắp xếp lại thứ tự ngữ cảnh đã giúp mô hình tập trung được ngữ cảnh.

Trong bối cảnh sử dụng thực tế, Precision trung bình không nhất thiết là dấu hiệu cho thấy hệ thống yếu, nếu mục tiêu là hỗ trợ người dùng tra cứu trong tài liệu doanh nghiệp. Một lượng ngữ cảnh liên quan bổ sung có thể mang lại lợi ích như giúp người dùng thấy thêm các điều khoản gần kề, tăng khả năng gợi ý thông tin liên quan, và giảm rủi ro bỏ sót chi tiết quan trọng khi truy vấn mơ hồ. Vì vậy, với việc Overall duy trì cao, có thể xem cấu hình truy xuất hiện tại đang đạt một điểm cân bằng hợp lý theo hướng ưu tiên độ bao phủ và tính an toàn thông tin.

HEATMAP

Tổng hợp toàn bộ chỉ số

Dataset	Recall	Precision	Faithfulness	Completeness	Modality	Format	Overall
29gp_eval.json	1.000	0.275	5.000	5.000	5.000	5.000	10.00
thông-báo-ngहि-lễ-quốc-khánh-292025_eval.json	1.000	0.500	5.000	4.700	5.000	5.000	9.70
tb-ngहि-tết-nguyên-đán-ất-tý-2025_eval.json	1.000	0.500	5.000	4.600	5.000	5.000	9.60
quyết-định-thường-tết-duơng-lich-2026_eval.json	1.000	0.500	4.800	4.700	5.000	5.000	9.50
tb-lich-ngहि-lễ-giỗ-tổ-và-ngहि-lễ-304_eval.json	1.000	0.500	5.000	4.900	5.000	4.900	9.44
quyết-định-thường-tết-duơng-lich-2025_eval.json	1.000	0.500	4.900	4.900	5.000	4.800	9.38
duong_sat_eval.json	1.000	0.167	4.833	4.867	5.000	4.900	9.35
quy-định-nội-quy-its-t112025_eval.json	0.967	0.161	4.700	4.650	5.000	4.783	9.25
thông-báo-ngहि-tết-duơng-lich-2026_eval.json	1.000	0.500	4.800	4.600	5.000	4.500	9.20
ns_eval.json	0.783	0.131	4.350	4.417	5.000	4.983	8.62
2025_04_02_công_văn_triển_khai_kinh_doanh_chính_thức_dịch_eval.json	0.950	0.158	4.225	4.100	5.000	4.825	8.57
tt_thi_dua_chiến_dịch_40_ngày_tăng_trưởng_game_eval.json	1.000	0.167	4.600	4.550	5.000	4.800	8.38
kich-ban-yep-1712025-tom-tắt_eval.json	1.000	1.000	4.400	4.200	4.800	5.000	8.20
test_pdf_eval.json	0.967	0.161	4.433	4.433	5.000	4.933	8.14
tờ_trình_về_việc_tổ_chức_giải_billiards_cho_nhân_viên_its_eval.json	1.000	0.167	4.167	4.200	4.833	4.533	8.03

Hình 5.4 Heatmap tổng hợp các chỉ số đánh giá theo dataset

5.3 Hạn chế hiện tại

Mặc dù kết quả định lượng cho thấy hệ thống đạt chất lượng cao ở mức end-to-end, vẫn tồn tại các hạn chế đáng kể, chủ yếu đến từ độ khó vốn có của dữ liệu đa phương thức và thách thức tiền xử lý trong môi trường local.

5.3.1 Hạn chế đối với hình ảnh

Trong pipeline multimodal, hình ảnh chỉ thực sự hữu ích khi ảnh được trích xuất đúng từ PDF, tiêu đề hay biểu diễn vector tốt, đủ phân biệt để truy xuất đúng và mô hình sinh có thể sử dụng ảnh/hoặc mô tả ảnh làm bằng chứng. Trên thực tế, các sơ đồ vector, biểu đồ phức tạp hoặc hình có chữ nhỏ thường là nhóm khó nhất và nếu tiêu đề chỉ mang tính tổng quát hoặc không bắt được tiêu đề thì truy xuất sẽ kém ổn định. Trong môi trường local, việc sử dụng các mô hình tạo chú thích yếu có thể khiến mô tả thiếu chi tiết, từ đó ảnh hưởng trực tiếp đến chất lượng trả lời cho các câu hỏi phụ thuộc thông tin thị giác.

5.3.2 Bảng lớn và cấu trúc phức tạp

Bảng trong PDF tạo ra thách thức kép. Một mặt, bảng lớn tiêu tốn token rất nhanh nếu đưa nguyên vào ngữ cảnh, làm giảm lượng token để sử dụng cho các ngữ cảnh khác. Mặt khác, nếu cắt nhỏ bảng để giảm token, hệ thống có nguy cơ mất quan hệ tiêu đề và dữ liệu trong bảng hoặc mất tính toàn vẹn của phép so sánh theo cột. Ngoài ra, các bảng có các ô kết hợp, cấu trúc đa cột hoặc bảng lồng bảng thường bị lỗi khi chuyển sang Markdown, dẫn đến hiện tượng lỗi lan truyền từ tiền xử lý sẽ kéo dài sang truy xuất và sinh kết quả, khiến giảm faithfulness và format fidelity đối với các câu hỏi dựa trên số liệu bảng.

5.3.3 Công thức toán học

Công thức có tính phụ thuộc ngữ cảnh cao, các ký hiệu và biến số chỉ có ý nghĩa khi gắn với đoạn văn định nghĩa và điều kiện áp dụng. Nếu công thức bị chuyển đổi sai hoặc bị tách khỏi phần giải thích, truy xuất có thể gom các công thức rời rạc lại nhưng không đủ ngữ cảnh để mô hình sinh diễn giải chính xác. Vì vậy, chất lượng câu trả lời liên quan công thức thường tốt hơn khi hệ thống đồng thời truy xuất được thông tin lân cận.

5.3.4 Giới hạn của đánh giá nội bộ

Mặc dù dự án đã xây dựng được pipeline đánh giá, tập đánh giá hiện tại vẫn mang tính nội bộ và có thể chưa bao phủ hết sự đa dạng của tình huống thực tế. Ngoài ra, cơ chế chấm bằng VLM phụ thuộc vào prompt và mô hình đánh giá, do đó cần mô tả tiêu chí chấm rõ ràng, đồng thời có thể bổ sung kiểm tra chéo để giảm thiên lệch. Việc thừa nhận

giới hạn này giúp đảm bảo tính trung thực khoa học của báo cáo và tránh diễn giải quá mức.

5.4 Hướng phát triển

Các hướng phát triển được đề xuất theo ba trục ưu tiên: mở rộng kênh tương tác và định dạng dữ liệu, tăng năng lực truy xuất cho dữ liệu đa phương thức, hoàn thiện các yêu cầu vận hành khi triển khai doanh nghiệp.

5.4.1 Mở rộng kênh tương tác giọng nói

Một hướng phát triển có tính ứng dụng cao là bổ sung truy vấn giọng nói thông qua mô-đun STT chạy nội bộ, ví dụ Whisper hoặc mô hình STT tương đương. Khi tích hợp, âm thanh sẽ được chuyển sang văn bản trước khi đi vào pipeline có. Giá trị thực tiễn nằm ở việc giảm rào cản nhập liệu. Thách thức chính là chất lượng nhận dạng tiếng Việt (dấu và tên riêng), cơ chế sửa lỗi STT, và tối ưu độ trễ để không làm giảm trải nghiệm hội thoại.

5.4.2 Hỗ trợ thêm định dạng tài liệu đầu vào

Trong môi trường doanh nghiệp, tri thức không chỉ nằm trong PDF. Hướng phát triển hợp lý là mở rộng ingestion adapters cho các định dạng phổ biến như DOCX, HTML hoặc TXT có thể chuyển đổi trực tiếp trực tiếp, với Excel cần chuyển sheet thành bảng chuẩn, với PowerPoint cần trích xuất tiêu đề, danh sách và hình ảnh theo slide, với ảnh cần OCR kết hợp tạo chú thích. Lợi thế của thiết kế multimodal là khi các thành phần mới được thêm vào, đầu ra vẫn quy về các loại dữ liệu thống nhất, giúp pipeline không phải thay đổi lớn.

5.4.3 Tối ưu cho các loại dữ liệu

Để giải quyết đồng thời vấn đề giới hạn token và độ khó multimodal, có thể phát triển cơ chế nén thích ứng theo modality. Với bảng lớn, thay vì đưa nguyên bảng vào context, hệ thống có thể tạo lớp tóm tắt cấu và chỉ truy xuất theo hàng, cột liên quan khi truy vấn yêu cầu chi tiết. Với hình ảnh, có thể kết hợp mô hình OCR mạnh hơn và tạo tiêu đề chi tiết để tăng khả năng truy xuất đúng. Với công thức, cần ràng buộc công thức với đoạn văn định nghĩa công thức và tạo mô tả theo ngôn ngữ tự nhiên để cải thiện truy xuất theo câu hỏi dạng nghĩa không cụ thể.

5.4.4 Hoàn thiện yêu cầu đáp ứng triển khai doanh nghiệp

Để tiến gần hơn tới triển khai thực tế, hệ thống cần bổ sung các năng lực vận hành gồm xác thực và phân quyền tài liệu theo người dùng, phòng ban, ghi lại chi tiết log cho truy vấn và tài liệu được trích dẫn, giám sát độ trễ theo từng pha, và chính sách vòng đời dữ liệu. Nhóm cải tiến này không trực tiếp làm tăng điểm benchmark nhưng quyết định khả năng đưa hệ thống vào môi trường vận hành lâu dài.

5.5 Kết luận chung

Dự án đã xây dựng thành công một hệ thống RAG vận hành hoàn toàn nội bộ, phù hợp với yêu cầu bảo mật và kiểm soát dữ liệu của môi trường doanh nghiệp. Kết quả đánh giá cho thấy chất lượng end-to-end duy trì ở mức cao trên nhiều bộ dữ liệu khi Overall từ 8.03 đến 10.00, trong khi Recall ở mức rất cao và ổn định. Precision ở mức trung bình phản ánh chiến lược truy xuất ưu tiên bao phủ bằng chứng, trong bối cảnh Overall vẫn cao, đặc trưng này có thể xem là lựa chọn hợp lý cho kịch bản tra cứu tài liệu doanh nghiệp, nơi ưu tiên tính đầy đủ và an toàn thông tin. Bên cạnh các kết quả tích cực, hệ thống vẫn còn hạn chế đáng kể đối với các loại dữ liệu khó như hình ảnh, bảng phức tạp hay công thức, chủ yếu do thách thức tiền xử lý.

TÀI LIỆU THAM KHẢO

- [1] OpenAI, "gpt-4," OpenAI, 3 2023. [Online]. Available: <https://platform.openai.com/docs/models/gpt-4>. [Accessed 26 10 2025].
- [2] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni and P. Liang, "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, p. 157–173, 2023.
- [3] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, D. Chen, W. Dai, H. S. Chan, A. Madotto and P. Fung, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 248, pp. 1-38, 2022.
- [4] G. Billiris, A. Gill and M. Bandara, "Privacy in the Age of AI: A Taxonomy of Data Risks," arXiv , 2025.
- [5] N. Soveizi, F. Turkmen and D. Karastoyanova, "Security and privacy concerns in cloud-based scientific and business workflows: A systematic review," *Future Generation Computer Systems*, vol. 148, pp. 184-200, 2023.
- [6] Viettel, "Cáp quang," Viettel, 2025. [Online]. Available: <https://netviettel.vn/tin-tuc-dut-cap-quang/>. [Accessed 2025].
- [7] VnExpress, "Khoa học công nghệ," FPT, 2025. [Online]. Available: <https://vnexpress.net/cap-quang-bien-xuong-song-mong-manh-cua-ai-va-internet-toan-cau-4962318.html>. [Accessed 2025].
- [8] Y. Huang and J. Huang, "A Survey on Retrieval-Augmented Text Generation for Large Language Models," arXiv, 2024.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, "Attention Is All You Need," in *Neural Information Processing Systems (NeurIPS)* , 2017 .
- [10] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child and Ra, "Language Models are Few-Shot Learners," in *Neural Information Processing Systems (NeurIPS)*, 2020.
- [11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Neural Information Processing Systems (NeurIPS)*, 2022.
- [12] J. Lu, D. Batra, D. Parikh and S. Lee, "ViLBERT: Pretraining Task-Agnostic Visiolinguistic Representations for Vision-and-Language Tasks," in *Neural Information Processing Systems (NeurIPS)*, 2019.
- [13] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger and I. Sutskever, "Learning Transferable Visual Models From Natural Language Supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.
- [14] OpenAI, "GPT-4 Technical Report," arXiv, 2023.
- [15] Y. Wang, S. Si, D. Li, M. Lukasik, F. Yu, C.-J. Hsieh, I. S. Dhillon and S. Kumar, "Two-stage LLM Fine-tuning with Less Specialization and More Generalization," in *International Conference on Learning Representations (ICLR)*, 2024.

- [16] J. Maynez, S. Narayan, B. Bohnet and R. McDonald, "On Faithfulness and Factuality in Abstractive Summarization," in *The 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [18] J. Donahue, J.-B. Alayrac, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong and S. Samangooei, "Flamingo: a Visual Language Model for Few-Shot Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [19] A. M. Mohsin, M. Umer, A. Bilal, Z. Memon, M. I. Qadir, S. Bhattacharya, H. Rizwan, A. R. Gorle, M. Z. Kazmi, A. Mohsin, M. U. Rafique, Z. He, P. Mehta and M. Jamshed, "On the Fundamental Limits of LLMs at Scale," arXiv, 2025.
- [20] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *ArXiv*, vol. abs/2312.10997, 2023.
- [21] T. Chen, H. Wang, S. Chen, W. Yu, K. Ma, X. Zhao, H. Zhang and D. Yu, "Dense X Retrieval: What Retrieval Granularity Should We Use?," in *The 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [22] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *NAACL-HLT*, 2019.
- [23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *EMNLP-IJCNLP*, 2019.
- [24] J. Lin, R. Nogueira and A. Yates, "Pretrained Transformers for Text Ranking: BERT and Beyond," in *Foundations and Trends® in Information Retrieval*, 2020.
- [25] S. E. Robertson and H. Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, Now Publishers Inc, 2009.
- [26] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen and W.-t. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [27] G. Salton and C. Buckley, "Term Weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513 - 523, 1988.
- [28] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi and L. Zettlemoyer, "Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.
- [29] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith and M. Lewis, "Measuring and Narrowing the Compositionality Gap in Language Models," in *Findings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [30] W. Li, W. Wu, M. Chen, J. Liu, X. Xiao and H. Wu, "Faithfulness in Natural Language Generation: A Systematic Survey of Analysis, Evaluation and Optimization Methods," arXiv, 2022.
- [31] S. Barnett, S. Kurniawan, S. Thudumu, Z. Brannelly and M. Abdelrazek, "Seven Failure Points When Engineering a Retrieval Augmented Generation System," in *International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 2024.

- [32] S. Gupta, R. Ranjan and S. N. Singh, "A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions," arXiv, 2024.
- [33] L. Mei, S. Mo, Z. Yang and C. Chen, "A Survey of Multimodal Retrieval-Augmented Generation," arXiv, 2025.
- [34] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li, S. Yan, G. Dai, X.-P. Zhang, Y. Dong and Y. Wang, "A Survey on Efficient Inference for Large Language Models," arXiv, 2024.
- [35] A. Mekala, A. Atmakuru, Y. Song, M. Karpinska and M. Iyyer, "Does quantization affect models' performance on long-context tasks?," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.
- [36] R. Zhen, J. Li, Y. Ji, Z. Yang, T. Liu, Q. Xia, X. Duan, Z. Wang, B. Huai and M. Zhang, "Taming the Titans: A Survey of Efficient LLM Inference Serving," in *International Conference on Natural Language Generation (INLG)*, 2025.
- [37] M. M. Abootorabi, A. Zobeiri, M. Dehghani, M. Mohammadkhani, B. Mohammadi, O. Ghahroodi, M. S. Baghshah and E. Asgari, "Ask in Any Modality: A Comprehensive Survey on Multimodal Retrieval-Augmented Generation," in *Findings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [38] S. Gao, S. Zhao, X. Jiang, L. Duan, Y. X. Chng, Q.-G. Chen, W. Luo, K. Zhang, J.-W. Bian and M. Gong, "Scaling Beyond Context: A Survey of Multimodal Retrieval-Augmented Generation for Document Understanding," arXiv, 2025.
- [39] R. Ren, J. Ma and Z. Zheng, "Large language model for interpreting research policy using adaptive two-stage retrieval augmented fine-tuning method," *Expert Systems with Applications*, vol. 278, 2025.
- [40] X. Zhu, Y. Xie, Y. Liu, Y. Li and W. Hu, "Knowledge Graph-Guided Retrieval Augmented Generation," in *NAACL-HLT*, 2025.
- [41] datalab-to, "marker," 2024. [Online]. Available: <https://github.com/datalab-to/marker>. [Accessed 2026].
- [42] Qdrant, "Vector Database Benchmarks," 2024. [Online]. Available: <https://qdrant.tech/benchmarks/>. [Accessed 2026].
- [43] Qdrant, "Indexing," [Online]. Available: <https://qdrant.tech/documentation/concepts/indexing/>. [Accessed 2026].
- [44] Qdrant, "Filtering," [Online]. Available: <https://qdrant.tech/documentation/concepts/filtering/>. [Accessed 2026].